C

Linux C
函数库参考手册

字符测试 · 数据转换 · 内存配置 · 时间与字符串
数学计算 · 用户与组 · 数据结构 · 进程与文件
信号 · 管道 · Socket · IPC · 正则表达式

共 25 章　400+ 个函数
每条含：头文件 · 函数原型 · 说明 · 返回值 · 示例代码
矢量文字版，可复制、可检索

重制清晰版 · 依据原书结构重新编排

关于本手册

本书按原书 25 章的结构重新编排，收录 395 个 Linux C 常用库函数，并补充常用错误码、编译选项等资料作为附录。全部内容以矢量文字重新排版，可直接复制、检索与打印。

条目结构

- **头文件**：使用该函数前需要包含的头文件
- **原型**：函数的声明形式
- **说明**：参数含义与使用方法
- **返回值**：成功与失败时的返回值
- **附加说明**：可移植性、安全性等注意事项
- **相关函数**：可对照查阅的其它函数
- **示例代码**：可直接编译运行的最小示例

阅读约定

- 函数名按原书顺序（大致依字母序）排列，正文可依目录或书签跳转
- 示例代码为便于阅读做了精简，省略了部分返回值检查
- 使用数学函数需在编译时加 `-lm`，动态加载函数需加 `-ldl`，线程函数需加 `-lpthread`
- 需要 root 权限或特定系统环境的函数已在条目中注明

> 建议阅读方式：先按章节了解某一类函数的整体用法，再用左侧书签或 PDF 搜索功能（Ctrl/Cmd + F）按函数名直接定位。

目　录

第1章 字符测试函数

isalnum 测试字符是否为英文字母或数字 10
isalpha 测试字符是否为英文字母 10
isascii 测试字符是否为 ASCII 码字符 11
isblank 测试字符是否为空格字符 12
iscntrl 测试字符是否为 ASCII 码的控制字符 11
isdigit 测试字符是否为阿拉伯数字 10
isgraph 测试字符是否为可打印字符 13
islower 测试字符是否为小写英文字母 10
isprint 测试字符是否为可打印字符 14
isspace 测试字符是否为空格字符 15
ispunct 测试字符是否为标点符号或特殊符号 15
isupper 测试字符是否为大写英文字母 14
isxdigit 测试字符是否为 16 进制数字 16

第2章 数据转换函数

atof 将字符串转换成浮点型数 17
atoi 将字符串转换成整型数 17
atol 将字符串转换成长整型数 17
ecvt 将浮点型数转换成字符串，取四舍五入 19
fcvt 将浮点型数转换为字符串，取四舍五入 19
gcvt 将浮点型数转换为字符串，取四舍五入 20
strtod 将字符串转换成浮点型数 17
strtol 将字符串转换成长整型数 18
strtoul 将字符串转换成无符号长整型数 22
toascii 将整型数转换成合法的 ASCII 码字符 23
tolower 将大写字母转换成小写字母 23
toupper 将小写字母转换成大写字母 23

第3章 内存配置函数

alloca 配置内存空间 25
brk 改变数据字节的范围 26
calloc 配置内存空间 25
free 释放原先配置的内存 25
getpagesize 取得内存分页大小 28
malloc 配置内存空间 25
mmap 建立内存映射 30
munmap 解除内存映射 30
realloc 更改已配置的内存空间 28
sbrk 增加程序可用的数据空间 26

第4章 时间函数

asctime 将时间和日期以字符串格式表示 33
clock 取得进程占用 CPU 的大约时间 34
ctime 将时间和日期以字符串格式表示 33
difftime 计算时间差距 35
ftime 取得目前的时间和日期 35
gettimeofday 取得目前的时间 35
gmtime 取得目前的时间和日期 33
localtime 取得当地目前的时间和日期 33
mktime 将时间结构数据转换成经过的秒数 38
settimeofday 设置目前的时间 36
strftime 格式化日期和时间 33
time 取得目前的时间 34
tzset 设置时区以供时间转换 41

第5章 字符串处理函数

bcmp 比较内存内容 42
bcopy 拷贝内存内容 42
bzero 将一段内存内容全清为零 42
ffs 在一整型数中查找第一个值为真的位 44
index 查找字符串中第一个出现的指定字符 44
memccpy 拷贝内存内容 45
memchr 在某一内存范围中查找一特定字符 45
memcmp 比较内存内容 42
memcpy 拷贝内存内容 43
memfrob 对内存区域编码 47
memmove 拷贝内存内容 43
memset 将一段内存空间填入某值 43
rindex 查找字符串中最后一个出现的指定字符 44
strcasecmp 忽略大小写比较字符串 49
strcat 连接两字符串 49
strchr 查找字符串中第一个出现的指定字符 44
strcmp 比较字符串 46
strcoll 采用目前区域的字符排列次序来比较字符串 51
strcpy 拷贝字符串 51
strcspn 返回字符串中连续不含指定字符串内容的字符数 52
strdup 复制字符串 52
strfry 随机重组字符串内的字符 47
strlen 返回字符串长度 53
strncasecmp 忽略大小写比较字符串 49
strncat 连接两字符串 49
strncmp 比较字符串 46
strncpy 拷贝字符串 51
strpbrk 查找字符串中第一个出现的指定字符 56
strrchr 查找字符串中最后一个出现的指定字符 48
strspn 返回字符串中连续含指定字符串内容的字符数 52
strstr 在一字符串中查找指定的字符串 57
strtok 分割字符串 58

第6章 数学计算函数

abs 计算整型数的绝对值 59
acos 取反余弦函数值 59
asin 取反正弦函数值 59
atan 取反正切函数值 59
atan2 取得反正切函数值 60
ceil 取不小于参数的最小整型数 61
cos 取余弦函数值 62
cosh 取双曲线余弦函数值 62
div 取得两整型数相除后的商及余数 63
exp 计算指数 63
fabs 计算浮点型数的绝对值 64
frexp 将浮点型数分为底数与指数 64
hypot 计算直角三角形斜边长 65
labs 计算长整型数的绝对值 59
ldexp 计算 2 的次方值 64
ldiv 取得两长整数相除后的商及余数 66
log 计算以 e 为底的对数值 63
log10 计算以 10 为底的对数值 67
modf 将浮点型数分解成整数与小数 68
pow 计算次方值 68
sin 取正弦函数值 62
sinh 取双曲线正弦函数值 62
sqrt 计算平方根值 65
tan 取正切函数值 60
tanh 取双曲线正切函数值 71

第7章 用户和组函数

cuserid 取得用户帐号名称 72
endgrent 关闭组文件 73
endpwent 关闭密码文件 74
endutent 关闭 utmp 文件 74
fgetgrent 从指定的文件来读取组格式 75
fgetpwent 从指定的文件来读取密码格式 76
getegid 取得有效的组识别码 77
geteuid 取得有效的用户识别码 77
getgid 取得真实的组识别码 77
getgrent 从组文件中取得帐号的数据 73
getgrgid 从组文件中取得指定 gid 的数据 80
getgrnam 从组文件中取得指定组的数据 80
getgroups 取得组代码 82
getlogin 取得登录的用户帐号名称 72
getpw 取得指定用户的密码文件数据 83
getpwent 从密码文件中取得帐号的数据 74
getpwnam 从密码文件中取得指定帐号的数据 82
getpwuid 从密码文件中取得指定 uid 的数据 83
getuid 取得真实的用户识别码 77
getutent 从 utmp 文件中取得帐号登录数据 74
getutid 从 utmp 文件中查找特定的记录 88
getutline 从 utmp 文件中查找特定的记录 88
initgroups 初始化组清单 89
logwtmp 将一登录数据记录到 wtmp 文件 89
pututline 将 utmp 记录写入文件 90
setegid 设置有效的组识别码 77
seteuid 设置有效的用户识别码 91
setfsgid 设置文件系统的组识别码 92
setfsuid 设置文件系统的用户识别码 92
setgid 设置真实的组识别码 89
setgrent 从头读取组文件中的组数据 79
setgroups 设置组代码 82
setpwent 从头读取密码文件中的帐号数据 84
setregid 设置真实及有效的组识别码 91
setreuid 设置真实及有效的用户识别码 91
setuid 设置真实的用户识别码 77
setutent 从头读取 utmp 文件中的登录数据 87
updwtmp 将一登录数据记录到 wtmp 文件 89
utmpname 设置 utmp 文件路径 100

第8章 数据加密函数

crypt 将密码或数据编码 101
getpass 取得一密码输入 102

第9章 数据结构函数

bsearch 二元搜索 103
hcreate 建立哈希表 104
hdestroy 删除哈希表 104
hsearch 哈希表搜索 104
insque 加入一项目至队列中 106
lfind 线性搜索 103
lsearch 线性搜索 107
qsort 利用快速排序法排列数组 103
remque 从队列中删除一项目 110
tdelete 从二叉树中删除数据 111
tfind 搜索二叉树 111
tsearch 二叉树 111
twalk 走访二叉树 114

第10章 随机数函数

drand48 产生一个正的浮点型随机数 115
erand48 产生一个正的浮点型随机数 115
initstate 建立随机数状态数组 116
jrand48 产生一个长整型数随机数 116
lcong48 设置 48 位运算的随机数种子 117
lrand48 产生一个正的长整型随机数 115
mrand48 产生一个长整型随机数 116
nrand48 产生一个正的长整型随机数 115
rand 产生随机数 115
random 产生随机数 115
seed48 设置 48 位运算的随机数种子 120
setstate 建立随机数状态数组 116
srand 设置随机数种子 53
srand48 设置 48 位运算的随机数种子 115
srandom 设置随机数种子 120

第11章 初级 I/O 函数

close 关闭文件 123
creat 建立文件 123
dup 复制文件描述词 125
dup2 复制文件描述词 125
fcntl 文件描述词操作 126
flock 锁定文件或解除锁定 128
fsync 将缓冲区数据写回磁盘 129
lseek 移动文件的读写位置 130
mkstemp 建立唯一的临时文件 131
open 打开文件 30
read 由已打开的文件读取数据 130
sync 将缓冲区数据写回磁盘 129
write 将数据写入已打开的文件内 30

第12章 标准 I/O 函数

clearerr 清除文件流的错误旗标 135
fclose 关闭文件 135
fdopen 将文件描述词转为文件指针 136
feof 检查文件流是否读到了文件尾 135
fflush 更新缓冲区 137
fgetc 由文件中读取一个字符 137
fgetpos 取得文件流的读取位置 138
fgets 由文件中读取一字符串 138
fileno 返回文件流所使用的文件描述词 136
fopen 打开文件 135
fputc 将一指定字符写入文件流中 140
fputs 将一指定的字符串写入文件内 138
fread 从文件流读取数据 141
freopen 打开文件 139
fseek 移动文件流的读写位置 138
fsetpos 移动文件流的读写位置 138
ftell 取得文件流的读取位置 138
fwrite 将数据写至文件流 141
getc 由文件中读取一个字符 137
getchar 由标准输入设备内读进一字符 137
gets 由标准输入设备内读进一字符串 138
mktemp 产生唯一的临时文件文件名 131
putc 将一指定字符写入文件中 140
putchar 将指定的字符写到标准输出设备 140
puts 将指定的字符串写到标准输出设备 140
rewind 重设文件流的读写位置为文件开头 142
setbuf 设置文件流的缓冲区 148
setbuffer 设置文件流的缓冲区 148
setlinebuf 设置文件流为线性缓冲区 149
setvbuf 设置文件流的缓冲区 148
tmpfile 建立临时文件 131
ungetc 将一指定字符写回文件流中 151

第13章 进程及流程控制

abort 以异常方式结束进程 152
assert 若测试的条件不成立则终止进程 152
atexit 设置程序正常结束前调用的函数 152
execl 执行文件 155
execle 执行文件 155
execlp 从 PATH 环境变量中查找文件并执行 155
execv 执行文件 155
execve 执行文件 155
execvp 执行文件 156
exit 正常结束进程 152
_exit 结束进程执行 158
fork 建立一个新的进程 158
getpgid 取得进程组识别码 160
getpgrp 取得进程组识别码 160
getpid 取得进程识别码 161
getppid 取得父进程的进程识别码 161
getpriority 取得程序进程执行优先权 162
longjmp 跳转到原先 setjmp 保存的堆栈环境 163
nice 改变进程优先顺序 162
on_exit 设置程序正常结束前调用的函数 158
ptrace 进程追踪 165
setjmp 保存目前堆栈环境 163
setpgid 设置进程组识别码 160
setpgrp 设置进程组识别码 167
setpriority 设置程序进程执行优先权 162
siglongjmp 跳转到原先 sigsetjmp 保存的堆栈环境 169
sigsetjmp 保存目前堆栈环境 169
system 执行 shell 命令 170
wait 等待子进程中断或结束 159
waitpid 等待子进程中断或结束 159

第14章 格式化输入输出函数

fprintf 格式化输出数据至文件 173
fscanf 格式化字符串输入 173
printf 格式化输出数据 173
scanf 格式化字符串输入 173
snprintf 格式化字符串复制 19
sprintf 格式化字符串复制 19
sscanf 格式化字符串输入 173
vfprintf 格式化输出数据至文件 177
vfscanf 格式化字符串输入 178
vprintf 格式化输出数据 177
vscanf 格式化字符串输入 178
vsnprintf 格式化字符串复制 177
vsprintf 格式化字符串复制 179
vsscanf 格式化字符串输入 178

第15章 文件及目录函数

access 判断是否具有存取文件的权限 184
alphasort 依字母顺序排序目录结构 185
chdir 改变当前的工作目录 186
chmod 改变文件的权限 184
chown 改变文件的所有者 187
chroot 改变根目录 189
closedir 关闭目录 190
fchdir 改变当前的工作目录 186
fchmod 改变文件的权限 187
fchown 改变文件的所有者 188
fstat 由文件描述词取得文件状态 194
ftruncate 改变文件大小 195
ftw 遍历目录树 196
get_current_dir_name 取得当前的工作目录 197
getcwd 取得当前的工作目录 186
getwd 取得当前的工作目录 197
lchown 改变文件的所有者 188
link 建立文件连接 199
lstat 取得文件状态 199
nftw 遍历目录树 196
opendir 打开目录 190
readdir 读取目录 201
readlink 取得符号连接所指的文件 203
realpath 将相对目录路径转换成绝对路径 203
remove 删除文件 204
rename 更改文件名称或位置 204
rewinddir 重设读取目录的位置为开头位置 205
scandir 读取特定的目录数据 185
seekdir 设置下回读取目录的位置 205
stat 取得文件状态 184
symlink 建立文件符号连接 199
telldir 取得目录流的读取位置 207
truncate 改变文件大小 195
umask 设置建立新文件时的权限遮罩 124
unlink 删除文件 131
utime 修改文件的存取时间和更改时间 213
utimes 修改文件的存取时间和更改时间 213

第16章 信号函数

alarm 设置信号传送闹钟 215
kill 传送信号给指定的进程 216
pause 让进程暂停直到信号出现 215
psignal 列出信号描述和指定字符串 217
raise 传送信号给目前的进程 216
sigaction 查询或设置信号处理方式 219
sigaddset 增加一个信号至信号集 220
sigdelset 从信号集里删除一个信号 220
sigemptyset 初始化信号集 220
sigfillset 将所有信号加入至信号集 221
sigismember 测试某个信号是否已加入至信号集里 222
signal 设置信号处理方式 215
sigpause 暂停直到信号到来 224
sigpending 查询被搁置的信号 225
sigprocmask 查询或设置信号遮罩 219
sigsuspend 暂停直到信号到来 217
sleep 让进程暂停执行一段时间 228

第17章 错误处理函数

ferror 检查文件流是否有错误发生 136
perror 打印出错误原因信息字符串 217
strerror 返回错误原因的描述字符串 230

第18章 管道相关函数

mkfifo 建立具名管道 231
pclose 关闭管道 I/O 232
pipe 建立管道 231
popen 建立管道 I/O 232

第19章 Socket 相关函数

accept 接受 socket 连线 56
bind 对 socket 定位 235
connect 建立 socket 连线 237
endprotoent 结束网络协议数据的读取 238
endservent 结束网络服务数据的读取 238
gethostbyaddr 由 IP 地址取得网络数据 239
gethostbyname 由主机名称取得网络数据 239
getprotobyname 由网络协议名称取得协议数据 240
getprotobynumber 由网络协议编号取得协议数据 240
getprotoent 取得网络协议数据 238
getservbyname 依名称取得网络服务的数据 242
getservbyport 依 port 号码取得网络服务的数据 242
getservent 取得主机网络服务的数据 238
getsockopt 取得 socket 状态 244
herror 打印出网络错误原因信息字符串 245
hstrerror 返回网络错误原因的描述字符串 245
htonl 将 32 位主机字符顺序转换成网络字符顺序 235
htons 将 16 位主机字符顺序转换成网络字符顺序 242
inet_addr 将网络地址转成网络二进制的数字 247
inet_aton 将网络地址转成网络二进制的数字 247
inet_ntoa 将网络二进制的数字转换成网络地址 247
listen 等待连接 235
ntohl 将 32 位网络字符顺序转换成主机字符顺序 246
ntohs 将 16 位网络字符顺序转换成主机字符顺序 242
recv 经 socket 接收数据 251
recvfrom 经 socket 接收数据 251
recvmsg 经 socket 接收数据 253
send 经 socket 传送数据 251
sendmsg 经 socket 传送数据 253
sendto 经 socket 传送数据 252
setprotoent 打开网络协议的数据文件 241
setservent 打开主机网络服务的数据文件 243
setsockopt 设置 socket 状态 244
shutdown 终止 socket 通信 259
socket 建立一个 socket 通信 235

第20章 进程通信（IPC）函数

ftok 将文件路径和计划代号转为 System V IPC key 261
msgctl 控制信息队列的运作 262
msgget 建立信息队列 261
msgrcv 从信息队列读取信息 264
msgsnd 将信息送入信息队列 262
semctl 控制信号队列的操作 266
semget 配置信号队列 261
semop 信号处理 267
shmat attach 共享内存 269
shmctl 控制共享内存的操作 270
shmdt detach 共享内存 269
shmget 配置共享内存 261

第21章 记录函数

closelog 关闭信息记录 273
openlog 准备做信息记录 273
syslog 将信息记录至系统日志文件 273

第22章 环境变量函数

getenv 取得环境变量内容 275
putenv 改变或增加环境变量 275
setenv 改变或增加环境变量 275
unsetenv 清除环境变量内容 276

第23章 正则表达式

regcomp 编译正则表达式字符串 278
regerror 取得正则搜索的错误原因 278
regexec 进行正则表达式的搜索 278
regfree 释放正则表达式使用的内存 278

第24章 动态函数

dlclose 关闭动态函数库文件 282
dlerror 动态函数错误处理 282
dlopen 打开动态函数库文件 282
dlsym 从共享对象中搜索动态函数 282

第25章 其他函数

getopt 分析命令行参数 285
isatty 判断文件描述词是否是为终端机 286
select I/O 多工机制 287
ttyname 返回一终端机名称 286

附录

第1章 字符测试函数

本章函数用于测试字符的类别，均声明于 `<ctype.h>`，且大多以宏的形式实现。参数类型为 `int`，其值必须能表示为 `unsigned char` 或 `EOF`，否则结果未定义。

`isalnum` — 测试字符是否为英文字母或数字

头文件 `#include <ctype.h>`

原型 `int isalnum(int c);`

说明 检查参数 `c` 是否为英文字母或阿拉伯数字，等价于 `(isalpha(c) || isdigit(c))`。

返回值 若 `c` 为字母或数字则返回非 0（TRUE），否则返回 0。

相关函数 `isalpha`、`isdigit`、`islower`、`isupper`

```
/* 找出字符串中为字母或数字的字符 */
#include <ctype.h>
#include <stdio.h>

int main(void)
{
    char str[] = "123c@#FDsP[e?";
    int i;
    for (i = 0; str[i] != 0; i++)
        if (isalnum(str[i]))
            printf("%c is an alphanumeric character\n", str[i]);
    return 0;
}
```

isalpha — 测试字符是否为英文字母

头文件 `#include <ctype.h>`

原型 `int isalpha(int c);`

说明 检查参数 `c` 是否为英文字母，等价于 `(isupper(c) || islower(c))`。

返回值 若 `c` 为英文字母则返回非 0，否则返回 0。

相关函数 `isalnum`、`islower`、`isupper`

```
/* 找出字符串中的英文字母 */
#include <ctype.h>
#include <stdio.h>

int main(void)
{
    char str[] = "123c@#FDsP[e?";
    int i;
    for (i = 0; str[i] != 0; i++)
        if (isalpha(str[i]))
            printf("%c is an alphabetic character\n", str[i]);
    return 0;
}
```

isascii — 测试字符是否为 ASCII 码字符

头文件 `#include <ctype.h>`

原型 `int isascii(int c);`

说明 检查参数 `c` 是否为 ASCII 码字符，即判断 `c` 是否落在 0～127 之间。

返回值 若 `c` 的范围在 0～127 之间则返回非 0，否则返回 0。

相关函数 `iscntrl`、`toascii`

```
/* 判断 int 值是否具有对应的 ASCII 码字符 */
#include <ctype.h>
#include <stdio.h>

int main(void)
{
    int i;
    for (i = 125; i < 130; i++)
        if (isascii(i))
            printf("%d is an ascii character: %c\n", i, i);
        else
            printf("%d is not an ascii character\n", i);
    return 0;
}
```

isblank — 测试字符是否为空格字符

头文件 `#include <ctype.h>`

原型 `int isblank(int c);`

说明 检查参数 `c` 是否为空格字符，即空格（space，ASCII 32）或定位字符（tab，ASCII 9）。

返回值 若 `c` 为空格或定位字符则返回非 0，否则返回 0。

相关函数 `isspace`

```
/* 找出字符串中的空格字符并显示其 ASCII 码 */
#include <ctype.h>
#include <stdio.h>

int main(void)
{
    char str[] = "123c@# FD\tsP[e?";
    int i;
    for (i = 0; str[i] != 0; i++)
        if (isblank(str[i]))
            printf("str[%d] is blank character: %d\n", i, str[i]);
    return 0;
}
```

iscntrl — 测试字符是否为 ASCII 码的控制字符

头文件 `#include <ctype.h>`

原型 `int iscntrl(int c);`

说明 检查参数 `c` 是否为 ASCII 控制字符，即判断 `c` 是否落在 0～31 之间（含 DEL 0x7f）。

返回值 若 `c` 为控制字符则返回非 0，否则返回 0。

相关函数 `isascii`

```
/* 找出字符串中的控制字符 */
#include <ctype.h>
#include <stdio.h>

int main(void)
{
    char str[] = "a\tb\nc";
    int i;
    for (i = 0; str[i] != 0; i++)
        if (iscntrl(str[i]))
            printf("str[%d] is a control character: %d\n", i, str[i]);
    return 0;
}
```

isdigit — 测试字符是否为阿拉伯数字

头文件 `#include <ctype.h>`

原型 `int isdigit(int c);`

说明 检查参数 `c` 是否为阿拉伯数字 0～9。

返回值 若 `c` 为数字则返回非 0，否则返回 0。

相关函数 `isxdigit`

```
/* 找出字符串中的阿拉伯数字 */
#include <ctype.h>
#include <stdio.h>

int main(void)
{
    char str[] = "123c@#FDsP[e?";
    int i;
    for (i = 0; str[i] != 0; i++)
        if (isdigit(str[i]))
            printf("%c is a digit character\n", str[i]);
    return 0;
}
```

isgraph — 测试字符是否为可打印字符

头文件 `#include <ctype.h>`

原型 `int isgraph(int c);`

说明 检查参数 `c` 是否为可打印字符（不含空格）。

返回值 若 `c` 所对应的 ASCII 码可打印且非空格则返回非 0，否则返回 0。

相关函数 `isprint`

```
/* 判断字符串中哪些为可打印字符 */
#include <ctype.h>
#include <stdio.h>

int main(void)
{
    char str[] = "a5 @;";
    int i;
    for (i = 0; str[i] != 0; i++)
        if (isgraph(str[i]))
            printf("str[%d] is printable character: %c\n", i, str[i]);
    return 0;
}
```

islower — 测试字符是否为小写英文字母

头文件 `#include <ctype.h>`

原型 `int islower(int c);`

说明 检查参数 `c` 是否为小写英文字母 a～z。

返回值 若 `c` 为小写字母则返回非 0，否则返回 0。

相关函数 `isalpha`、`isupper`、`tolower`

```
/* 找出字符串中的小写英文字母 */
#include <ctype.h>
#include <stdio.h>

int main(void)
{
    char str[] = "123c@#FDsP[e?";
    int i;
    for (i = 0; str[i] != 0; i++)
        if (islower(str[i]))
            printf("%c is a lower-case character\n", str[i]);
    return 0;
}
```

isprint — 测试字符是否为可打印字符

头文件 `#include <ctype.h>`

原型 `int isprint(int c);`

说明 检查参数 `c` 是否为可打印字符，包含空格字符（与 `isgraph` 的差别）。

返回值 若 `c` 所对应的 ASCII 码可打印则返回非 0，否则返回 0。

相关函数 `isgraph`

```
/* 判断字符串中哪些为可打印字符（含空格） */
#include <ctype.h>
#include <stdio.h>

int main(void)
{
    char str[] = "a5 @;";
    int i;
    for (i = 0; str[i] != 0; i++)
        if (isprint(str[i]))
            printf("str[%d] is printable character: %c\n", i, str[i]);
    return 0;
}
```

isspace — 测试字符是否为空格字符

头文件 `#include <ctype.h>`

原型 `int isspace(int c);`

说明 检查参数 `c` 是否为空白字符，包括空格 `' '`、定位字符 `'\t'`、回车 `'\r'`、换行 `'\n'`、垂直定位 `'\v'` 与翻页 `'\f'`。

返回值 若 `c` 为空白字符则返回非 0，否则返回 0。

相关函数 `isblank`、`isgraph`

```
/* 找出字符串中的空白字符并显示其 ASCII 码 */
#include <ctype.h>
#include <stdio.h>

int main(void)
{
    char str[] = "123c@# FD\tsP[e?\n";
    int i;
    for (i = 0; str[i] != 0; i++)
        if (isspace(str[i]))
            printf("str[%d] is a white-space character: %d\n", i, str[i]);
    return 0;
}
```

ispunct — 测试字符是否为标点符号或特殊符号

头文件 `#include <ctype.h>`

原型 `int ispunct(int c);`

说明 检查参数 `c` 是否为标点符号或特殊符号，即 `isgraph(c)` 为真且 `isalnum(c)` 为假的字符。

返回值 若 `c` 为标点符号或特殊符号则返回非 0，否则返回 0。

相关函数 `isgraph`、`isalnum`

```
/* 找出字符串中的标点符号 */
#include <ctype.h>
#include <stdio.h>

int main(void)
{
    char str[] = "123c@#FDsP[e?";
    int i;
    for (i = 0; str[i] != 0; i++)
        if (ispunct(str[i]))
            printf("%c is a punctuation character\n", str[i]);
    return 0;
}
```

isupper — 测试字符是否为大写英文字母

头文件 `#include <ctype.h>`

原型 `int isupper(int c);`

说明 检查参数 `c` 是否为大写英文字母 A～Z。

返回值 若 `c` 为大写字母则返回非 0，否则返回 0。

相关函数 `isalpha`、`islower`、`toupper`

```
/* 找出字符串中的大写英文字母 */
#include <ctype.h>
#include <stdio.h>

int main(void)
{
    char str[] = "123c@#FDsP[e?";
    int i;
    for (i = 0; str[i] != 0; i++)
        if (isupper(str[i]))
            printf("%c is an upper-case character\n", str[i]);
    return 0;
}
```

isxdigit — 测试字符是否为 16 进制数字

头文件 `#include <ctype.h>`

原型 `int isxdigit(int c);`

说明 检查参数 `c` 是否为 16 进制数字，即 0～9、A～F 或 a～f。

返回值 若 `c` 为 16 进制数字则返回非 0，否则返回 0。

相关函数 `isdigit`

```
/* 找出字符串中的 16 进制数字 */
#include <ctype.h>
#include <stdio.h>

int main(void)
{
    char str[] = "123c@#FDsP[e?";
    int i;
    for (i = 0; str[i] != 0; i++)
        if (isxdigit(str[i]))
            printf("%c is a hexadecimal digit\n", str[i]);
    return 0;
}
```

第2章 数据转换函数

本章函数用于字符串与数值之间的转换，以及字符大小写的转换。

atof — 将字符串转换成浮点型数

头文件 `#include <stdlib.h>`

原型 `double atof(const char *nptr);`

说明 扫描参数 `nptr` 字符串，跳过前面的空白字符，直到遇上数字或正负号才开始转换，再遇到非数字或字符串结束符 `'\0'` 时结束转换，结果以 `double` 返回。字符串可包含正负号、小数点与 `E(e)` 表示指数部分，如 `123.456e-2`。

返回值 返回转换后的浮点型数；无法转换时返回 0.0。

附加说明 不检测溢出与错误，建议使用 `strtod` 取代。

相关函数 `atoi`、`atol`、`strtod`

```
/* 将字符串 a 与 b 转换为浮点型数并相加 */
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    char *a = "-100.23";
    char *b = "200e-2";
    printf("a+b = %f\n", atof(a) + atof(b));
    return 0;
}
/* 执行结果: a+b = -98.230000 */
```

atoi — 将字符串转换成整型数

头文件 `#include <stdlib.h>`

原型 `int atoi(const char *nptr);`

说明 扫描参数 `nptr` 字符串，跳过空白字符，直到遇上数字或正负号才开始转换，遇到非数字或 `'\0'` 时结束，结果以 `int` 返回。

返回值 返回转换后的整型数；无法转换时返回 0。

附加说明 不检测溢出，建议使用 `strtol`。

相关函数 `atof`、`atol`、`strtol`

```
/* 将字符串 a 与 b 转换为整型数并相加 */
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    char a[] = "-100";
    char b[] = "456";
    printf("a+b = %d\n", atoi(a) + atoi(b));
    return 0;
}
/* 执行结果: a+b = 356 */
```

atol — 将字符串转换成长整型数

头文件 `#include <stdlib.h>`

原型 `long atol(const char *nptr);`

说明 与 `atoi` 用法相同，只是返回值为 `long` 类型。

返回值 返回转换后的长整型数；无法转换时返回 0。

附加说明 建议使用 `strtol` 取代。

相关函数 `atof`、`atoi`、`strtol`

```
/* 将字符串转换为长整型数 */
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    char a[] = "100000000";
    char b[] = " 234567890";
    printf("a+b = %ld\n", atol(a) + atol(b));
    return 0;
}
/* 执行结果: a+b = 334567890 */
```

ecvt — 将浮点型数转换成字符串，取四舍五入

头文件 `#include <stdlib.h>`

原型 `char *ecvt(double number, int ndigits, int *decpt, int *sign);`

说明 将参数 `number` 转换成以 `'\0'` 结尾的 ASCII 字符串，参数 `ndigits` 表示总共显示的位数。转换后的字符串不含小数点：`decpt` 返回小数点在该字符串中的位置（负数表示小数点在返回字符串左侧），`sign` 表示正负（0 为正，非 0 为负）。

返回值 返回字符串指针，其内容在下次调用相关函数时会被覆盖。

附加说明 建议使用 `sprintf` 或 `snprintf` 取代。

相关函数 `fcvt`、`gcvt`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    int decpt, sign;
    char *str = ecvt(123.456, 5, &decpt, &sign);
    printf("str=%s decpt=%d sign=%d\n", str, decpt, sign);
    return 0;
}
/* 执行结果: str=12346 decpt=3 sign=0 */
```

fcvt — 将浮点型数转换为字符串，取四舍五入

头文件 `#include <stdlib.h>`

原型 `char *fcvt(double number, int ndigits, int *decpt, int *sign);`

说明 与 `ecvt` 类似，但参数 `ndigits` 表示小数点后的位数，因此结果会按小数位数做四舍五入。

返回值 返回字符串指针，内容会在下次调用相关函数时被覆盖。

附加说明 建议使用 `snprintf` 取代。

相关函数 `ecvt`、`gcvt`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    int decpt, sign;
    char *str = fcvt(123.456, 2, &decpt, &sign);
    printf("str=%s decpt=%d sign=%d\n", str, decpt, sign);
    return 0;
}
/* 执行结果: str=12346 decpt=3 sign=0 */
```

gcvt — 将浮点型数转换为字符串，取四舍五入

头文件 `#include <stdlib.h>`

原型 `char *gcvt(double number, size_t ndigit, char *buf);`

说明 将 `number` 转换成 ASCII 字符串，参数 `ndigit` 表示显示的位数。转换结果包含小数点与正负号，并尽可能以指数形式表示。

返回值 返回参数 `buf` 指针，即转换结果的字符串。

附加说明 建议使用 `snprintf` 取代。

相关函数 `ecvt`、`fcvt`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    char buf[64];
    printf("%s\n", gcvt(123.456, 5, buf));   /* 123.46 */
    printf("%s\n", gcvt(-1234.5678, 5, buf)); /* -1234.6 */
    return 0;
}
```

`strtod` — 将字符串转换成浮点型数

头文件 `#include <stdlib.h>`

原型 `double strtod(const char *nptr, char **endptr);`

说明 扫描 `nptr` 字符串，跳过前导空白，将数值部分转换为 `double`。若 `endptr` 不为 NULL，则将遇到不合条件而终止的字符位置由 `endptr` 传回；`nptr` 可包含正负号、小数点、`E(e)` 或 `H(h)` 表示指数。

返回值 返回转换后的浮点型数；字符串无法转换时返回 0.0，并通过 `endptr` 传回原字符串。超出可表示范围时返回 `HUGE_VAL` 并设置 `errno` 为 `ERANGE`。

相关函数 `atof`、`strtol`、`strtoul`

```
/* 以 'e' 为分隔符，将字符串分段转换为浮点型数 */
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    char a[] = "100e-2-33.5";
    char *end;
    printf("%f\n", strtod(a, &end));
    printf("%s\n", end);
    return 0;
}
/* 执行结果：1.000000  -33.5 */
```

strtol — 将字符串转换成长整型数

头文件 `#include <stdlib.h>`

原型 `long int strtol(const char *nptr, char **endptr, int base);`

说明 将 `nptr` 字符串按参数 `base`（2～36，或 0 表示自动判别：0 开头为八进制、0x 为十六进制、否则十进制）转换成长整型数。遇到不合条件的字符结束转换，`endptr` 不为 NULL 时传回该字符位置。

返回值 返回转换后的长整型数；无法转换时返回 0，超出范围时返回 `LONG_MAX` 或 `LONG_MIN` 并设置 `errno` 为 `ERANGE`。

相关函数 `atol`、`strtod`、`strtoul`

```
/* 将十六进制字符串 "0x1f" 与二进制 "101" 转换后相加 */
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    char a[] = "0x1f-2";
    char b[] = "101";
    char *end;
    printf("%ld\n", strtol(a, &end, 0) + strtol(b, NULL, 2));
    return 0;
}
/* 执行结果：36 */
```

strtoul — 将字符串转换成无符号长整型数

头文件 `#include <stdlib.h>`

原型 `unsigned long int strtoul(const char *nptr, char **endptr, int base);`

说明 用法与 `strtol` 相同，只是结果为无符号长整型数，可表示范围为 `0` 至 `ULONG_MAX`。

返回值 返回转换后的无符号长整型数；超出范围时返回 `ULONG_MAX` 并设置 `errno` 为 `ERANGE`。

相关函数 `strtol`、`strtod`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    char a[] = "1000000000";
    char b[] = " 0xffffffff";
    printf("%lu\n", strtoul(a, NULL, 10) + strtoul(b, NULL, 16));
    return 0;
}
/* 执行结果：5294967295 */
```

toascii — 将整型数转换成合法的 ASCII 码字符

头文件 `#include <ctype.h>`

原型 `int toascii(int c);`

说明 将参数 `c` 转换成 7 位的 `unsigned char` 值，即屏蔽高位置零并保留低 7 位，使 `c` 落在 0～127 之间。

返回值 返回转换后的 ASCII 码字符。

相关函数 `isascii`

```
#include <ctype.h>
#include <stdio.h>

int main(void)
{
    int a = 217;
    printf("%d -> %d '%c'\n", a, toascii(a), toascii(a));
    return 0;
}
/* 执行结果: 217 -> 89 'Y' */
```

tolower — 将大写字母转换成小写字母

头文件 `#include <ctype.h>`

原型 `int tolower(int c);`

说明 若参数 `c` 为大写英文字母则转换成对应的小写字母，其他字符原样返回。

返回值 返回转换后的小写字母；若无需转换则返回原参数值。

相关函数 `toupper`、`islower`

```
/* 将字符串中的大写字母转换为小写字母 */
#include <ctype.h>
#include <stdio.h>

int main(void)
{
    char s[] = "aBcDeF";
    int i;
    for (i = 0; s[i] != 0; i++)
        s[i] = tolower(s[i]);
    printf("%s\n", s);   /* abcdef */
    return 0;
}
```

toupper — 将小写字母转换成大写字母

头文件 `#include <ctype.h>`

原型 `int toupper(int c);`

说明 若参数 `c` 为小写英文字母则转换成对应的大写字母，其他字符原样返回。

返回值 返回转换后的大写字母；若无需转换则返回原参数值。

相关函数 `tolower`、`isupper`

```
/* 将字符串中的小写字母转换为大写字母 */
#include <ctype.h>
#include <stdio.h>

int main(void)
{
    char s[] = "aBcDeF";
    int i;
    for (i = 0; s[i] != 0; i++)
        s[i] = toupper(s[i]);
    printf("%s\n", s);   /* ABCDEF */
    return 0;
}
```

第3章 内存配置函数

本章函数用于配置、释放与管理进程的内存空间。

`alloca` — 配置内存空间

头文件 `#include <alloca.h>`

原型 `void *alloca(size_t size);`

说明 从栈（stack）中配置 `size` 字节的内存空间。函数返回时该空间自动释放，无需调用 `free`。

返回值 返回配置内存的起始地址；若栈空间不足，行为未定义。

附加说明 不属于标准 C，移植性较差；若分配失败不会返回 NULL，可能导致程序崩溃。不建议在循环或函数参数中调用。

相关函数 `malloc`、`calloc`、`free`

```
#include <alloca.h>
#include <stdio.h>
#include <string.h>

int main(void)
{
    char *p = (char *)alloca(16);
    strcpy(p, "hello");
    printf("%s\n", p);   /* 无需 free */
    return 0;
}
```

brk — 改变数据字节的范围

头文件	`#include <unistd.h>`
原型	`int brk(void *end_data_segment);`
说明	将数据段（data segment）的结束地址设置为 `end_data_segment`，用于调整进程堆的大小。
返回值	成功返回 0，失败返回 -1 并设置 `errno` 为 `ENOMEM`。
附加说明	通常程序使用 `malloc` / `sbrk` 即可，`brk` 是底层系统调用。
相关函数	`sbrk`、`malloc`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    void *old = sbrk(0);          /* 当前堆顶 */
    brk((char *)old + 4096);      /* 扩展一页 */
    printf("old=%p\n", old);
    return 0;
}
```

`calloc` — 配置内存空间

头文件	`#include <stdlib.h>`
原型	`void *calloc(size_t nmemb, size_t size);`
说明	配置 `nmemb` 个长度为 `size` 的连续内存空间，共 `nmemb * size` 字节，并将内容全部初始化为 0。
返回值	成功返回配置内存的起始地址，失败返回 NULL。
附加说明	与 `malloc` 的差别在于 `calloc` 会把内存清零，适合配置数组。
相关函数	`malloc`、`free`、`realloc`

```
/* 动态配置 10 个结构体数组 */
#include <stdlib.h>
#include <stdio.h>

struct test { int a[10]; char b[16]; };

int main(void)
{
    struct test *ptr = calloc(10, sizeof(struct test));
    if (ptr == NULL) {
        perror("calloc");
        return 1;
    }
    printf("a[0]=%d b[0]=%d\n", ptr[0].a[0], ptr[0].b[0]); /* 已被清零 */
    free(ptr);
    return 0;
}
```

free — 释放原先配置的内存

头文件 `#include <stdlib.h>`

原型 `void free(void *ptr);`

说明 释放由 `malloc`、`calloc` 或 `realloc` 配置的指针所指的内存空间，交由系统回收。

返回值 无返回值。

附加说明 参数 `ptr` 必须是配置内存时返回的指针，重复释放或释放非堆内存会导致错误；`ptr` 为 NULL 时函数不做任何事。

相关函数 `malloc`、`calloc`、`realloc`

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>

int main(void)
{
    char *p = malloc(32);
    strcpy(p, "abc");
    printf("%s\n", p);
    free(p);
    return 0;
}
```

getpagesize — 取得内存分页大小

头文件 `#include <unistd.h>`

原型 `size_t getpagesize(void);`

说明 返回系统内存分页（page）的大小，单位字节。

返回值 内存分页大小，通常为 4096 字节。

附加说明 在移植性要求高的程序中可使用 `sysconf(_SC_PAGESIZE)` 取代。

相关函数 `sbrk`、`mmap`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    printf("page size = %zu bytes\n", getpagesize());
    return 0;
}
```

`malloc` — 配置内存空间

头文件 `#include <stdlib.h>`

原型 `void *malloc(size_t size);`

说明 配置 `size` 字节的内存空间，所配置的内存内容未被初始化。

返回值 成功返回指向所配置内存的指针，失败返回 NULL。

附加说明 使用前务必检查返回值是否为 NULL；使用后须以 `free` 释放，以免内存泄漏。

相关函数 `calloc`、`free`、`realloc`

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>

int main(void)
{
    char *p = malloc(32);
    if (p == NULL) {
        perror("malloc");
        return 1;
    }
    strcpy(p, "hello malloc");
    printf("%s\n", p);
    free(p);
    return 0;
}
```

`mmap` — 建立内存映射

头文件

原型 `void *mmap(void *start, size_t length, int prot, int flags, int fd, off_t offset);`

说明 将某个文件内容或设备映射到进程的内存中，读写内存即相当于读写文件，可直接用指针访问而不必使用 `read` / `write`。参数 `start` 为期望的起始地址（通常设为 0 由系统决定）；`length` 为映射长度；`prot` 为保护方式，可取 `PROT_READ`、`PROT_WRITE`、`PROT_EXEC`、`PROT_NONE`；`flags` 影响映射特性，常用的有 `MAP_SHARED`（改动反映到文件）、`MAP_PRIVATE`（写时复制，不影响文件）、`MAP_ANONYMOUS`（匿名映射，忽略 `fd`）；`fd` 为已打开的文件描述词；`offset` 为文件偏移，通常为页大小的整数倍。

返回值 成功返回映射区的起始地址，失败返回 `MAP_FAILED`（即 `(void *)-1`）并设置 `errno`。

附加说明 映射完成后可用 `msync` 同步到磁盘，用 `munmap` 解除映射。

相关函数 `munmap`、`open`、`msync`

```
#include <unistd.h>
#include <sys/mman.h>
```

```
/* 利用 mmap 读取文件内容 */
#include <unistd.h>
#include <sys/mman.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>

int main(void)
{
    int fd = open("/etc/passwd", O_RDONLY);
    struct stat sb;
    fstat(fd, &sb);
    char *p = mmap(NULL, sb.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
    if (p == MAP_FAILED) {
        perror("mmap");
        return 1;
    }
    printf("%.40s\n", p);       /* 直接以指针访问文件内容 */
    munmap(p, sb.st_size);
    close(fd);
    return 0;
}
```

`munmap` — 解除内存映射

头文件

原型 `int munmap(void *start, size_t length);`

说明 解除由 `mmap` 建立的、从 `start` 开始长度为 `length` 的内存映射。进程结束时映射也会自动解除，但关闭文件描述词并不会解除映射。

返回值 成功返回 0，失败返回 -1 并设置 `errno` 为 `EINVAL`。

相关函数 `mmap`

```
#include <unistd.h>
#include <sys/mman.h>
```

```
#include <unistd.h>
#include <sys/mman.h>
#include <stdio.h>

int main(void)
{
    char *p = mmap(NULL, 4096, PROT_READ | PROT_WRITE,
                   MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (p == MAP_FAILED) {
        perror("mmap");
        return 1;
    }
    p[0] = 'A';
    munmap(p, 4096);
    return 0;
}
```

realloc — 更改已配置的内存空间

头文件 `#include <stdlib.h>`

原型 `void *realloc(void *ptr, size_t size);`

说明 将 `ptr` 所指的内存空间大小改为 `size` 字节。原内容保持不变（缩小时超出部分丢失，扩大时新增部分未初始化）；若原位置不足会自动搬移到新地址。

返回值 成功返回新的内存地址（可能与 `ptr` 不同），失败返回 NULL 而原内存保持不变。

附加说明 调用后应使用返回值更新指针，不要直接使用旧指针。

相关函数 `malloc`、`calloc`、`free`

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>

int main(void)
{
    char *p = malloc(16);
    strcpy(p, "hello");
    p = realloc(p, 64);        /* 注意使用返回值 */
    strcat(p, " world");
    printf("%s\n", p);
    free(p);
    return 0;
}
```

sbrk — 增加程序可用的数据空间

头文件 `#include <unistd.h>`

原型 `void *sbrk(ptrdiff_t increment);`

说明 将程序的数据段（堆）增加 `increment` 字节；`increment` 为 0 时仅返回当前堆顶地址。

返回值 成功返回增加前的堆顶地址，失败返回 `(void *)-1` 并设置 `errno` 为 `ENOMEM`。

附加说明 一般由 `malloc` 内部使用，应用程序较少直接调用。

相关函数 `brk`、`malloc`、`getpagesize`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    void *old = sbrk(0);
    void *p = sbrk(4096);
    printf("old=%p new=%p\n", old, p);
    return 0;
}
```

第4章 时间函数

本章函数用于取得、设置、计算与格式化时间和日期。使用时通常需包含 `<time.h>`。

`asctime` — 将时间和日期以字符串格式表示

头文件 `#include <time.h>`

原型 `char *asctime(const struct tm *timeptr);`

说明 将参数 `timeptr` 所指的 `tm` 结构中的时间和日期转换成字符串，格式固定为 “Wed Jun 30 21:49:08 1993\n”。

返回值 返回字符串指针，内容为表示时间与日期的字符串；其内容由静态内存提供，下次调用会被覆盖。

附加说明 已过时，建议改用可自定义格式的 `strftime`。

相关函数 `ctime`、`gmtime`、`localtime`、`strftime`

```
#include <time.h>
#include <stdio.h>

int main(void)
{
    time_t t = time(NULL);
    printf("%s", asctime(gmtime(&t)));
    return 0;
}
```

clock — 取得进程占用 CPU 的大约时间

头文件 `#include <time.h>`

原型 `clock_t clock(void);`

说明 返回进程自启动以来所占用的 CPU 时间，单位为 `CLOCKS_PER_SEC` 分之一秒（通常为微秒）。

返回值 返回 CPU 时间；无法取得时返回 -1。

附加说明 常用于测量一段代码的运行耗时。

相关函数 `time`、`gettimeofday`

```
#include <time.h>
#include <stdio.h>

int main(void)
{
    clock_t start = clock();
    for (long i = 0; i < 20000000L; i++)
        ;
    printf("used %.3f sec\n",
           (double)(clock() - start) / CLOCKS_PER_SEC);
    return 0;
}
```

ctime — 将时间和日期以字符串格式表示

头文件 `#include <time.h>`

原型 `char *ctime(const time_t *timep);`

说明 将参数 `timep` 所指的 `time_t` 值转换成当地时区的日期与时间字符串，等价于 `asctime(localtime(timep))`。

返回值 返回字符串指针，格式为 “Wed Jun 30 21:49:08 1993\n”；由静态内存提供。

相关函数 `asctime`、`localtime`、`strftime`

```
#include <time.h>
#include <stdio.h>

int main(void)
{
    time_t t = time(NULL);
    printf("%s", ctime(&t));
    return 0;
}
```

difftime — 计算时间差距

头文件 `#include <time.h>`

原型 `double difftime(time_t time1, time_t time0);`

说明 计算两个时间值 `time1 - time0` 的差距，单位为秒。

返回值 返回以 `double` 表示的秒数差。

相关函数 `time`、`mktime`

```
#include <time.h>
#include <stdio.h>

int main(void)
{
    time_t t0 = time(NULL);
    sleep(2);
    printf("elapsed %.0f sec\n", difftime(time(NULL), t0));
    return 0;
}
```

ftime — 取得目前的时间和日期

头文件 `#include <sys/timeb.h>`

原型 `int ftime(struct timeb *tp);`

说明 将当前时间与日期存入 `tp` 所指的结构。`struct timeb` 含有 `time`（自 1970 年起的秒数）、`millitm`（毫秒）、`timezone`（与格林威治的时差，单位为分钟）、`dstflag`（夏令时标志）。

返回值 总是返回 0。

附加说明 已过时，建议使用 `gettimeofday`。

相关函数 `time`、`gettimeofday`

```
#include <sys/timeb.h>
#include <stdio.h>

int main(void)
{
    struct timeb tb;
    ftime(&tb);
    printf("sec=%ld ms=%u\n", (long)tb.time, tb.millitm);
    return 0;
}
```

gettimeofday — 取得目前的时间

头文件

原型 `int gettimeofday(struct timeval *tv, struct timezone *tz);`

说明 将目前的时间存入 `tv` 所指的结构，当地时区信息存入 `tz` 所指的结构。`struct timeval` 含 `tv_sec`（秒）与 `tv_usec`（微秒）；`tz` 通常设为 NULL。

返回值 成功返回 0，失败返回 -1 并设置 `errno`。

附加说明 可提供微秒级精度，常用于性能测量。

相关函数 `time`、`settimeofday`、`ftime`

```
#include <sys/time.h>
#include <unistd.h>
```

```
#include <sys/time.h>
#include <stdio.h>

int main(void)
{
    struct timeval tv;
    gettimeofday(&tv, NULL);
    printf("sec=%ld usec=%ld\n", (long)tv.tv_sec, (long)tv.tv_usec);
    return 0;
}
```

gmtime — 取得目前的时间和日期

头文件 `#include <time.h>`

原型 `struct tm *gmtime(const time_t *timep);`

说明 将 `timep` 所指的 `time_t` 值转换成世界协调时间（UTC/GMT）的 `struct tm` 结构。结构中含 `tm_sec`、`tm_min`、`tm_hour`、`tm_mday`、`tm_mon`（0～11）、`tm_year`（自 1900 年起算）、`tm_wday`、`tm_yday` 等成员。

返回值 返回指向 `struct tm` 的指针，该结构由静态内存提供，多线程环境下应使用 `gmtime_r`。

相关函数 `localtime`、`asctime`、`time`

```
#include <time.h>
#include <stdio.h>

int main(void)
{
    time_t t = time(NULL);
    struct tm *g = gmtime(&t);
    printf("%04d-%02d-%02d %02d:%02d:%02d UTC\n",
           g->tm_year + 1900, g->tm_mon + 1, g->tm_mday,
           g->tm_hour, g->tm_min, g->tm_sec);
    return 0;
}
```

localtime — 取得当地目前的时间和日期

头文件 `#include <time.h>`

原型 `struct tm *localtime(const time_t *timep);`

说明 将 `timep` 所指的 `time_t` 值转换成当地时区的 `struct tm` 结构，会考虑时区与夏令时。

返回值 返回指向 `struct tm` 的指针，结构由静态内存提供。

附加说明 多线程环境建议使用 `localtime_r`。

相关函数 `gmtime`、`ctime`、`tzset`

```
#include <time.h>
#include <stdio.h>

int main(void)
{
    time_t t = time(NULL);
    struct tm *l = localtime(&t);
    printf("%04d-%02d-%02d %02d:%02d:%02d\n",
           l->tm_year + 1900, l->tm_mon + 1, l->tm_mday,
           l->tm_hour, l->tm_min, l->tm_sec);
    return 0;
}
```

`mktime` — 将时间结构数据转换成经过的秒数

头文件 `#include <time.h>`

原型 `time_t mktime(struct tm *timeptr);`

说明 将 `timeptr` 所指的 `struct tm` 数据转换成自 1970 年 1 月 1 日 0 时 0 分 0 秒起所经过的秒数；同时会把结构中超出范围的成员正规化，并填入 `tm_wday` 与 `tm_yday`。

返回值 返回经过的秒数；无法表示时返回 `(time_t)-1`。

相关函数 `time`、`localtime`、`gmtime`

```
/* 计算两个日期之间相差的天数 */
#include <time.h>
#include <stdio.h>

int main(void)
{
    struct tm t1 = {0}, t2 = {0};
    t1.tm_year = 2000 - 1900; t1.tm_mon = 0;  t1.tm_mday = 1;
    t2.tm_year = 2001 - 1900; t2.tm_mon = 0;  t2.tm_mday = 1;
    double d = difftime(mktime(&t2), mktime(&t1));
    printf("days = %.0f\n", d / 86400);
    return 0;
}
```

settimeofday — 设置目前的时间

头文件

原型 `int settimeofday(const struct timeval *tv, const struct timezone *tz);`

说明 将系统当前时间设置为 `tv` 所指的时间，时区信息设置为 `tz` 所指的内容（`tz` 一般设为 NULL）。

返回值 成功返回 0，失败返回 -1 并设置 `errno` 为 `EPERM`（权限不足）或 `EINVAL`。

附加说明 只有 root 权限才能设置系统时间。

相关函数 `gettimeofday`、`time`

```
#include <sys/time.h>
#include <unistd.h>
```

```
#include <sys/time.h>
#include <stdio.h>

int main(void)   /* 需以 root 执行 */
{
    struct timeval tv;
    gettimeofday(&tv, NULL);
    tv.tv_sec += 1;
    if (settimeofday(&tv, NULL) < 0)
        perror("settimeofday");
    return 0;
}
```

strftime — 格式化日期和时间

头文件 `#include <time.h>`

原型 `size_t strftime(char *s, size_t max, const char *format, const struct tm *tm);`

说明 按 `format` 指定的格式将 `tm` 中的时间日期格式化到字符串 `s` 中，`max` 为 `s` 的最大长度。常用格式控制符：`%Y` 四位年、`%m` 月、`%d` 日、`%H` 时（24 小时制）、`%M` 分、`%S` 秒、`%A` 星期名、`%B` 月份名、`%F` 等价于 `%Y-%m-%d`、`%T` 等价于 `%H:%M:%S`、`%s` 秒数、`%z` 时区。

返回值 返回写入 `s` 的字符数（不含结尾 `'\0'`）；若结果超过 `max` 则返回 0 且 `s` 内容不确定。

相关函数 `asctime`、`ctime`、`strptime`

```
#include <time.h>
#include <stdio.h>

int main(void)
{
    char buf[64];
    time_t t = time(NULL);
    strftime(buf, sizeof(buf), "%Y-%m-%d %H:%M:%S %A", localtime(&t));
    printf("%s\n", buf);
    return 0;
}
```

time — 取得目前的时间

头文件 `#include <time.h>`

原型 `time_t time(time_t *t);`

说明 返回自 1970 年 1 月 1 日 0 时 0 分 0 秒（UTC）起至当前所经过的秒数。若参数 `t` 非 NULL，则同时把结果存入 `t` 所指的位置。

返回值 成功返回秒数，失败返回 `(time_t)-1`。

相关函数 `ctime`、`gmtime`、`localtime`、`gettimeofday`

```
#include <time.h>
#include <stdio.h>

int main(void)
{
    time_t t = time(NULL);
    printf("seconds since epoch: %ld\n", (long)t);
    return 0;
}
```

`tzset` — 设置时区以供时间转换

头文件 `#include <time.h>`

原型 `void tzset(void);`

说明 读取环境变量 `TZ` 来设置时区变量 `timezone`（与 UTC 相差的秒数）、`daylight`（是否使用夏令时）与 `tzname[]`（时区名称），供 `localtime`、`ctime` 等函数使用。若 `TZ` 未设置，则使用 `/etc/localtime`。

返回值 无返回值。

附加说明 使用 `localtime` 前若修改了 `TZ`，应先调用 `tzset`。

相关函数 `localtime`、`ctime`

```
#include <time.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    setenv("TZ", "Asia/Shanghai", 1);
    tzset();
    time_t t = time(NULL);
    printf("%s", ctime(&t));
    return 0;
}
```

第5章 字符串处理函数

本章函数用于内存与字符串的比较、拷贝、查找、连接与分割等操作。

bcmp — 比较内存内容

头文件 `#include <strings.h>`

原型 `int bcmp(const void *s1, const void *s2, size_t n);`

说明 比较 `s1` 与 `s2` 所指内存区间前 `n` 个字节的内容。

返回值 内容完全相同返回 0，否则返回非 0。

附加说明 已过时，建议使用 `memcmp`。

相关函数 `bcopy`、`bzero`、`memcmp`

```
#include <strings.h>
#include <stdio.h>

int main(void)
{
    char a[] = "abc", b[] = "abd";
    printf("%d\n", bcmp(a, b, 3));   /* 非 0: 不相等 */
    return 0;
}
```

bcopy — 拷贝内存内容

头文件 `#include <strings.h>`

原型 `void bcopy(const void *src, void *dest, size_t n);`

说明 将 `src` 所指内存的前 `n` 个字节拷贝到 `dest`，可正确处理内存重叠的情况（注意参数顺序与 `memcpy` 相反）。

返回值 无返回值。

附加说明 已过时，建议使用 `memcpy` 或 `memmove`。

相关函数 `bcmp`、`memcpy`、`memmove`

```
#include <strings.h>
#include <stdio.h>

int main(void)
{
    char s[] = "hello";
    bcopy(s + 1, s, 4);       /* 处理重叠区域 */
    printf("%s\n", s);
    return 0;
}
```

bzero — 将一段内存内容全清为零

头文件 `#include <strings.h>`

原型 `void bzero(void *s, size_t n);`

说明 将 `s` 所指内存区间的前 `n` 个字节全部填入 0。

返回值 无返回值。

附加说明 建议使用标准的 `memset` 取代。

相关函数 `memset`、`bcopy`

```
#include <strings.h>
#include <stdio.h>

int main(void)
{
    char buf[16] = "hello";
    bzero(buf, sizeof(buf));
    printf("[%s]\n", buf);   /* 已清空 */
    return 0;
}
```

ffs — 在一整型数中查找第一个值为真的位

头文件 `#include <strings.h>`

原型 `int ffs(int i);`

说明 从低位开始查找参数 `i` 中第一个值为 1 的位（bit）。

返回值 返回该位的位置（最低位为 1）；若 `i` 为 0 则返回 0。

相关函数 `ffsl`、`ffsll`

```
#include <strings.h>
#include <stdio.h>

int main(void)
{
    printf("%d\n", ffs(0));    /* 0 */
    printf("%d\n", ffs(8));    /* 4: 第 4 位为 1 */
    return 0;
}
```

index — 查找字符串中第一个出现的指定字符

头文件 `#include <strings.h>`

原型 `char *index(const char *s, int c);`

说明 在字符串 `s` 中查找字符 `c` 第一次出现的位置。

返回值 找到则返回该字符的地址，未找到则返回 NULL。

附加说明 等价于 `strchr`，建议使用 `strchr`。

相关函数 `rindex`、`strchr`、`strrchr`

```
#include <strings.h>
#include <stdio.h>

int main(void)
{
    char s[] = "01234567890123456789";
    printf("%s\n", index(s, '5'));
    return 0;
}
```

memccpy — 拷贝内存内容

头文件 `#include <string.h>`

原型 `void *memccpy(void *dest, const void *src, int c, size_t n);`

说明 从 `src` 拷贝字节到 `dest`，最多拷贝 `n` 个字节；遇到字符 `c` 时停止（该字符也会被拷贝）。

返回值 若拷贝到字符 `c`，返回 `dest` 中 `c` 之后一个字节的地址；若 `n` 个字节内未出现 `c` 则返回 NULL。

相关函数 `memcpy`、`memmove`

```
#include <string.h>
#include <stdio.h>

int main(void)
{
    char src[] = "hello,world", dst[32];
    void *p = memccpy(dst, src, ',', strlen(src));
    if (p)
        *(char *)p = '\0';
    printf("%s\n", dst);   /* hello, */
    return 0;
}
```

memchr — 在某一内存范围中查找一特定字符

头文件 `#include <string.h>`

原型 `void *memchr(const void *s, int c, size_t n);`

说明 在 `s` 所指内存区间的前 `n` 个字节中查找字符 `c`（以 `unsigned char` 比较）。

返回值 找到则返回该位置的地址，未找到返回 NULL。

相关函数 `strchr`、`index`、`memcmp`

```
#include <string.h>
#include <stdio.h>

int main(void)
{
    char s[] = "0123456789";
    char *p = memchr(s, '5', 10);
    printf("%s\n", p);   /* 56789 */
    return 0;
}
```

memcmp — 比较内存内容

头文件 `#include <string.h>`

原型 `int memcmp(const void *s1, const void *s2, size_t n);`

说明 比较 `s1` 与 `s2` 所指内存前 `n` 个字节的内容，比较方式以 `unsigned char` 进行。

返回值 相同返回 0；`s1` 大于 `s2` 返回正值；`s1` 小于 `s2` 返回负值。

附加说明 不依赖字符串结尾 `'\0'`，可比较含 `\0` 的内存块。

相关函数 `strcmp`、`strncmp`、`bcmp`

```
#include <string.h>
#include <stdio.h>

int main(void)
{
    char a[] = "aBc", b[] = "aBd";
    printf("%d\n", memcmp(a, b, 3));   /* 负值 */
    return 0;
}
```

memcpy — 拷贝内存内容

头文件 `#include <string.h>`

原型 `void *memcpy(void *dest, const void *src, size_t n);`

说明 从 `src` 拷贝 `n` 个字节到 `dest`。

返回值 返回 `dest` 的地址。

附加说明 `src` 与 `dest` 的内存区域不可重叠，否则应改用 `memmove`；`memcpy` 遇到 `'\0'` 不会停止。

相关函数 `memmove`、`memccpy`、`strcpy`

```
#include <string.h>
#include <stdio.h>

int main(void)
{
    char a[32] = "abc", b[32];
    memcpy(b, a, strlen(a) + 1);
    printf("%s\n", b);
    return 0;
}
```

memfrob — 对内存区域编码

头文件 `#include <string.h>`

原型 `void *memfrob(void *s, size_t n);`

说明 将 `s` 所指内存的前 `n` 个字节与 42（0x2a）做异或（XOR）运算，得到简单的编码结果；再执行一次即可还原。

返回值 返回 `s` 的地址。

附加说明 仅用于简单的混淆，不具备真正的加密强度；为 GNU 扩展函数。

相关函数 `strfry`、`memset`

```
#define _GNU_SOURCE
#include <string.h>
#include <stdio.h>

int main(void)
{
    char s[] = "hello";
    memfrob(s, strlen(s));
    memfrob(s, strlen(s));   /* 还原 */
    printf("%s\n", s);
    return 0;
}
```

memmove — 拷贝内存内容

头文件 `#include <string.h>`

原型 `void *memmove(void *dest, const void *src, size_t n);`

说明 与 `memcpy` 相同，但会先使用临时缓冲，因此可以正确处理 `src` 与 `dest` 重叠的情况。

返回值 返回 `dest` 的地址。

相关函数 `memcpy`、`bcopy`

```
#include <string.h>
#include <stdio.h>

int main(void)
{
    char s[] = "abcdef";
    memmove(s + 2, s, 4);    /* 区域重叠，安全 */
    printf("%s\n", s);
    return 0;
}
```

memset — 将一段内存空间填入某值

头文件 `#include <string.h>`

原型 `void *memset(void *s, int c, size_t n);`

说明 将 `s` 所指内存的前 `n` 个字节全部填入字符 `c`（以 `unsigned char` 取值）。

返回值 返回 `s` 的地址。

附加说明 常用于清空缓冲区或初始化结构体。

相关函数 `bzero`、`memcpy`

```
#include <string.h>
#include <stdio.h>

int main(void)
{
    char buf[16];
    memset(buf, 'A', sizeof(buf) - 1);
    buf[sizeof(buf) - 1] = '\0';
    printf("%s\n", buf);
    return 0;
}
```

rindex — 查找字符串中最后一个出现的指定字符

头文件 `#include <strings.h>`

原型 `char *rindex(const char *s, int c);`

说明 在字符串 `s` 中查找字符 `c` 最后一次出现的位置。

返回值 找到则返回该地址，未找到返回 NULL。

附加说明 等价于 `strrchr`，建议使用 `strrchr`。

相关函数 `index`、`strrchr`

```
#include <strings.h>
#include <stdio.h>

int main(void)
{
    char s[] = "01234567890123456789";
    printf("%s\n", rindex(s, '5'));
    return 0;
}
```

strcasecmp — 忽略大小写比较字符串

头文件 `#include <strings.h>`

原型 `int strcasecmp(const char *s1, const char *s2);`

说明 比较两个字符串，比较时忽略字母的大小写差异。

返回值 相同返回 0；`s1` 大于 `s2` 返回正值；小于则返回负值。

附加说明 属 POSIX 函数，适用于英文环境。

相关函数 `strncasecmp`、`strcmp`

```
#include <strings.h>
#include <stdio.h>

int main(void)
{
    printf("%d\n", strcasecmp("Hello", "hello"));   /* 0 */
    return 0;
}
```

strcat — 连接两字符串

头文件 `#include <string.h>`

原型 `char *strcat(char *dest, const char *src);`

说明 将 `src` 字符串连接到 `dest` 字符串的尾端，`dest` 原有的结尾 `'\0'` 会被覆盖。

返回值 返回 `dest` 字符串起始地址。

附加说明 `dest` 必须有足够的空间，否则会造成缓冲区溢出；建议改用 `strncat`。

相关函数 `strncat`、`strcpy`

```
#include <string.h>
#include <stdio.h>

int main(void)
{
    char d[32] = "hello ";
    strcat(d, "world");
    printf("%s\n", d);
    return 0;
}
```

strchr — 查找字符串中第一个出现的指定字符

头文件 `#include <string.h>`

原型 `char *strchr(const char *s, int c);`

说明 在字符串 `s` 中查找字符 `c` 第一次出现的位置。

返回值 找到则返回该地址，未找到返回 NULL。若 `c` 为 `'\0'`，则返回指向结尾 `'\0'` 的指针。

相关函数 `strrchr`、`index`、`strstr`

```
#include <string.h>
#include <stdio.h>

int main(void)
{
    char s[] = "0123456789";
    printf("%s\n", strchr(s, '5'));   /* 56789 */
    return 0;
}
```

strcmp — 比较字符串

头文件 `#include <string.h>`

原型 `int strcmp(const char *s1, const char *s2);`

说明 按字典顺序逐个比较两个字符串的字符，直到出现不同字符或遇到 `'\0'`。

返回值 相同返回 0；`s1` 大于 `s2` 返回正值；小于则返回负值。

相关函数 `strncmp`、`strcasecmp`、`strcoll`

```
#include <string.h>
#include <stdio.h>

int main(void)
{
    printf("%d %d\n", strcmp("abc", "abc"), strcmp("abc", "abd"));
    return 0;
}
```

strcoll — 采用目前区域的字符排列次序来比较字符串

头文件 `#include <string.h>`

原型 `int strcoll(const char *s1, const char *s2);`

说明 依当前区域（locale，由 `LC_COLLATE` 决定）的字符排列次序比较字符串。

返回值 相同返回 0，否则依排列次序返回正值或负值。

附加说明 默认（C locale）下与 `strcmp` 结果相同；使用前可用 `setlocale` 设置区域。

相关函数 `strcmp`、`setlocale`

```
#include <string.h>
#include <locale.h>
#include <stdio.h>

int main(void)
{
    setlocale(LC_COLLATE, "en_US.UTF-8");
    printf("%d\n", strcoll("abc", "ABC"));
    return 0;
}
```

strcpy — 拷贝字符串

头文件 `#include <string.h>`

原型 `char *strcpy(char *dest, const char *src);`

说明 将 `src` 所指的字符串（含结尾 `'\0'`）拷贝到 `dest`。

返回值 返回 `dest` 的地址。

附加说明 `dest` 空间不足会造成溢出；建议改用 `strncpy` 或 `snprintf`。

相关函数 `strncpy`、`memcpy`、`strdup`

```
#include <string.h>
#include <stdio.h>

int main(void)
{
    char d[32];
    strcpy(d, "hello strcpy");
    printf("%s\n", d);
    return 0;
}
```

strcspn — 返回字符串中连续不含指定字符串内容的字符数

头文件 `#include <string.h>`

原型 `size_t strcspn(const char *s, const char *reject);`

说明 从 `s` 的开头开始计算连续不含 `reject` 中出现字符的长度。

返回值 返回起始段中不含 `reject` 字符的字符数。

相关函数 `strspn`、`strpbrk`

```
#include <string.h>
#include <stdio.h>

int main(void)
{
    printf("%zu\n", strcspn("Linux C", " "));   /* 5 */
    return 0;
}
```

strdup — 复制字符串

头文件 `#include <string.h>`

原型 `char *strdup(const char *s);`

说明 先用 `malloc` 配置与 `s` 相同长度的空间，再将 `s` 的内容复制过去。

返回值 返回指向新字符串的指针，失败返回 NULL。

附加说明 返回的内存需由调用者用 `free` 释放。

相关函数 `strcpy`、`malloc`、`free`

```
#include <string.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    char *p = strdup("hello strdup");
    printf("%s\n", p);
    free(p);
    return 0;
}
```

strfry — 随机重组字符串内的字符

头文件 `#include <string.h>`

原型 `char *strfry(char *string);`

说明 随机重组（打乱）参数字符串内的字符顺序，结果因随机而不同。

返回值 返回参数 `string` 的地址。

附加说明 为 GNU 扩展函数，会直接修改原字符串；随机种子可用 `srand` 设置。

相关函数 `memfrob`、`rand`

```
#define _GNU_SOURCE
#include <string.h>
#include <stdio.h>

int main(void)
{
    char s[] = "abcdef";
    printf("%s\n", strfry(s));   /* 每次执行结果不同 */
    return 0;
}
```

strlen — 返回字符串长度

头文件 `#include <string.h>`

原型 `size_t strlen(const char *s);`

说明 返回字符串 `s` 的长度，即结尾 `'\0'` 之前的字符个数。

返回值 返回字符个数（不含结尾 `'\0'`）。

相关函数 `sizeof`、`strnlen`

```
#include <string.h>
#include <stdio.h>

int main(void)
{
    printf("%zu\n", strlen("hello"));   /* 5 */
    return 0;
}
```

strncasecmp — 忽略大小写比较字符串

头文件 `#include <strings.h>`

原型 `int strncasecmp(const char *s1, const char *s2, size_t n);`

说明 与 `strcasecmp` 相同，但只比较前 `n` 个字符。

返回值 前 `n` 个字符（忽略大小写）相同返回 0，否则返回正值或负值。

相关函数 `strcasecmp`、`strncmp`

```
#include <strings.h>
#include <stdio.h>

int main(void)
{
    printf("%d\n", strncasecmp("Hello", "HELL", 4));   /* 0 */
    return 0;
}
```

strncat — 连接两字符串

头文件 `#include <string.h>`

原型 `char *strncat(char *dest, const char *src, size_t n);`

说明 从 `src` 最多取 `n` 个字符连接到 `dest` 尾端，并在最后加上 `'\0'`，比 `strcat` 安全。

返回值 返回 `dest` 的地址。

相关函数 `strcat`、`strncpy`

```
#include <string.h>
#include <stdio.h>

int main(void)
{
    char d[8] = "ab";
    strncat(d, "cdefgh", 3);
    printf("%s\n", d);   /* abcde */
    return 0;
}
```

strncmp — 比较字符串

头文件 `#include <string.h>`

原型 `int strncmp(const char *s1, const char *s2, size_t n);`

说明 与 `strcmp` 相同，但只比较前 `n` 个字符。

返回值 前 `n` 个字符相同返回 0，否则返回正值或负值。

相关函数 `strcmp`、`strncasecmp`

```
#include <string.h>
#include <stdio.h>

int main(void)
{
    printf("%d\n", strncmp("abcdef", "abcxyz", 3));   /* 0 */
    return 0;
}
```

strncpy — 拷贝字符串

头文件 `#include <string.h>`

原型 `char *strncpy(char *dest, const char *src, size_t n);`

说明 从 `src` 最多拷贝 `n` 个字符到 `dest`。

返回值 返回 `dest` 的地址。

附加说明 若 `src` 长度小于 `n`，则以 `'\0'` 填满剩余空间；若 `src` 长度不小于 `n`，拷贝结果不会以 `'\0'` 结尾，需自行补上。

相关函数 `strcpy`、`memcpy`

```
#include <string.h>
#include <stdio.h>

int main(void)
{
    char d[8];
    strncpy(d, "abcdefghij", sizeof(d) - 1);
    d[sizeof(d) - 1] = '\0';
    printf("%s\n", d);   /* abcdefg */
    return 0;
}
```

strpbrk — 查找字符串中第一个出现的指定字符

头文件 `#include <string.h>`

原型 `char *strpbrk(const char *s, const char *accept);`

说明 在 `s` 中查找第一个属于 `accept` 字符串中任意字符的位置，即第一个“分隔字符”。

返回值 找到则返回该地址，未找到返回 NULL。

相关函数 `strchr`、`strcspn`、`strtok`

```
#include <string.h>
#include <stdio.h>

int main(void)
{
    printf("%s\n", strpbrk("hello,world;", ",;"));   /* ,world; */
    return 0;
}
```

strrchr — 查找字符串中最后一个出现的指定字符

头文件 `#include <string.h>`

原型 `char *strrchr(const char *s, int c);`

说明 在字符串 `s` 中查找字符 `c` 最后一次出现的位置。

返回值 找到则返回该地址，未找到返回 NULL。

相关函数 `strchr`、`rindex`

```
/* 取得路径中的文件名部分 */
#include <string.h>
#include <stdio.h>

int main(void)
{
    char path[] = "/usr/local/bin/gcc";
    printf("%s\n", strrchr(path, '/') + 1);   /* gcc */
    return 0;
}
```

strspn — 返回字符串中连续含指定字符串内容的字符数

头文件 `#include <string.h>`

原型 `size_t strspn(const char *s, const char *accept);`

说明 从 `s` 的开头计算连续由 `accept` 中字符组成的长度。

返回值 返回起始段中全部由 `accept` 字符组成的字符数。

相关函数 `strcspn`、`strpbrk`

```
#include <string.h>
#include <stdio.h>

int main(void)
{
    printf("%zu\n", strspn("4231abc", "0123456789"));   /* 4 */
    return 0;
}
```

strstr — 在一字符串中查找指定的字符串

头文件 `#include <string.h>`

原型 `char *strstr(const char *haystack, const char *needle);`

说明 在字符串 `haystack` 中查找子串 `needle` 第一次出现的位置。

返回值 找到则返回该地址，未找到返回 NULL；`needle` 为空字符串时返回 `haystack`。

相关函数 `strchr`、`index`

```
#include <string.h>
#include <stdio.h>

int main(void)
{
    printf("%s\n", strstr("hello world", "wor"));   /* world */
    return 0;
}
```

strtok — 分割字符串

头文件 `#include <string.h>`

原型 `char *strtok(char *str, const char *delim);`

说明 以 `delim` 中的字符为分隔符将字符串 `str` 分割成片段。第一次调用时传入 `str`，之后调用须传入 NULL 才能继续取得后续片段；函数会把分隔符替换为 `'\0'`。

返回值 返回下一个分割后的片段，无剩余片段时返回 NULL。

附加说明 会修改原字符串，且不可重入（多线程应使用 `strtok_r`）。

相关函数 `strsep`、`strpbrk`、`strstr`

```
/* 以逗号分割字符串 */
#include <string.h>
#include <stdio.h>

int main(void)
{
    char s[] = "apple,banana,orange";
    char *p = strtok(s, ",");
    while (p) {
        printf("%s\n", p);
        p = strtok(NULL, ",");
    }
    return 0;
}
```

第6章 数学计算函数

本章函数用于常见的数学运算，声明于 `<math.h>`（整数相关函数在 `<stdlib.h>`），编译时需加上 `-lm` 参数链接数学库。

abs — 计算整型数的绝对值

头文件 `#include <stdlib.h>`

原型 `int abs(int j);`

说明 计算参数 `j` 的绝对值。

返回值 返回 `j` 的绝对值。

相关函数 `labs`、`fabs`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    printf("%d\n", abs(-12));   /* 12 */
    return 0;
}
```

acos — 取反余弦函数值

头文件 `#include <math.h>`

原型 `double acos(double x);`

说明 计算 `x` 的反余弦值，参数范围为 -1 至 1，超出范围则返回 `NaN` 并设置 `errno` 为 `EDOM`。

返回值 返回 0 至 π 之间的弧度值。

相关函数 `asin`、`atan`、`cos`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    double v = acos(0.5);
    printf("%f rad = %f deg\n", v, v * 180 / M_PI);
    return 0;
}
```

asin — 取反正弦函数值

头文件 `#include <math.h>`

原型 `double asin(double x);`

说明 计算 `x` 的反正弦值，参数范围为 -1 至 1。

返回值 返回 -π/2 至 π/2 之间的弧度值；超出范围返回 `NaN` 并设置 `errno` 为 `EDOM`。

相关函数 `acos`、`atan`、`sin`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    printf("%f\n", asin(1.0));   /* 1.570796 */
    return 0;
}
```

atan — 取反正切函数值

头文件 `#include <math.h>`

原型 `double atan(double x);`

说明 计算 `x` 的反正切值，参数可为任意实数。

返回值 返回 -π/2 至 π/2 之间的弧度值。

相关函数 `atan2`、`tan`、`asin`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    printf("%f\n", atan(1.0));   /* 0.785398 */
    return 0;
}
```

atan2 — 取得反正切函数值

头文件 `#include <math.h>`

原型 `double atan2(double y, double x);`

说明 依据 `y/x` 的值计算反正切值，并依 `(x, y)` 所在象限决定结果的范围，因此结果落在 -π 至 π 之间。`x` 为 0 亦可正确运算。

返回值 返回 -π 至 π 之间的弧度值。

相关函数 `atan`、`tan`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    printf("%f\n", atan2(1.0, -1.0));   /* 2.356194（第二象限） */
    return 0;
}
```

ceil — 取不小于参数的最小整型数

头文件 `#include <math.h>`

原型 `double ceil(double x);`

说明 返回不小于 `x` 的最小整数值，以 `double` 表示（向上取整）。

返回值 返回不小于 `x` 的最小整数值。

相关函数 `floor`、`fabs`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    printf("%.0f %.0f\n", ceil(3.2), ceil(-3.2));   /* 4 -3 */
    return 0;
}
```

cos — 取余弦函数值

头文件 `#include <math.h>`

原型 `double cos(double x);`

说明 计算弧度参数 `x` 的余弦值。

返回值 返回 -1 至 1 之间的值。

相关函数 `sin`、`tan`、`acos`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    printf("%f\n", cos(M_PI / 3));   /* 0.500000 */
    return 0;
}
```

cosh — 取双曲线余弦函数值

头文件 `#include <math.h>`

原型 `double cosh(double x);`

说明 计算参数 `x` 的双曲线余弦值，即 `(e^x + e^-x) / 2`。

返回值 返回双曲线余弦值；结果过大时返回 `HUGE_VAL` 并设置 `errno` 为 `ERANGE`。

相关函数 `sinh`、`tanh`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    printf("%f\n", cosh(1.0));   /* 1.543081 */
    return 0;
}
```

div — 取得两整型数相除后的商及余数

头文件 `#include <stdlib.h>`

原型 `div_t div(int numerator, int denominator);`

说明 将 `numerator` 除以 `denominator`，结果以 `div_t` 结构返回，其中 `quot` 为商、`rem` 为余数。

返回值 返回 `div_t` 结构；若 `denominator` 为 0，结果未定义。

相关函数 `ldiv`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    div_t r = div(17, 5);
    printf("quot=%d rem=%d\n", r.quot, r.rem);   /* quot=3 rem=2 */
    return 0;
}
```

exp — 计算指数

头文件 `#include <math.h>`

原型 `double exp(double x);`

说明 计算自然常数 e 的 `x` 次方，即 e^x。

返回值 返回 e 的 `x` 次方值；结果过大时返回 `HUGE_VAL` 并设置 `errno` 为 `ERANGE`。

相关函数 `log`、`pow`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    printf("%f\n", exp(1.0));   /* 2.718282 */
    return 0;
}
```

fabs — 计算浮点型数的绝对值

头文件 `#include <math.h>`

原型 `double fabs(double x);`

说明 计算浮点型数 `x` 的绝对值。

返回值 返回 `x` 的绝对值。

相关函数 `abs`、`labs`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    printf("%f\n", fabs(-3.14));   /* 3.140000 */
    return 0;
}
```

frexp — 将浮点型数分为底数与指数

头文件 `#include <math.h>`

原型 `double frexp(double x, int *expptr);`

说明 将 `x` 分解为 `0.5 ≤ 底数 < 1` 的尾数与指数，使 `x = 底数 * 2^指数`，尾数作为返回值，指数存入 `expptr`。

返回值 返回尾数（0.5 至 1 之间）。

相关函数 `ldexp`、`modf`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    int e;
    double m = frexp(8.0, &e);
    printf("%f * 2^%d = %f\n", m, e, ldexp(m, e));
    return 0;
}
```

hypot — 计算直角三角形斜边长

头文件 `#include <math.h>`

原型 `double hypot(double x, double y);`

说明 计算两直角边长度为 `x` 与 `y` 的直角三角形斜边长，即 `sqrt(x*x + y*y)`，且在中间运算溢出时仍能给出正确结果。

返回值 返回斜边长；结果过大时返回 `HUGE_VAL`。

相关函数 `sqrt`、`pow`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    printf("%f\n", hypot(3.0, 4.0));   /* 5.000000 */
    return 0;
}
```

labs — 计算长整型数的绝对值

头文件 `#include <stdlib.h>`

原型 `long labs(long j);`

说明 计算长整型数 `j` 的绝对值。

返回值 返回 `j` 的绝对值。

相关函数 `abs`、`fabs`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    printf("%ld\n", labs(-1234567890L));
    return 0;
}
```

ldexp — 计算 2 的次方值

头文件 `#include <math.h>`

原型 `double ldexp(double x, int exp);`

说明 计算 `x` 乘以 2 的 `exp` 次方，即 `x * 2^exp`。

返回值 返回计算结果；结果过大时返回 `HUGE_VAL` 并设置 `errno` 为 `ERANGE`。

相关函数 `frexp`、`exp`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    printf("%f\n", ldexp(3.0, 2));   /* 12.000000 */
    return 0;
}
```

ldiv — 取得两长整数相除后的商及余数

头文件 `#include <stdlib.h>`

原型 `ldiv_t ldiv(long numerator, long denominator);`

说明 与 `div` 相同，但操作对象为长整型数，结果以 `ldiv_t` 结构返回（`quot` 为商，`rem` 为余数）。

返回值 返回 `ldiv_t` 结构。

相关函数 `div`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    ldiv_t r = ldiv(1000000L, 7L);
    printf("quot=%ld rem=%ld\n", r.quot, r.rem);
    return 0;
}
```

log — 计算以 e 为底的对数值

头文件 `#include <math.h>`

原型 `double log(double x);`

说明 计算 `x` 的自然对数（以 e 为底）。

返回值 返回对数值；`x` 为 0 时返回 `-HUGE_VAL` 并设置 `errno` 为 `ERANGE`，`x` 为负数时返回 `NaN` 并设置 `errno` 为 `EDOM`。

相关函数 `log10`、`exp`、`pow`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    printf("%f\n", log(M_E));   /* 1.000000 */
    return 0;
}
```

log10 — 计算以 10 为底的对数值

头文件 `#include <math.h>`

原型 `double log10(double x);`

说明 计算 `x` 以 10 为底的对数值。

返回值 返回对数值；`x` 为 0 或负数时的处理与 `log` 相同。

相关函数 `log`、`pow`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    printf("%f\n", log10(1000.0));   /* 3.000000 */
    return 0;
}
```

modf — 将浮点型数分解成整数与小数

头文件 `#include <math.h>`

原型 `double modf(double x, double *iptr);`

说明 将 `x` 分解为整数部分与小数部分，整数部分存入 `iptr` 所指位置，小数部分由函数返回。两部分的正负号都与 `x` 相同。

返回值 返回 `x` 的小数部分。

相关函数 `frexp`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    double ip;
    double fp = modf(3.14159, &ip);
    printf("int=%f frac=%f\n", ip, fp);
    return 0;
}
```

pow — 计算次方值

头文件 `#include <math.h>`

原型 `double pow(double x, double y);`

说明 计算 `x` 的 `y` 次方，即 x^y。

返回值 返回计算结果；`x` 为负数且 `y` 非整数时返回 `NaN` 并设置 `errno` 为 `EDOM`，结果过大或过小时设置 `errno` 为 `ERANGE`。

相关函数 `exp`、`sqrt`、`ldexp`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    printf("%f\n", pow(2.0, 10.0));   /* 1024.000000 */
    return 0;
}
```

sin — 取正弦函数值

头文件 `#include <math.h>`

原型 `double sin(double x);`

说明 计算弧度参数 `x` 的正弦值。

返回值 返回 -1 至 1 之间的值。

相关函数 `cos`、`tan`、`asin`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    printf("%f\n", sin(M_PI / 6));   /* 0.500000 */
    return 0;
}
```

sinh — 取双曲线正弦函数值

头文件 `#include <math.h>`

原型 `double sinh(double x);`

说明 计算参数 `x` 的双曲线正弦值，即 `(e^x - e^-x) / 2`。

返回值 返回双曲线正弦值；结果过大时返回 `HUGE_VAL` 并设置 `errno` 为 `ERANGE`。

相关函数 `cosh`、`tanh`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    printf("%f\n", sinh(1.0));   /* 1.175201 */
    return 0;
}
```

sqrt — 计算平方根值

头文件 `#include <math.h>`

原型 `double sqrt(double x);`

说明 计算 `x` 的平方根，参数不可为负数。

返回值 返回平方根值；`x` 为负数时返回 `NaN` 并设置 `errno` 为 `EDOM`。

相关函数 `hypot`、`pow`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    printf("%f\n", sqrt(2.0));   /* 1.414214 */
    return 0;
}
```

tan — 取正切函数值

头文件 `#include <math.h>`

原型 `double tan(double x);`

说明 计算弧度参数 `x` 的正切值。

返回值 返回正切值；当 `x` 接近 π/2 的奇数倍时结果趋于无穷大。

相关函数 `sin`、`cos`、`atan`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    printf("%f\n", tan(M_PI / 4));   /* 1.000000 */
    return 0;
}
```

tanh — 取双曲线正切函数值

头文件 `#include <math.h>`

原型 `double tanh(double x);`

说明 计算参数 `x` 的双曲线正切值，即 `sinh(x) / cosh(x)`。

返回值 返回 -1 至 1 之间的值。

相关函数 `sinh`、`cosh`

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    printf("%f\n", tanh(1.0));   /* 0.761594 */
    return 0;
}
```

第7章 用户和组函数

本章函数用于取得与设置用户、组以及登录记录等信息。多数函数需包含 `<sys/types.h>`、`<pwd.h>`、`<grp.h>`、`<utmp.h>` 等头文件。

cuserid — 取得用户帐号名称

头文件 `#include <stdio.h>`

原型 `char *cuserid(char *s);`

说明 取得当前用户的帐号名称。若参数 `s` 不为 NULL，则将名称写入该缓冲区（需至少 `L_cuserid` 字节）。

返回值 返回指向帐号名称字符串的指针；失败返回 NULL。

附加说明 建议使用 `getlogin` 或 `getpwuid(getuid())` 取代。

相关函数 `getlogin`、`getpwuid`

```
#include <stdio.h>

int main(void)
{
    printf("user = %s\n", cuserid(NULL));
    return 0;
}
```

endgrent — 关闭组文件

头文件

原型 `void endgrent(void);`

说明 关闭由 `getgrent` 打开的组文件（`/etc/group`），并释放相关资源。

返回值 无返回值。

相关函数 `getgrent`、`setgrent`

```
#include <sys/types.h>
#include <grp.h>
```

```
#include <sys/types.h>
#include <grp.h>

int main(void)
{
    struct group *g;
    while ((g = getgrent()) != NULL)
        printf("%s\n", g->gr_name);
    endgrent();   /* 用完务必关闭 */
    return 0;
}
```

endpwent — 关闭密码文件

头文件

原型 `void endpwent(void);`

说明 关闭由 `getpwent` 打开的密码文件（`/etc/passwd`）。

返回值 无返回值。

相关函数 `getpwent`、`setpwent`

```
#include <sys/types.h>
#include <pwd.h>
```

```
#include <sys/types.h>
#include <pwd.h>

int main(void)
{
    struct passwd *p;
    while ((p = getpwent()) != NULL)
        printf("%s\n", p->pw_name);
    endpwent();
    return 0;
}
```

endutent — 关闭 utmp 文件

头文件 `#include <utmp.h>`

原型 `void endutent(void);`

说明 关闭由 `getutent` 打开的 utmp 文件（`/var/run/utmp`）。

返回值 无返回值。

相关函数 `getutent`、`setutent`

```
#include <utmp.h>
#include <stdio.h>

int main(void)
{
    struct utmp *u;
    while ((u = getutent()) != NULL)
        printf("%s\n", u->ut_user);
    endutent();
    return 0;
}
```

`fgetgrent` — 从指定的文件来读取组格式

头文件

原型 `struct group *fgetgrent(FILE *stream);`

说明 从参数 `stream` 所指的文件中读取一行组格式数据，并填入 `struct group` 结构返回。

返回值 成功返回 `struct group` 指针，读到文件尾或失败返回 NULL。

相关函数 `getgrent`、`setgrent`

```
#include <stdio.h>
#include <sys/types.h>
#include <grp.h>
```

```
#include <stdio.h>
#include <sys/types.h>
#include <grp.h>

int main(void)
{
    FILE *fp = fopen("/etc/group", "r");
    struct group *g;
    while ((g = fgetgrent(fp)) != NULL)
        printf("%s gid=%d\n", g->gr_name, g->gr_gid);
    fclose(fp);
    return 0;
}
```

`fgetpwent` — 从指定的文件来读取密码格式

头文件

原型 `struct passwd *fgetpwent(FILE *stream);`

说明 从参数 `stream` 所指的文件中读取一行密码格式数据，并填入 `struct passwd` 结构返回。

返回值 成功返回 `struct passwd` 指针，读到文件尾或失败返回 NULL。

相关函数 `getpwent`、`setpwent`

```
#include <stdio.h>
#include <sys/types.h>
#include <pwd.h>
```

```
#include <stdio.h>
#include <sys/types.h>
#include <pwd.h>

int main(void)
{
    FILE *fp = fopen("/etc/passwd", "r");
    struct passwd *p;
    while ((p = fgetpwent(fp)) != NULL)
        printf("%s uid=%d\n", p->pw_name, p->pw_uid);
    fclose(fp);
    return 0;
}
```

getegid — 取得有效的组识别码

头文件

原型 `gid_t getegid(void);`

说明 取得当前进程的有效组识别码（effective gid）。

返回值 返回有效组识别码，此函数不会失败。

相关函数 `getgid`、`setegid`、`geteuid`

```
#include <unistd.h>
#include <sys/types.h>
```

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    printf("gid=%d egid=%d\n", getgid(), getegid());
    return 0;
}
```

geteuid — 取得有效的用户识别码

头文件

原型 `uid_t geteuid(void);`

说明 取得当前进程的有效用户识别码（effective uid），通常与 `getuid` 相同，执行 setuid 程序时可能不同。

返回值 返回有效用户识别码，此函数不会失败。

相关函数 `getuid`、`seteuid`

```
#include <unistd.h>
#include <sys/types.h>
```

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    printf("uid=%d euid=%d\n", getuid(), geteuid());
    return 0;
}
```

`getgid` — 取得真实的组识别码

头文件

原型 `gid_t getgid(void);`

说明 取得当前进程的真实组识别码（real gid）。

返回值 返回真实组识别码，此函数不会失败。

相关函数 `getegid`、`setgid`

```
#include <unistd.h>
#include <sys/types.h>
```

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    printf("gid = %d\n", getgid());
    return 0;
}
```

`getgrent` — 从组文件中取得帐号的数据

头文件

原型 `struct group *getgrent(void);`

说明 从组文件（`/etc/group`）中读取一项组数据并填入 `struct group`，第一次调用时会自动打开文件，连续调用可逐项读取。结构成员包括 `gr_name`（组名）、`gr_passwd`（组密码）、`gr_gid`（组识别码）、`gr_mem`（组成员列表）。

返回值 成功返回 `struct group` 指针，读到文件尾或失败返回 NULL。

附加说明 读取完毕后应调用 `endgrent` 关闭文件；可用 `setgrent` 从头读取。

相关函数 `setgrent`、`endgrent`、`getgrgid`

```
#include <sys/types.h>
#include <grp.h>
```

```
#include <sys/types.h>
#include <grp.h>
#include <stdio.h>

int main(void)
{
    struct group *g;
    while ((g = getgrent()) != NULL)
        printf("%s:%s:%d\n", g->gr_name, g->gr_passwd, g->gr_gid);
    endgrent();
    return 0;
}
```

`getgrgid` — 从组文件中取得指定 gid 的数据

头文件

原型 `struct group *getgrgid(gid_t gid);`

说明 在组文件中搜索组识别码为 `gid` 的项，并填入 `struct group` 返回。

返回值 成功返回 `struct group` 指针，找不到或失败返回 NULL。

相关函数 `getgrnam`、`getgrent`

```
#include <sys/types.h>
#include <grp.h>
```

```
#include <sys/types.h>
#include <grp.h>
#include <stdio.h>

int main(void)
{
    struct group *g = getgrgid(0);
    if (g)
        printf("gid 0 -> %s\n", g->gr_name);
    return 0;
}
```

getgrnam — 从组文件中取得指定组的数据

头文件

原型 `struct group *getgrnam(const char *name);`

说明 在组文件中搜索组名为 `name` 的项，并填入 `struct group` 返回。

返回值 成功返回 `struct group` 指针，找不到或失败返回 NULL。

相关函数 `getgrgid`、`getgrent`

```
#include <sys/types.h>
#include <grp.h>
```

```
#include <sys/types.h>
#include <grp.h>
#include <stdio.h>

int main(void)
{
    struct group *g = getgrnam("root");
    if (g)
        printf("root gid = %d\n", g->gr_gid);
    return 0;
}
```

getgroups — 取得组代码

头文件

原型 `int getgroups(int size, gid_t list[]);`

说明 取得当前用户所属的附加组识别码，存入 `list` 数组中；`size` 为数组可容纳的元素个数，设为 0 时函数仅返回组数。

返回值 成功返回组识别码个数，失败返回 -1 并设置 `errno`（`size` 不足时为 `EINVAL`）。

相关函数 `setgroups`、`initgroups`

```
#include <unistd.h>
#include <sys/types.h>
```

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    int n = getgroups(0, NULL);
    gid_t list[32];
    getgroups(n < 32 ? n : 32, list);
    for (int i = 0; i < n && i < 32; i++)
        printf("%d ", list[i]);
    printf("\n");
    return 0;
}
```

getlogin — 取得登录的用户帐号名称

头文件 `#include <unistd.h>`

原型 `char *getlogin(void);`

说明 取得当前登录用户的帐号名称。

返回值 成功返回指向名称字符串的指针，失败返回 NULL。

附加说明 返回的字符串为静态内存，后续调用可能被覆盖；不可靠时可改用 `getpwuid(getuid())`。

相关函数 `getpwnam`、`cuserid`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    printf("login = %s\n", getlogin());
    return 0;
}
```

`getpw` — 取得指定用户的密码文件数据

头文件

原型 `int getpw(uid_t uid, char *buf);`

说明 依据用户识别码 `uid` 从密码文件中取得该用户的一行数据，并以字符串形式存入 `buf`（需至少 `BUFSIZ` 字节）。

返回值 成功返回 0，失败返回 -1。

附加说明 已过时，建议使用 `getpwuid`。

相关函数 `getpwuid`、`getpwnam`

```
#include <sys/types.h>
#include <pwd.h>
```

```
#include <sys/types.h>
#include <pwd.h>
#include <stdio.h>

int main(void)
{
    char buf[BUFSIZ];
    if (getpw(0, buf) == 0)
        printf("%s\n", buf);
    return 0;
}
```

`getpwent` — 从密码文件中取得帐号的数据

头文件

原型 `struct passwd *getpwent(void);`

说明 从密码文件（`/etc/passwd`）读取一项用户数据并填入 `struct passwd`，连续调用可逐项读取。结构成员包括 `pw_name`（用户名）、`pw_passwd`（密码）、`pw_uid`、`pw_gid`、`pw_gecos`（用户信息）、`pw_dir`（家目录）、`pw_shell`。

返回值 成功返回 `struct passwd` 指针，读到文件尾或失败返回 NULL。

附加说明 读取完毕应调用 `endpwent` 关闭文件。

相关函数 `setpwent`、`endpwent`、`getpwnam`

```
#include <sys/types.h>
#include <pwd.h>
```

```
#include <sys/types.h>
#include <pwd.h>
#include <stdio.h>

int main(void)
{
    struct passwd *p;
    while ((p = getpwent()) != NULL)
        printf("%s uid=%d home=%s\n", p->pw_name, p->pw_uid, p->pw_dir);
    endpwent();
    return 0;
}
```

`getpwnam` — 从密码文件中取得指定帐号的数据

头文件

原型 `struct passwd *getpwnam(const char *name);`

说明 在密码文件中搜索用户名为 `name` 的项，并填入 `struct passwd` 返回。

返回值 成功返回 `struct passwd` 指针，找不到或失败返回 NULL。

相关函数 `getpwuid`、`getpwent`

```
#include <sys/types.h>
#include <pwd.h>
```

```
#include <sys/types.h>
#include <pwd.h>
#include <stdio.h>

int main(void)
{
    struct passwd *p = getpwnam("root");
    if (p)
        printf("uid=%d home=%s shell=%s\n", p->pw_uid, p->pw_dir, p->pw_shell);
    return 0;
}
```

getpwuid — 从密码文件中取得指定 uid 的数据

头文件

原型 `struct passwd *getpwuid(uid_t uid);`

说明 在密码文件中搜索用户识别码为 `uid` 的项，并填入 `struct passwd` 返回。

返回值 成功返回 `struct passwd` 指针，找不到或失败返回 NULL。

相关函数 `getpwnam`、`getpwent`

```
#include <sys/types.h>
#include <pwd.h>
```

```
#include <sys/types.h>
#include <pwd.h>
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    struct passwd *p = getpwuid(getuid());
    if (p)
        printf("I am %s\n", p->pw_name);
    return 0;
}
```

getuid — 取得真实的用户识别码

头文件

原型 `uid_t getuid(void);`

说明 取得当前进程的真实用户识别码（real uid）。

返回值 返回真实用户识别码，此函数不会失败。

相关函数 `geteuid`、`setuid`

```
#include <unistd.h>
#include <sys/types.h>
```

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    printf("uid = %d\n", getuid());
    return 0;
}
```

`getutent` — 从 utmp 文件中取得帐号登录数据

头文件 `#include <utmp.h>`

原型 `struct utmp *getutent(void);`

说明 从 utmp 文件（`/var/run/utmp`）中读取一项登录记录并填入 `struct utmp`，连续调用可逐项读取。结构中常用成员有 `ut_type`（记录类型）、`ut_pid`、`ut_line`（终端名）、`ut_user`（用户名）、`ut_host`（远程主机）、`ut_time`（登录时间）。

返回值 成功返回 `struct utmp` 指针，读到文件尾或失败返回 NULL。

附加说明 使用完毕应调用 `endutent` 关闭文件。

相关函数 `setutent`、`endutent`、`getutid`

```
#include <utmp.h>
#include <stdio.h>

int main(void)
{
    struct utmp *u;
    while ((u = getutent()) != NULL)
        if (u->ut_type == USER_PROCESS)
            printf("%s on %s\n", u->ut_user, u->ut_line);
    endutent();
    return 0;
}
```

getutid — 从 utmp 文件中查找特定的记录

头文件 `#include <utmp.h>`

原型 `struct utmp *getutid(const struct utmp *ut);`

说明 从 utmp 文件的当前读写位置开始向后搜索与 `ut` 中指定条件相符的记录。`ut->ut_type` 为 `RUN_LVL`、`BOOT_TIME`、`NEW_TIME` 或 `OLD_TIME` 时按 `ut_type` 查找；为 `INIT_PROCESS`、`LOGIN_PROCESS`、`USER_PROCESS` 或 `DEAD_PROCESS` 时按 `ut_id` 查找。

返回值 成功返回相符的 `struct utmp` 指针，找不到返回 NULL。

相关函数 `getutent`、`getutline`、`pututline`

```
#include <utmp.h>
#include <stdio.h>

int main(void)
{
    struct utmp u = {0};
    u.ut_type = RUN_LVL;
    struct utmp *r = getutid(&u);
    printf("runlevel = %s\n", r ? r->ut_line : "unknown");
    return 0;
}
```

getutline — 从 utmp 文件中查找特定的记录

头文件 `#include <utmp.h>`

原型 `struct utmp *getutline(const struct utmp *ut);`

说明 从 utmp 文件的当前读写位置向后搜索 `ut_type` 为 `LOGIN_PROCESS` 或 `USER_PROCESS`，且 `ut_line` 与 `ut` 相符的记录。

返回值 成功返回 `struct utmp` 指针，找不到返回 NULL。

相关函数 `getutid`、`pututline`

```
#include <utmp.h>
#include <stdio.h>

int main(void)
{
    struct utmp u = {0};
    u.ut_type = USER_PROCESS;
    snprintf(u.ut_line, sizeof(u.ut_line), "pts/0");
    struct utmp *r = getutline(&u);
    printf("user = %s\n", r ? r->ut_user : "none");
    return 0;
}
```

initgroups — 初始化组清单

头文件

原型 `int initgroups(const char *user, gid_t group);`

说明 读取组文件，将 `user` 所属的所有组与附加的 `group` 一起设置为当前进程的组清单。

返回值 成功返回 0，失败返回 -1 并设置 `errno` 为 `EPERM`。

附加说明 需 root 权限；常与 `setgid`、`setuid` 配合使用。

相关函数 `setgroups`、`getgroups`

```
#include <grp.h>
#include <sys/types.h>
```

```
#include <grp.h>
#include <stdio.h>

int main(void)   /* 需 root 权限 */
{
    if (initgroups("root", 0) < 0)
        perror("initgroups");
    return 0;
}
```

logwtmp — 将一登录数据记录到 wtmp 文件

头文件 `#include <utmp.h>`

原型 `void logwtmp(const char *line, const char *name, const char *host);`

说明 将一条登录或注销记录写入 wtmp 文件（`/var/log/wtmp`）。`name` 为 NULL 或 `line` 为空字符串时表示注销记录。

返回值 无返回值。

附加说明 为 BSD 兼容函数，Linux 下建议使用 `updwtmp`。

相关函数 `updwtmp`、`pututline`

```
#include <utmp.h>

int main(void)
{
    logwtmp("pts/0", "user", "localhost");
    logwtmp("pts/0", "", "");           /* 注销记录 */
    return 0;
}
```

`pututline` — 将 utmp 记录写入文件

头文件 `#include <utmp.h>`

原型 `void pututline(const struct utmp *ut);`

说明 将 `ut` 所指的 utmp 记录写入 utmp 文件。写入前先用 `getutid` 定位可避免重复追加。

返回值 无返回值。

相关函数 `getutent`、`getutid`、`updwtmp`

```
#include <utmp.h>
#include <string.h>
#include <time.h>

int main(void)
{
    struct utmp u = {0};
    u.ut_type = USER_PROCESS;
    u.ut_pid = getpid();
    strncpy(u.ut_line, "pts/0", sizeof(u.ut_line) - 1);
    strncpy(u.ut_user, "tester", sizeof(u.ut_user) - 1);
    u.ut_time = time(NULL);
    pututline(&u);
    endutent();
    return 0;
}
```

setegid — 设置有效的组识别码

头文件

原型 `int setegid(gid_t egid);`

说明 设置当前进程的有效组识别码为 `egid`。

返回值 成功返回 0，失败返回 -1 并设置 `errno` 为 `EPERM`。

相关函数 `setgid`、`setregid`、`getegid`

```
#include <unistd.h>
#include <sys/types.h>
```

```
#include <unistd.h>
#include <stdio.h>

int main(void)   /* 需相应权限 */
{
    if (setegid(0) < 0)
        perror("setegid");
    printf("egid=%d\n", getegid());
    return 0;
}
```

seteuid — 设置有效的用户识别码

头文件

原型 `int seteuid(uid_t euid);`

说明 设置当前进程的有效用户识别码为 `euid`。

返回值 成功返回 0，失败返回 -1 并设置 `errno` 为 `EPERM`。

相关函数 `setuid`、`setreuid`、`geteuid`

```
#include <unistd.h>
#include <sys/types.h>
```

```
#include <unistd.h>
#include <stdio.h>

int main(void)   /* 需 root 权限 */
{
    if (seteuid(0) < 0)
        perror("seteuid");
    printf("euid=%d\n", geteuid());
    return 0;
}
```

setfsgid — 设置文件系统的组识别码

头文件 `#include <sys/fsuid.h>`

原型 `int setfsgid(gid_t fsgid);`

说明 设置进程进行文件系统权限检查时使用的组识别码，通常与有效 gid 相同，Linux 下可单独设置。

返回值 返回设置前的文件系统组识别码。

附加说明 为 Linux 特有函数，一般程序无须使用。

相关函数 `setfsuid`、`setgid`

```
#include <sys/fsuid.h>
#include <stdio.h>

int main(void)
{
    printf("old fsgid=%d\n", setfsgid(0));
    return 0;
}
```

setfsuid — 设置文件系统的用户识别码

头文件 `#include <sys/fsuid.h>`

原型 `int setfsuid(uid_t fsuid);`

说明 设置进程进行文件系统权限检查时使用的用户识别码。

返回值 返回设置前的文件系统用户识别码。

附加说明 为 Linux 特有函数。

相关函数 `setfsgid`、`setuid`

```
#include <sys/fsuid.h>
#include <stdio.h>

int main(void)
{
    printf("old fsuid=%d\n", setfsuid(0));
    return 0;
}
```

`setgid` — 设置真实的组识别码

头文件

原型 `int setgid(gid_t gid);`

说明 设置当前进程的真实组识别码为 `gid`（超级用户可同时设置有效与保存的 gid）。

返回值 成功返回 0，失败返回 -1 并设置 `errno` 为 `EPERM`。

相关函数 `setegid`、`setregid`、`getgid`

```
#include <unistd.h>
#include <sys/types.h>
```

```
#include <unistd.h>
#include <stdio.h>

int main(void)   /* 需 root 权限 */
{
    if (setgid(0) < 0)
        perror("setgid");
    return 0;
}
```

setgrent — 从头读取组文件中的组数据

头文件

原型 `void setgrent(void);`

说明 将组文件的读写位置重设到文件开头，以便从头读取组数据。

返回值 无返回值。

相关函数 `getgrent`、`endgrent`

```
#include <sys/types.h>
#include <grp.h>
```

```
#include <sys/types.h>
#include <grp.h>
#include <stdio.h>

int main(void)
{
    struct group *g = getgrent();
    printf("first: %s\n", g->gr_name);
    setgrent();                 /* 回到开头 */
    g = getgrent();
    printf("again: %s\n", g->gr_name);
    endgrent();
    return 0;
}
```

`setgroups` — 设置组代码

头文件

原型 `int setgroups(size_t size, const gid_t *list);`

说明 将当前进程的附加组清单设置为 `list` 中的 `size` 个组识别码。

返回值 成功返回 0，失败返回 -1 并设置 `errno` 为 `EPERM`（权限不足）或 `EINVAL`。

附加说明 需 root 权限。

相关函数 `getgroups`、`initgroups`

```
#include <grp.h>
#include <sys/types.h>
```

```
#include <grp.h>
#include <stdio.h>

int main(void)   /* 需 root 权限 */
{
    gid_t list[] = {0};
    if (setgroups(1, list) < 0)
        perror("setgroups");
    return 0;
}
```

setpwent — 从头读取密码文件中的帐号数据

头文件

原型 `void setpwent(void);`

说明 将密码文件的读写位置重设到文件开头。

返回值 无返回值。

相关函数 `getpwent`、`endpwent`

```
#include <sys/types.h>
#include <pwd.h>
```

```
#include <sys/types.h>
#include <pwd.h>
#include <stdio.h>

int main(void)
{
    struct passwd *p = getpwent();
    printf("first: %s\n", p->pw_name);
    setpwent();
    p = getpwent();
    printf("again: %s\n", p->pw_name);
    endpwent();
    return 0;
}
```

setregid — 设置真实及有效的组识别码

头文件 `#include <unistd.h>`

原型 `int setregid(gid_t rgid, gid_t egid);`

说明 同时设置当前进程的真实组识别码与有效组识别码；参数为 -1 表示该项保持不变。

返回值 成功返回 0，失败返回 -1 并设置 `errno` 为 `EPERM`。

相关函数 `setgid`、`setegid`、`setreuid`

```
#include <unistd.h>
#include <stdio.h>

int main(void)   /* 需相应权限 */
{
    if (setregid(-1, 0) < 0)     /* 仅改有效 gid */
        perror("setregid");
    return 0;
}
```

`setreuid` — 设置真实及有效的用户识别码

头文件	`#include <unistd.h>`
原型	`int setreuid(uid_t ruid, uid_t euid);`
说明	同时设置当前进程的真实用户识别码与有效用户识别码；参数为 -1 表示该项保持不变。
返回值	成功返回 0，失败返回 -1 并设置 `errno` 为 `EPERM`。
附加说明	常用来在 root 与一般用户身份之间临时切换（丢弃权限再恢复）。
相关函数	`setuid`、`seteuid`、`setregid`

```
#include <unistd.h>
#include <stdio.h>

int main(void)   /* 需 root 权限 */
{
    uid_t old = geteuid();
    setreuid(-1, 1000);         /* 降权 */
    printf("now euid=%d\n", geteuid());
    setreuid(-1, old);          /* 恢复 */
    return 0;
}
```

`setuid` — 设置真实的用户识别码

头文件

原型 `int setuid(uid_t uid);`

说明 设置当前进程的用户识别码。超级用户可任意设置真实、有效与保存的 uid；一般用户仅能设置为自己的 uid。

返回值 成功返回 0，失败返回 -1 并设置 `errno` 为 `EPERM` 或 `EINVAL`。

附加说明 一旦超级用户降权为一般用户，便无法自行恢复，需用 `setreuid` 或 `seteuid` 保留恢复能力。

相关函数 `seteuid`、`setreuid`、`getuid`

```
#include <unistd.h>
#include <sys/types.h>
```

```
#include <unistd.h>
#include <stdio.h>

int main(void)   /* 需 root 权限 */
{
    if (setuid(1000) < 0)
        perror("setuid");
    printf("uid = %d\n", getuid());
    return 0;
}
```

`setutent` — 从头读取 utmp 文件中的登录数据

头文件 `#include <utmp.h>`

原型 `void setutent(void);`

说明 将 utmp 文件的读写位置重设到文件开头，并视需要重新打开文件。

返回值 无返回值。

相关函数 `getutent`、`endutent`

```
#include <utmp.h>
#include <stdio.h>

int main(void)
{
    setutent();
    struct utmp *u = getutent();
    while (u) {
        printf("%s\n", u->ut_line);
        u = getutent();
    }
    endutent();
    return 0;
}
```

updwtmp — 将一登录数据记录到 wtmp 文件

头文件 `#include <utmp.h>`

原型 `void updwtmp(const char *wtmp_file, const struct utmp *ut);`

说明 将 `ut` 所指的登录记录写入参数 `wtmp_file` 指定的文件（通常为 `/var/log/wtmp`）。

返回值 无返回值。

附加说明 为 Linux 特有函数；相关函数还有更新 lastlog 的 `updwtmpx`、`logwtmp`。

相关函数 `logwtmp`、`pututline`

```
#include <utmp.h>
#include <string.h>
#include <time.h>

int main(void)
{
    struct utmp u = {0};
    u.ut_type = USER_PROCESS;
    strncpy(u.ut_user, "tester", sizeof(u.ut_user) - 1);
    u.ut_time = time(NULL);
    updwtmp("/var/log/wtmp", &u);
    return 0;
}
```

utmpname — 设置 utmp 文件路径

头文件 `#include <utmp.h>`

原型 `void utmpname(const char *file);`

说明 设置 utmp 函数（`getutent`、`pututline` 等）所要访问的文件路径，默认值为 `/var/run/utmp`。

返回值 无返回值。

附加说明 应在调用其他 utmp 函数之前设置。

相关函数 `getutent`、`setutent`、`endutent`

```
#include <utmp.h>
#include <stdio.h>

int main(void)
{
    utmpname("/var/log/wtmp");   /* 改为读取 wtmp */
    setutent();
    struct utmp *u = getutent();
    printf("%s\n", u ? u->ut_line : "empty");
    endutent();
    return 0;
}
```

第8章 数据加密函数

crypt — 将密码或数据编码

头文件 `#include <crypt.h>`（编译时需加 `-lcrypt`）

原型 `char *crypt(const char *key, const char *salt);`

说明 以 DES 或其他算法将 `key` 编码，返回 13 个字符的单向散列字符串。`salt` 为两个字符的扰动字符串（可取 `a-zA-Z0-9./`），也可使用 `$1$`、`$5$`、`$6$` 等前缀指定 MD5、SHA-256、SHA-512 算法。常用于与 `/etc/shadow` 中的密码字段比对。

返回值 返回编码后的字符串指针；内容为静态内存，每次调用会被覆盖。失败返回 NULL。

附加说明 为单向编码，无法由结果反推原文；相同 `key` 与 `salt` 会产生相同结果。

相关函数 `getpass`

```
#include <crypt.h>
#include <stdio.h>

int main(void)
{
    char *p = crypt("mypass", "ab");
    printf("%s\n", p);
    /* 校验：把用户输入加密后与存储的密文比较 */
    printf("%s\n", strcmp(crypt("mypass", p), p) == 0 ? "OK" : "FAIL");
    return 0;
}
```

getpass — 取得一密码输入

头文件 `#include <unistd.h>`

原型 `char *getpass(const char *prompt);`

说明 在终端显示 `prompt` 提示字符串，读取用户输入的一行密码，并且不回显输入内容。

返回值 返回指向密码字符串的指针；内容为静态内存，长度通常限制为 128 字节以内。

附加说明 已过时且不适用于多线程，建议使用 `termios` 关闭 `ECHO` 自行实现。

相关函数 `crypt`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    char *p = getpass("Input password: ");
    printf("\nyour password is: %s\n", p);
    return 0;
}
```

第9章 数据结构函数

本章函数提供搜索、排序、队列与树等常用数据结构的实现。

bsearch — 二元搜索

头文件 `#include <stdlib.h>`

原型 `void *bsearch(const void *key, const void *base, size_t nmemb, size_t size, int (*compar)(const void *, const void *));`

说明 在已排序的数组 `base`（含 `nmemb` 个元素，每个元素 `size` 字节）中以二分法查找与 `key` 相符的元素，比较方式由 `compar` 函数决定。

返回值 找到则返回该元素的地址，未找到或数组为空返回 NULL。

附加说明 数组必须事先排序，且排序与查找应使用相同的比较函数；通常用 `qsort` 排序。

相关函数 `qsort`、`lfind`、`lsearch`

```
#include <stdlib.h>
#include <stdio.h>

int cmpint(const void *a, const void *b)
{
    return *(const int *)a - *(const int *)b;
}

int main(void)
{
    int arr[] = {1, 3, 5, 7, 9};
    int key = 7;
    int *p = bsearch(&key, arr, 5, sizeof(int), cmpint);
    printf("%s\n", p ? "found" : "not found");
    return 0;
}
```

hcreate — 建立哈希表

头文件 `#include <search.h>`

原型 `int hcreate(size_t nel);`

说明 建立可容纳 `nel` 个元素的哈希表。使用前必须先调用本函数。

返回值 成功返回非 0，失败返回 0。

附加说明 表中数据为 `ENTRY` 结构（含 `char *key` 与 `void *data`）；同名 key 只能存在一个。

相关函数 `hsearch`、`hdestroy`

```
#include <search.h>
#include <stdio.h>
#include <string.h>

int main(void)
{
    ENTRY e, *ep;
    hcreate(30);
    e.key = "apple"; e.data = (void *)"red";
    hsearch(e, ENTER);
    e.key = "apple";
    ep = hsearch(e, FIND);
    printf("%s -> %s\n", ep->key, (char *)ep->data);
    hdestroy();
    return 0;
}
```

hdestroy — 删除哈希表

头文件 `#include <search.h>`

原型 `void hdestroy(void);`

说明 删除由 `hcreate` 建立的哈希表，释放相关内存。表中 key 与 data 所指的内存需自行释放。

返回值 无返回值。

相关函数 `hcreate`、`hsearch`

```
#include <search.h>

int main(void)
{
    hcreate(10);
    /* ... 使用哈希表 ... */
    hdestroy();
    return 0;
}
```

hsearch — 哈希表搜索

头文件 `#include <search.h>`

原型 `ENTRY *hsearch(ENTRY item, ACTION action);`

说明 在哈希表中搜索 `item.key`。`action` 为 `FIND` 时仅查找；为 `ENTER` 时若找不到则把该元素加入表中（key 需为静态或长期有效的字符串）。

返回值 `FIND` 时找到返回元素地址，找不到返回 NULL；`ENTER` 时若表已满则失败返回 NULL。

附加说明 使用前需先调用 `hcreate`；本函数不可重入。

相关函数 `hcreate`、`hdestroy`、`tsearch`

```
#include <search.h>
#include <stdio.h>

int main(void)
{
    ENTRY e, *ep;
    hcreate(30);
    e.key = "linux"; e.data = (void *)"kernel";
    hsearch(e, ENTER);
    e.key = "linux";
    ep = hsearch(e, FIND);
    printf("%s -> %s\n", ep->key, (char *)ep->data);
    hdestroy();
    return 0;
}
```

insque — 加入一项目至队列中

头文件 `#include <search.h>`

原型 `void insque(void *elem, void *prev);`

说明 将 `elem` 所指的项目插入到双向队列中 `prev` 之后。队列项目的头两个成员必须为指向前一项与后一项的指针。

返回值 无返回值。

附加说明 `prev` 为 NULL 时表示将 `elem` 作为队列头插入。

相关函数 `remque`

```
#include <search.h>
#include <stdio.h>
#include <stdlib.h>

struct node {
    struct node *next, *prev;   /* 必须是前两个成员 */
    int value;
};

int main(void)
{
    struct node a = {NULL, NULL, 1};
    struct node b = {NULL, NULL, 2};
    insque(&a, NULL);     /* a 作为队头 */
    insque(&b, &a);       /* b 插到 a 之后 */
    printf("%d %d\n", a.value, a.next->value);
    return 0;
}
```

`lfind` — 线性搜索

头文件 `#include <search.h>`

原型 `void *lfind(const void *key, const void *base, size_t *nmemb, size_t size, int (*compar)(const void *, const void *));`

说明 在数组 `base` 中以线性方式逐个查找与 `key` 相符的元素，`*nmemb` 为元素个数，`size` 为每个元素的大小。数组无需事先排序。

返回值 找到返回该元素地址，未找到返回 NULL。

附加说明 与 `lsearch` 的差别在于找不到时不加入新元素。

相关函数 `lsearch`、`bsearch`

```
#include <search.h>
#include <stdio.h>

int cmpint(const void *a, const void *b)
{
    return *(const int *)a - *(const int *)b;
}

int main(void)
{
    int arr[] = {5, 2, 9, 1};
    size_t n = 4;
    int key = 9;
    int *p = lfind(&key, arr, &n, sizeof(int), cmpint);
    printf("%s\n", p ? "found" : "not found");
    return 0;
}
```

lsearch — 线性搜索

头文件 `#include <search.h>`

原型 `void *lsearch(const void *key, void *base, size_t *nmemb, size_t size, int (*compar)(const void *, const void *));`

说明 与 `lfind` 相同，但找不到相符元素时会将 `key` 追加到数组末尾，并把 `*nmemb` 加 1。

返回值 返回找到（或新加入）的元素地址。

附加说明 `base` 所指的数组必须预留足够的空间。

相关函数 `lfind`、`bsearch`

```
#include <search.h>
#include <stdio.h>

int cmpint(const void *a, const void *b)
{
    return *(const int *)a - *(const int *)b;
}

int main(void)
{
    int arr[8] = {5, 2, 9};
    size_t n = 3;
    int key = 7;
    lsearch(&key, arr, &n, sizeof(int), cmpint);
    for (size_t i = 0; i < n; i++)
        printf("%d ", arr[i]);   /* 5 2 9 7 */
    printf("\n");
    return 0;
}
```

`qsort` — 利用快速排序法排列数组

头文件 `#include <stdlib.h>`

原型 `void qsort(void *base, size_t nmemb, size_t size, int (*compar)(const void *, const void *));`

说明 以快速排序法排列 `base` 所指的数组，共 `nmemb` 个元素，每个 `size` 字节。比较函数 `compar` 第一个参数大于、等于、小于第二个参数时，应分别返回正值、0、负值。

返回值 无返回值。

附加说明 排序结果不稳定（相等元素的相对次序可能改变）；需要稳定排序可使用 `qsort` 之外的实现或在比较函数中加入序号。

相关函数 `bsearch`

```
#include <stdlib.h>
#include <stdio.h>

int cmpint(const void *a, const void *b)
{
    return *(const int *)a - *(const int *)b;
}

int main(void)
{
    int arr[] = {5, 3, 9, 1, 7};
    qsort(arr, 5, sizeof(int), cmpint);
    for (int i = 0; i < 5; i++)
        printf("%d ", arr[i]);   /* 1 3 5 7 9 */
    printf("\n");
    return 0;
}
```

remque — 从队列中删除一项目

头文件 `#include <search.h>`

原型 `void remque(void *elem);`

说明 将 `elem` 所指的项目从双向队列中删除，并调整前后项目的链接。

返回值 无返回值。

相关函数 `insque`

```
#include <search.h>
#include <stdio.h>

struct node {
    struct node *next, *prev;
    int value;
};

int main(void)
{
    struct node a = {NULL, NULL, 1};
    struct node b = {NULL, NULL, 2};
    insque(&a, NULL);
    insque(&b, &a);
    remque(&a);                         /* 删除 a */
    printf("head value = %d\n", b.value);
    return 0;
}
```

`tdelete` — 从二叉树中删除数据

头文件 `#include <search.h>`

原型 `void *tdelete(const void *key, void **rootp, int (*compar)(const void *, const void *));`

说明 从 `rootp` 所指的二叉搜索树中删除与 `key` 相符的节点，并调整树的结构。若无相符节点则不执行删除。

返回值 返回被删除节点的父节点地址；未找到相符节点返回 NULL；树变为空时返回指向根（内容为 NULL）的指针。

相关函数 `tsearch`、`tfind`、`twalk`

```
#include <search.h>
#include <stdio.h>

int cmpstr(const void *a, const void *b)
{
    return strcmp(a, b);
}

int main(void)
{
    void *root = NULL;
    tsearch("banana", &root, cmpstr);
    tsearch("apple",  &root, cmpstr);
    tdelete("banana", &root, cmpstr);
    printf("%s\n", tfind("banana", &root, cmpstr) ? "still here" : "deleted");
    return 0;
}
```

`tfind` — 搜索二叉树

头文件 `#include <search.h>`

原型 `void *tfind(const void *key, const void **rootp, int (*compar)(const void *, const void *));`

说明 在 `rootp` 所指的二叉搜索树中查找与 `key` 相符的节点。

返回值 找到则返回指向该节点的指针（节点内容为指向数据项的指针），未找到返回 NULL。

相关函数 `tsearch`、`tdelete`、`twalk`

```
#include <search.h>
#include <stdio.h>
#include <string.h>

int cmpstr(const void *a, const void *b) { return strcmp(a, b); }

int main(void)
{
    void *root = NULL;
    tsearch("linux", &root, cmpstr);
    void *r = tfind("linux", (const void **)&root, cmpstr);
    printf("%s\n", r ? *(char **)r : "not found");
    return 0;
}
```

tsearch — 二叉树

头文件 `#include <search.h>`

原型 `void *tsearch(const void *key, void **rootp, int (*compar)(const void *, const void *));`

说明 在 `rootp` 所指的二叉搜索树中查找与 `key` 相符的节点，找不到时自动建立并插入新节点；`rootp` 指向 NULL 表示空树。

返回值 返回指向该节点的指针（已存在或新建），内存不足时返回 NULL。

相关函数 `tfind`、`tdelete`、`twalk`

```
#include <search.h>
#include <stdio.h>
#include <string.h>

int cmpstr(const void *a, const void *b) { return strcmp(a, b); }

int main(void)
{
    void *root = NULL;
    tsearch("dog", &root, cmpstr);
    tsearch("cat", &root, cmpstr);
    tsearch("bird", &root, cmpstr);
    printf("%s\n", *(char **)tfind("cat", (const void **)&root, cmpstr));
    return 0;
}
```

twalk — 走访二叉树

头文件 `#include <search.h>`

原型 `void twalk(const void *root, void (*action)(const void *nodep, const VISIT which, const int depth));`

说明 走访（遍历）`root` 所指的二叉树，每访问一个节点便调用一次 `action`。`which` 的取值为 `preorder`（首次到达）、`postorder`（离开该节点前）、`endorder`（处理完左右子树后）、`leaf`（叶子节点）。

返回值 无返回值。

相关函数 `tsearch`、`tfind`、`tdelete`

```
#include <search.h>
#include <stdio.h>
#include <string.h>

int cmpstr(const void *a, const void *b) { return strcmp(a, b); }

void print_node(const void *nodep, const VISIT which, const int depth)
{
    if (which == postorder || which == leaf)
        printf("depth=%d %s\n", depth, *(char **)nodep);
}

int main(void)
{
    void *root = NULL;
    tsearch("dog", &root, cmpstr);
    tsearch("cat", &root, cmpstr);
    tsearch("bird", &root, cmpstr);
    twalk(root, print_node);
    return 0;
}
```

第10章 随机数函数

本章函数用于产生随机数与设置随机数种子。`rand` 系列声明于 `<stdlib.h>`，`random` 与 48 位系列函数亦需包含 `<stdlib.h>`。

drand48 — 产生一个正的浮点型随机数

头文件 `#include <stdlib.h>`

原型 `double drand48(void);`

说明 以 48 位线性同余算法产生一个 [0.0, 1.0) 之间的双精度浮点随机数，并自动更新内部状态。

返回值 返回 0.0 至 1.0 之间（不含 1.0）的随机数。

相关函数 `srand48`、`lrand48`、`erand48`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    srand48(1234);
    for (int i = 0; i < 3; i++)
        printf("%f\n", drand48());
    return 0;
}
```

erand48 — 产生一个正的浮点型随机数

头文件 `#include <stdlib.h>`

原型 `double erand48(unsigned short xsubi[3]);`

说明 与 `drand48` 相同，但使用调用者提供的 48 位状态数组 `xsubi`，并就地更新该数组，因此适用于需要独立随机序列的场合（可重入）。

返回值 返回 [0.0, 1.0) 之间的随机数。

相关函数 `drand48`、`nrand48`、`jrand48`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    unsigned short state[3] = {1, 2, 3};
    for (int i = 0; i < 3; i++)
        printf("%f\n", erand48(state));
    return 0;
}
```

initstate — 建立随机数状态数组

头文件 `#include <stdlib.h>`

原型 `char *initstate(unsigned int seed, char *state, size_t n);`

说明 建立 `random` 使用的随机数状态数组。`seed` 为随机数种子，`state` 为状态数组，`n` 为数组大小（可为 8、32、64、128、256 字节，越大产生的序列越随机）。

返回值 成功返回前一个状态数组的指针，失败返回 NULL。

相关函数 `random`、`setstate`、`srandom`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    char state[256];
    initstate(1, state, sizeof(state));
    printf("%ld\n", random());
    return 0;
}
```

jrand48 — 产生一个长整型数随机数

头文件 `#include <stdlib.h>`

原型 `long jrand48(unsigned short xsubi[3]);`

说明 使用调用者提供的 48 位状态数组产生一个 32 位有符号长整型随机数（可为正或负），并更新状态数组。

返回值 返回 -2^31 至 2^31-1 之间的长整型随机数。

相关函数 `mrand48`、`nrand48`、`erand48`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    unsigned short state[3] = {7, 8, 9};
    printf("%ld\n", jrand48(state));
    return 0;
}
```

lcong48 — 设置 48 位运算的随机数种子

头文件 `#include <stdlib.h>`

原型 `void lcong48(unsigned short param[7]);`

说明 设置 48 位随机数系列函数所使用的种子与参数。`param` 前三个元素为种子，后三个为乘数，最后一个为加数；通常使用默认值即可，不必调用此函数。

返回值 无返回值。

相关函数 `srand48`、`seed48`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    unsigned short param[7] = {1, 2, 3, 0xe66d, 0xdeec, 0x5, 0xb};
    lcong48(param);
    printf("%f\n", drand48());
    return 0;
}
```

lrand48 — 产生一个正的长整型随机数

头文件 `#include <stdlib.h>`

原型 `long lrand48(void);`

说明 以 48 位线性同余算法产生一个非负长整型随机数，并更新内部状态。

返回值 返回 0 至 2^31-1 之间的长整型随机数。

相关函数 `drand48`、`mrand48`、`srand48`

```
#include <stdlib.h>
#include <time.h>
#include <stdio.h>

int main(void)
{
    srand48((long)time(NULL));
    printf("%ld\n", lrand48());
    return 0;
}
```

mrand48 — 产生一个长整型随机数

头文件 `#include <stdlib.h>`

原型 `long mrand48(void);`

说明 与 `lrand48` 相同，但返回的有符号长整型数可为正或负。

返回值 返回 -2^31 至 2^31-1 之间的长整型随机数。

相关函数 `lrand48`、`jrand48`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    printf("%ld\n", mrand48());
    return 0;
}
```

nrand48 — 产生一个正的长整型随机数

头文件 `#include <stdlib.h>`

原型 `long nrand48(unsigned short xsubi[3]);`

说明 使用调用者提供的状态数组产生 0 至 2^31-1 之间的非负长整型随机数，并更新该数组。

返回值 返回非负长整型随机数。

相关函数 `lrand48`、`jrand48`、`erand48`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    unsigned short state[3] = {11, 22, 33};
    printf("%ld\n", nrand48(state));
    return 0;
}
```

rand — 产生随机数

头文件 `#include <stdlib.h>`

原型 `int rand(void);`

说明 产生一个 0 至 `RAND_MAX`（至少为 32767）之间的伪随机整数。若不设置种子，第一次调用前相当于以 `srand(1)` 初始化，因此每次运行会得到相同序列。

返回值 返回 0 至 `RAND_MAX` 之间的随机整数。

附加说明 需要可重复的结果时用固定种子；需要不同序列时可用 `srand(time(NULL))`。产生某区间内的随机数可用取模运算。

相关函数 `srand`、`random`

```
#include <stdlib.h>
#include <time.h>
#include <stdio.h>

int main(void)
{
    srand(time(NULL));
    for (int i = 0; i < 5; i++)
        printf("%d ", rand() % 100);   /* 0~99 */
    printf("\n");
    return 0;
}
```

random — 产生随机数

头文件 `#include <stdlib.h>`

原型 `long random(void);`

说明 产生 0 至 `RAND_MAX`（通常为 2^31-1）之间的随机数，使用可配置的状态数组，随机性比 `rand` 更好。

返回值 返回长整型随机数。

附加说明 使用前可用 `srandom` 设置种子，或用 `initstate` 指定状态数组大小。

相关函数 `srandom`、`initstate`、`rand`

```
#include <stdlib.h>
#include <time.h>
#include <stdio.h>

int main(void)
{
    srandom(time(NULL));
    printf("%ld\n", random() % 1000);
    return 0;
}
```

seed48 — 设置 48 位运算的随机数种子

头文件 `#include <stdlib.h>`

原型 `unsigned short *seed48(unsigned short seed16v[3]);`

说明 设置 48 位随机数系列函数的种子为 `seed16v` 中的 48 位值，并把先前的种子存入内部数组返回。

返回值 返回指向保存旧种子数组的指针，内容为静态内存，下次调用会被覆盖。

相关函数 `srand48`、`lcong48`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    unsigned short seed[3] = {1, 2, 3};
    seed48(seed);
    printf("%f\n", drand48());
    return 0;
}
```

setstate — 建立随机数状态数组

头文件 `#include <stdlib.h>`

原型 `char *setstate(char *state);`

说明 切换 `random` 使用的状态数组为 `state`（必须是先前由 `initstate` 设置过的数组），实现多组独立随机序列。

返回值 成功返回前一个状态数组的指针，失败返回 NULL。

附加说明 与 `initstate` 不同，本函数不会重置种子。

相关函数 `initstate`、`random`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    char s1[256], s2[256];
    initstate(1, s1, sizeof(s1));
    printf("%ld\n", random());
    setstate(s2);            /* 切换到另一组状态 */
    return 0;
}
```

srand — 设置随机数种子

头文件 `#include <stdlib.h>`

原型 `void srand(unsigned int seed);`

说明 设置 `rand` 使用的随机数种子。相同种子会产生相同的随机数序列，因此调试时可使用固定种子。

返回值 无返回值。

相关函数 `rand`、`random`

```
#include <stdlib.h>
#include <time.h>
#include <stdio.h>

int main(void)
{
    srand((unsigned int)time(NULL));
    printf("%d\n", rand());
    return 0;
}
```

srand48 — 设置 48 位运算的随机数种子

头文件 `#include <stdlib.h>`

原型 `void srand48(long seedval);`

说明 设置 48 位随机数系列函数（`drand48`、`lrand48` 等）的种子，仅使用 `seedval` 的低 32 位，高 16 位固定为 0x330e。

返回值 无返回值。

相关函数 `drand48`、`seed48`、`lcong48`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    srand48(2002);
    printf("%f\n", drand48());
    return 0;
}
```

srandom — 设置随机数种子

头文件 `#include <stdlib.h>`

原型 `void srandom(unsigned int seed);`

说明 设置 `random` 使用的随机数种子。

返回值 无返回值。

相关函数 `random`、`initstate`、`srand`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    srandom(42);
    printf("%ld\n", random());
    return 0;
}
```

第11章 初级 I/O 函数

本章为基于文件描述词（file descriptor）的低级输入输出函数，属系统调用，不带缓冲。

`close` — 关闭文件

头文件 `#include <unistd.h>`

原型 `int close(int fd);`

说明 关闭参数 `fd` 所指的文件描述词，释放相关资源，之后该描述词可被重新分配使用。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（通常为 `EBADF`）。

附加说明 进程结束时所有文件会自动关闭，但长期运行的程序应主动关闭，以免耗尽描述词。

相关函数 `open`、`creat`、`read`

```
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>

int main(void)
{
    int fd = open("/etc/hostname", O_RDONLY);
    if (fd < 0) {
        perror("open");
        return 1;
    }
    close(fd);
    return 0;
}
```

creat — 建立文件

头文件

原型 `int creat(const char *pathname, mode_t mode);`

说明 以只写方式建立（或截断已存在）的文件，等价于 `open(pathname, O_WRONLY | O_CREAT | O_TRUNC, mode)`。

返回值 成功返回文件描述词，失败返回 -1 并设置 `errno`。

附加说明 无法打开已存在的文件进行读写时需改用 `open`； `mode` 会受 `umask` 影响。

相关函数 `open`、`close`、`umask`

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
```

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    int fd = creat("/tmp/test.txt", 0644);
    if (fd < 0) {
        perror("creat");
        return 1;
    }
    close(fd);
    return 0;
}
```

dup — 复制文件描述词

头文件 `#include <unistd.h>`

原型 `int dup(int oldfd);`

说明 复制参数 `oldfd` 所指的文件描述词，返回当前可用的最小编号描述词，两者共享同一文件偏移与状态。

返回值 成功返回新的文件描述词，失败返回 -1 并设置 `errno` 为 `EBADF` 或 `EMFILE`。

相关函数 `dup2`、`fcntl`

```
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>

int main(void)
{
    int fd = open("/tmp/test.txt", O_WRONLY | O_CREAT, 0644);
    int fd2 = dup(fd);
    write(fd2, "hello", 5);   /* 与 fd 共享偏移 */
    close(fd);
    close(fd2);
    return 0;
}
```

dup2 — 复制文件描述词

头文件	`#include <unistd.h>`
原型	`int dup2(int oldfd, int newfd);`
说明	将 `oldfd` 复制为指定的 `newfd`；若 `newfd` 已打开则先关闭它。常用于把标准输入、输出重定向到文件（如 shell 的重定向与管道）。
返回值	成功返回 `newfd`，失败返回 -1 并设置 `errno`。
附加说明	若 `oldfd` 与 `newfd` 相同，则直接返回 `newfd` 而不做处理。
相关函数	`dup`、`fcntl`、`pipe`

```
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>

int main(void)
{
    int fd = open("/tmp/out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    dup2(fd, STDOUT_FILENO);   /* 标准输出重定向到文件 */
    printf("this goes to the file\n");
    close(fd);
    return 0;
}
```

`fcntl` — 文件描述词操作

头文件

原型 `int fcntl(int fd, int cmd);`

说明 依 `cmd` 对文件描述词 `fd` 进行各种操作，常用的命令有：`F_DUPFD`（复制描述词）、`F_GETFD` / `F_SETFD`（取得/设置 close-on-exec 标志）、`F_GETFL` / `F_SETFL`（取得/设置文件状态标志，如 `O_APPEND`、`O_NONBLOCK`）、`F_GETLK` / `F_SETLK` / `F_SETLKW`（取得/设置/阻塞式设置文件锁）。

返回值 依 `cmd` 而不同，多数成功返回 0 或新描述词，失败返回 -1 并设置 `errno`。

相关函数 `open`、`dup2`、`flock`

```
#include <unistd.h>
#include <fcntl.h>
```

```
/* 设置非阻塞 I/O */
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>

int main(void)
{
    int flags = fcntl(STDIN_FILENO, F_GETFL, 0);
    fcntl(STDIN_FILENO, F_SETFL, flags | O_NONBLOCK);
    printf("flags = 0x%x\n", flags);
    return 0;
}
```

`int fcntl(int fd, int cmd, long arg);`

`flock` — 锁定文件或解除锁定

头文件 `#include <sys/file.h>`

原型 `int flock(int fd, int operation);`

说明 对整个文件加上建议性锁（advisory lock）。`operation` 可为 `LOCK_SH`（共享锁）、`LOCK_EX`（互斥锁）、`LOCK_UN`（解除锁定），并可与 `LOCK_NB`（不阻塞，无法加锁时立即返回）组合。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`LOCK_NB` 且无法加锁时为 `EWOULDBLOCK`）。

附加说明 锁只作用于文件本身，进程关闭描述词或结束时自动释放；与 `fcntl` 的锁互不影响。

相关函数 `fcntl`、`open`

```
#include <sys/file.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    int fd = open("/tmp/lock.txt", O_WRONLY | O_CREAT, 0644);
    if (flock(fd, LOCK_EX | LOCK_NB) < 0) {
        perror("flock");
        return 1;
    }
    write(fd, "locked\n", 7);
    flock(fd, LOCK_UN);
    close(fd);
    return 0;
}
```

fsync — 将缓冲区数据写回磁盘

头文件 `#include <unistd.h>`

原型 `int fsync(int fd);`

说明 将 `fd` 所指文件在缓冲区中尚未写入磁盘的数据强制写回，确保数据真正保存到磁盘。

返回值 成功返回 0，失败返回 -1 并设置 `errno`。

附加说明 在数据库、日志等对可靠性要求高的场合必须使用；注意它不保证目录项同步（可用 `fsync` 打开目录后同步）。

相关函数 `sync`、`fdatasync`

```
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>

int main(void)
{
    int fd = open("/tmp/log.txt", O_WRONLY | O_CREAT | O_APPEND, 0644);
    write(fd, "important\n", 10);
    fsync(fd);          /* 立即落盘 */
    close(fd);
    return 0;
}
```

lseek — 移动文件的读写位置

头文件

原型 `off_t lseek(int fd, off_t offset, int whence);`

说明 移动 `fd` 的读写位置。`offset` 为位移量，`whence` 可取 `SEEK_SET`（从文件开头）、`SEEK_CUR`（从当前位置）、`SEEK_END`（从文件尾端）。偏移可以超出文件尾端，形成文件空洞。

返回值 成功返回新的读写位置（距文件开头的字节数），失败返回 `(off_t)-1` 并设置 `errno`。

附加说明 取得文件大小可用 `lseek(fd, 0, SEEK_END)`。

相关函数 `read`、`write`、`fseek`

```
#include <sys/types.h>
#include <unistd.h>
```

```
#include <sys/types.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>

int main(void)
{
    int fd = open("/etc/hostname", O_RDONLY);
    off_t size = lseek(fd, 0, SEEK_END);
    printf("size = %ld\n", (long)size);
    lseek(fd, 0, SEEK_SET);
    close(fd);
    return 0;
}
```

`mkstemp` — 建立唯一的临时文件

头文件 `#include <stdlib.h>`

原型 `int mkstemp(char *template);`

说明 依参数 `template`（必须以 6 个 `X` 结尾，如 `/tmp/nameXXXXXX`）建立唯一且已打开的临时文件，避免竞态条件。

返回值 成功返回已打开的临时文件描述词（权限 0600），失败返回 -1。

附加说明 `template` 会被改写为实际文件名；建议使用本函数取代不安全的 `mktemp`；用完后应 `unlink` 删除。

相关函数 `mktemp`、`tmpfile`、`open`

```
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    char tpl[] = "/tmp/myprogXXXXXX";
    int fd = mkstemp(tpl);
    printf("temp file: %s\n", tpl);
    write(fd, "data", 4);
    close(fd);
    unlink(tpl);
    return 0;
}
```

open — 打开文件

头文件

原型 `int open(const char *pathname, int flags);`

说明 打开 `pathname` 指定的文件。`flags` 必须包含 `O_RDONLY`、`O_WRONLY` 或 `O_RDWR` 之一，并可与 `O_CREAT`（不存在则建立，此时需提供 `mode`）、`O_EXCL`（与 `O_CREAT` 合用，文件已存在则失败）、`O_TRUNC`（长度截为 0）、`O_APPEND`（写入时移到文件尾）、`O_NONBLOCK`、`O_SYNC` 等组合。

返回值 成功返回文件描述词，失败返回 -1 并设置 `errno`（常见 `ENOENT`、`EACCES`）。

附加说明 `mode` 的实际权限还会受到 `umask` 影响，例如 `0666 & ~umask`。

相关函数 `creat`、`close`、`read`、`write`

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
```

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    int fd = open("/tmp/demo.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("open");
        return 1;
    }
    write(fd, "hello open\n", 11);
    close(fd);
    return 0;
}
```

`int open(const char *pathname, int flags, mode_t mode);`

read — 由已打开的文件读取数据

头文件 `#include <unistd.h>`

原型 `ssize_t read(int fd, void *buf, size_t count);`

说明 从 `fd` 所指的文件读取最多 `count` 个字节到 `buf` 中，并把读写位置向后推进。

返回值 成功返回实际读取的字节数；读到文件尾返回 0，失败返回 -1 并设置 `errno`。

附加说明 实际读取的字节数可能少于 `count`（如管道、网络、终端），因此循环中必须判断返回值。

相关函数 `write`、`open`、`fread`

```
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>

int main(void)
{
    char buf[128];
    int fd = open("/etc/hostname", O_RDONLY);
    ssize_t n = read(fd, buf, sizeof(buf) - 1);
    buf[n > 0 ? n : 0] = '\0';
    printf("%s", buf);
    close(fd);
    return 0;
}
```

sync — 将缓冲区数据写回磁盘

头文件 `#include <unistd.h>`

原型 `void sync(void);`

说明 将系统缓冲区中所有尚未写入磁盘的数据写回磁盘，通常由系统守护进程定期调用（如 `update`、`bdflush`）。

返回值 无返回值。

附加说明 与 `fsync` 不同，本函数针对整个系统且不保证完成时数据已落盘。

相关函数 `fsync`

```
#include <unistd.h>

int main(void)
{
    sync();   /* 将所有缓冲数据写回磁盘 */
    return 0;
}
```

`write` — 将数据写入已打开的文件内

头文件	`#include <unistd.h>`
原型	`ssize_t write(int fd, const void *buf, size_t count);`
说明	将 `buf` 中的 `count` 个字节写入 `fd` 所指的文件，并把读写位置向后推进。
返回值	成功返回实际写入的字节数，失败返回 -1 并设置 `errno`（如 `EBADF`、`ENOSPC`）。
附加说明	写入字节数可能少于 `count`（如磁盘已满、信号中断、非阻塞管道），写入循环中应累计判断。
相关函数	`read`、`open`、`fwrite`

```
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <stdio.h>

int main(void)
{
    const char *msg = "hello write\n";
    int fd = open("/tmp/demo.txt", O_WRONLY | O_CREAT | O_APPEND, 0644);
    write(fd, msg, strlen(msg));
    close(fd);
    return 0;
}
```

第12章 标准 I/O 函数

本章为基于文件指针（`FILE *`）的带缓冲输入输出函数，声明于 `<stdio.h>`。

clearerr — 清除文件流的错误旗标

头文件 `#include <stdio.h>`

原型 `void clearerr(FILE *stream);`

说明 清除 `stream` 所指文件流的错误旗标与文件结束旗标，使后续 I/O 可继续进行。

返回值 无返回值。

相关函数 `feof`、`ferror`

```
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("/tmp/a.txt", "r");
    /* ... 读写 ... */
    clearerr(fp);        /* 清除 EOF / 错误旗标 */
    fclose(fp);
    return 0;
}
```

fclose — 关闭文件

头文件 `#include <stdio.h>`

原型 `int fclose(FILE *stream);`

说明 关闭 `stream` 所指的文件，并在关闭前把缓冲区中的数据写回文件。

返回值 成功返回 0，失败返回 `EOF` 并设置 `errno`。

附加说明 文件用完务必关闭，否则可能造成数据未写入或描述词耗尽。

相关函数 `fopen`、`fflush`

```
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("/tmp/a.txt", "w");
    fputs("hello", fp);
    fclose(fp);
    return 0;
}
```

fdopen — 将文件描述词转为文件指针

头文件 `#include <stdio.h>`

原型 `FILE *fdopen(int fd, const char *mode);`

说明 将已打开的文件描述词 `fd` 转换为对应的文件指针，之后可用标准 I/O 函数操作。`mode` 必须与 `fd` 原有的读写方式一致。

返回值 成功返回文件指针，失败返回 NULL 并设置 `errno`。

相关函数 `fopen`、`fileno`、`open`

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main(void)
{
    int fd = open("/tmp/a.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    FILE *fp = fdopen(fd, "w");
    fprintf(fp, "via fdopen\n");
    fclose(fp);          /* 同时关闭 fd */
    return 0;
}
```

feof — 检查文件流是否读到了文件尾

头文件 `#include <stdio.h>`

原型 `int feof(FILE *stream);`

说明 检查 `stream` 所指的文件流是否已读到文件尾（end-of-file 旗标是否被设置）。

返回值 已到文件尾返回非 0，否则返回 0。

附加说明 旗标通常在执行读取操作失败后才被设置，因此不适合作为循环的唯一判断条件。

相关函数 `clearerr`、`ferror`、`fgetc`

```
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("/etc/hostname", "r");
    int c;
    while ((c = fgetc(fp)) != EOF)
        putchar(c);
    if (feof(fp))
        printf("reached end of file\n");
    fclose(fp);
    return 0;
}
```

fflush — 更新缓冲区

头文件 `#include <stdio.h>`

原型 `int fflush(FILE *stream);`

说明 强制将 `stream` 缓冲区中的数据写入文件；`stream` 为 NULL 时刷新所有打开的输出流。

返回值 成功返回 0，失败返回 `EOF` 并设置 `errno`。

附加说明 用于确保数据及时写入（如日志、交互式提示）；注意 `fflush` 对输入流的行为依实现而定。

相关函数 `fclose`、`setvbuf`

```
#include <stdio.h>

int main(void)
{
    printf("processing...");
    fflush(stdout);      /* 立即显示到屏幕 */
    /* ... 耗时操作 ... */
    printf("done\n");
    return 0;
}
```

fgetc — 由文件中读取一个字符

头文件 `#include <stdio.h>`

原型 `int fgetc(FILE *stream);`

说明 从 `stream` 所指的文件读取一个字符，并把读写位置向后推进一个字节。

返回值 成功返回读取到的字符（转换为 `unsigned char` 后的 `int`），读到文件尾或出错返回 `EOF`。

相关函数 `getc`、`getchar`、`fputc`

```
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("/etc/hostname", "r");
    int c;
    while ((c = fgetc(fp)) != EOF)
        putchar(c);
    fclose(fp);
    return 0;
}
```

fgetpos — 取得文件流的读取位置

头文件 `#include <stdio.h>`

原型 `int fgetpos(FILE *stream, fpos_t *pos);`

说明 取得 `stream` 当前的读写位置，存入 `pos` 所指的位置，供 `fsetpos` 恢复使用。

返回值 成功返回 0，失败返回非 0 并设置 `errno`。

附加说明 `fpos_t` 为不透明类型，适合处理超大文件；一般用途可用 `ftell` / `fseek`。

相关函数 `fsetpos`、`ftell`、`fseek`

```
#include <stdio.h>

int main(void)
{
    fpos_t pos;
    FILE *fp = fopen("/etc/hostname", "r");
    fgetc(fp);
    fgetpos(fp, &pos);        /* 记住位置 */
    fclose(fp);
    fp = fopen("/etc/hostname", "r");
    fsetpos(fp, &pos);        /* 回到该位置 */
    putchar(fgetc(fp));
    fclose(fp);
    return 0;
}
```

fgets — 由文件中读取一字符串

头文件 `#include <stdio.h>`

原型 `char *fgets(char *s, int size, FILE *stream);`

说明 从 `stream` 读取字符存入 `s`，直到遇到换行符、读到 `size - 1` 个字符或到达文件尾为止，并自动在末尾补上 `'\0'`。换行符也会被读入并保留。

返回值 成功返回 `s`，失败或读到文件尾返回 NULL。

附加说明 比 `gets` 安全，可避免缓冲区溢出，建议取代 `gets`。

相关函数 `fputs`、`gets`、`fgetc`

```
#include <stdio.h>

int main(void)
{
    char buf[128];
    while (fgets(buf, sizeof(buf), stdin))
        fputs(buf, stdout);
    return 0;
}
```

fileno — 返回文件流所使用的文件描述词

头文件 `#include <stdio.h>`

原型 `int fileno(FILE *stream);`

说明 取得 `stream` 所使用的文件描述词，便于调用 `read`、`fcntl` 等底层函数。

返回值 返回文件描述词。

相关函数 `fdopen`、`open`

```
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    printf("stdin fd = %d\n", fileno(stdin));   /* 0 */
    return 0;
}
```

fopen — 打开文件

头文件 `#include <stdio.h>`

原型 `FILE *fopen(const char *pathname, const char *mode);`

说明 以 `mode` 指定的方式打开文件。常用模式：`"r"`（只读，文件须存在）、`"w"`（只写，不存在则建立、存在则截断）、`"a"`（追加写入）、`"r+"`（读写）、`"w+"`（读写并截断）、`"a+"`（读与追加）；可加 `"b"` 表示二进制模式（Linux 下无差别）。

返回值 成功返回文件指针，失败返回 NULL 并设置 `errno`。

附加说明 务必检查返回值；用完须 `fclose` 关闭。

相关函数 `fclose`、`freopen`、`open`

```
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("/tmp/a.txt", "w");
    if (fp == NULL) {
        perror("fopen");
        return 1;
    }
    fputs("hello fopen\n", fp);
    fclose(fp);
    return 0;
}
```

fputc — 将一指定字符写入文件流中

头文件 `#include <stdio.h>`

原型 `int fputc(int c, FILE *stream);`

说明 将字符 `c` 写入 `stream` 所指的文件流，并推进读写位置。

返回值 成功返回写入的字符，失败返回 `EOF`。

相关函数 `putc`、`putchar`、`fgetc`

```
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("/tmp/a.txt", "w");
    fputc('A', fp);
    fputc('\n', fp);
    fclose(fp);
    return 0;
}
```

fputs — 将一指定的字符串写入文件内

头文件 `#include <stdio.h>`

原型 `int fputs(const char *s, FILE *stream);`

说明 将字符串 `s` 写入 `stream`，直到遇到结尾 `'\0'`，但不会写入该 `'\0'`。

返回值 成功返回写入的字符数（非负），失败返回 `EOF`。

附加说明 不会自动添加换行符，需要时自行加上 `'\n'`。

相关函数 `fgets`、`puts`、`fprintf`

```
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("/tmp/a.txt", "w");
    fputs("first line\n", fp);
    fputs("second line\n", fp);
    fclose(fp);
    return 0;
}
```

fread — 从文件流读取数据

头文件 `#include <stdio.h>`

原型 `size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);`

说明 从 `stream` 读取 `nmemb` 项数据，每项 `size` 字节，存入 `ptr` 所指的内存。

返回值 返回实际读取的完整项数；若小于 `nmemb`，则可能已到文件尾或发生错误（用 `feof`、`ferror` 区分）。

相关函数 `fwrite`、`read`、`fgets`

```
#include <stdio.h>

int main(void)
{
    char buf[64];
    FILE *fp = fopen("/etc/hostname", "r");
    size_t n = fread(buf, 1, sizeof(buf), fp);
    printf("read %zu bytes: %.*s\n", n, (int)n, buf);
    fclose(fp);
    return 0;
}
```

freopen — 打开文件

头文件 `#include <stdio.h>`

原型 `FILE *freopen(const char *pathname, const char *mode, FILE *stream);`

说明 关闭原 `stream` 并以 `pathname`、`mode` 重新打开，文件指针仍然指向同一个 `FILE` 对象。常用于重定向标准输入、输出、错误。

返回值 成功返回该 `stream` 指针，失败返回 NULL。

相关函数 `fopen`、`fclose`、`dup2`

```
#include <stdio.h>

int main(void)
{
    freopen("/tmp/out.txt", "w", stdout);
    printf("redirected to file\n");
    fclose(stdout);
    return 0;
}
```

fseek — 移动文件流的读写位置

头文件 `#include <stdio.h>`

原型 `int fseek(FILE *stream, long offset, int whence);`

说明 移动 `stream` 的读写位置，`whence` 可为 `SEEK_SET`、`SEEK_CUR`、`SEEK_END`。

返回值 成功返回 0，失败返回非 0 并设置 `errno`。

附加说明 文本模式下 `offset` 必须为 0 或 `ftell` 的返回值；二进制模式下可任意定位。

相关函数 `ftell`、`rewind`、`fgetpos`

```
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("/etc/hostname", "r");
    fseek(fp, 0, SEEK_END);
    printf("size = %ld\n", ftell(fp));
    rewind(fp);
    fclose(fp);
    return 0;
}
```

fsetpos — 移动文件流的读写位置

头文件 `#include <stdio.h>`

原型 `int fsetpos(FILE *stream, const fpos_t *pos);`

说明 将 `stream` 的读写位置设置为 `pos`（由 `fgetpos` 取得）所记录的位置。

返回值 成功返回 0，失败返回非 0 并设置 `errno`。

相关函数 `fgetpos`、`fseek`

```
#include <stdio.h>

int main(void)
{
    fpos_t pos;
    FILE *fp = fopen("/etc/hostname", "r");
    fgets(NULL, 0, fp);          /* 读取一行 */
    fgetpos(fp, &pos);
    fsetpos(fp, &pos);           /* 回到刚才的位置 */
    fclose(fp);
    return 0;
}
```

ftell — 取得文件流的读取位置

头文件 `#include <stdio.h>`

原型 `long ftell(FILE *stream);`

说明 取得 `stream` 当前的读写位置，即距文件开头的字节数。

返回值 成功返回当前位置，失败返回 -1L 并设置 `errno`。

相关函数 `fseek`、`rewind`、`fgetpos`

```
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("/etc/hostname", "r");
    fseek(fp, 0, SEEK_END);
    printf("size = %ld\n", ftell(fp));
    fclose(fp);
    return 0;
}
```

fwrite — 将数据写至文件流

头文件 `#include <stdio.h>`

原型 `size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);`

说明 将 `ptr` 所指内存中的 `nmemb` 项数据（每项 `size` 字节）写入 `stream`。

返回值 返回实际写入的完整项数；小于 `nmemb` 表示发生错误（如磁盘已满）。

相关函数 `fread`、`write`、`fputs`

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    const char *s = "hello fwrite\n";
    FILE *fp = fopen("/tmp/a.txt", "w");
    fwrite(s, 1, strlen(s), fp);
    fclose(fp);
    return 0;
}
```

getc — 由文件中读取一个字符

头文件 `#include <stdio.h>`

原型 `int getc(FILE *stream);`

说明 与 `fgetc` 相同，从 `stream` 读取一个字符。通常以宏实现，参数不应带有副作用（如 `getc(fp++)`）。

返回值 成功返回读取到的字符，读到文件尾或出错返回 `EOF`。

相关函数 `fgetc`、`getchar`、`putc`

```
#include <stdio.h>

int main(void)
{
    int c;
    while ((c = getc(stdin)) != EOF)
        putc(c, stdout);
    return 0;
}
```

getchar — 由标准输入设备内读进一字符

头文件 `#include <stdio.h>`

原型 `int getchar(void);`

说明 从标准输入读取一个字符，等价于 `getc(stdin)`。

返回值 成功返回读取到的字符，读到文件尾或出错返回 `EOF`。

相关函数 `putchar`、`getc`、`fgetc`

```
#include <stdio.h>

int main(void)
{
    int c;
    while ((c = getchar()) != EOF)
        putchar(c);
    return 0;
}
```

gets — 由标准输入设备内读进一字符串

头文件 `#include <stdio.h>`

原型 `char *gets(char *s);`

说明 从标准输入读取一行字符到 `s`，读到换行符或 `EOF` 为止，换行符会被替换为 `'\0'`。

返回值 成功返回 `s`，失败或读到文件尾返回 NULL。

附加说明 因无法限制输入长度会造成缓冲区溢出，已从 C11 标准中移除，务必改用 `fgets`。

相关函数 `fgets`、`puts`

```
/* 仅为说明用法，实际编程请使用 fgets(buf, size, stdin) */
#include <stdio.h>

int main(void)
{
    char buf[128];
    if (fgets(buf, sizeof(buf), stdin)) {   /* 安全替代方案 */
        printf("you typed: %s", buf);
    }
    return 0;
}
```

mktemp — 产生唯一的临时文件文件名

头文件 `#include <stdlib.h>`

原型 `char *mktemp(char *template);`

说明 将 `template` 结尾的 `XXXXXX` 替换为随机字符，产生唯一的文件名。

返回值 成功返回 `template` 指针，失败返回 NULL（此时 `template` 内容未定义）。

附加说明 只产生文件名并不建立文件，存在竞态条件与安全风险，应使用 `mkstemp` 或 `tmpfile`。

相关函数 `mkstemp`、`tmpfile`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    char tpl[] = "/tmp/tmpXXXXXX";
    printf("name: %s\n", mktemp(tpl));   /* 建议改用 mkstemp */
    return 0;
}
```

putc — 将一指定字符写入文件中

头文件 `#include <stdio.h>`

原型 `int putc(int c, FILE *stream);`

说明 与 `fputc` 相同，将字符 `c` 写入 `stream`。通常以宏实现，参数不应带有副作用。

返回值 成功返回写入的字符，失败返回 `EOF`。

相关函数 `fputc`、`putchar`、`getc`

```
#include <stdio.h>

int main(void)
{
    for (int i = 0; i < 5; i++)
        putc('*', stdout);
    putc('\n', stdout);
    return 0;
}
```

putchar — 将指定的字符写到标准输出设备

头文件 `#include <stdio.h>`

原型 `int putchar(int c);`

说明 将字符 `c` 写到标准输出，等价于 `putc(c, stdout)`。

返回值 成功返回写入的字符，失败返回 `EOF`。

相关函数 `getchar`、`putc`、`puts`

```
#include <stdio.h>

int main(void)
{
    putchar('A');
    putchar('\n');
    return 0;
}
```

puts — 将指定的字符串写到标准输出设备

头文件 `#include <stdio.h>`

原型 `int puts(const char *s);`

说明 将字符串 `s` 写到标准输出，并自动在结尾添加一个换行符。

返回值 成功返回非负整数，失败返回 `EOF`。

相关函数 `fputs`、`putchar`、`printf`

```
#include <stdio.h>

int main(void)
{
    puts("hello puts");   /* 自动换行 */
    return 0;
}
```

rewind — 重设文件流的读写位置为文件开头

头文件 `#include <stdio.h>`

原型 `void rewind(FILE *stream);`

说明 将 `stream` 的读写位置重设到文件开头，并清除错误旗标，等价于 `fseek(stream, 0, SEEK_SET)` 加 `clearerr`。

返回值 无返回值。

相关函数 `fseek`、`ftell`、`fgetpos`

```
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("/etc/hostname", "r");
    char buf[64];
    fgets(buf, sizeof(buf), fp);
    rewind(fp);                         /* 回到开头 */
    fgets(buf, sizeof(buf), fp);
    printf("%s", buf);
    fclose(fp);
    return 0;
}
```

setbuf — 设置文件流的缓冲区

头文件 `#include <stdio.h>`

原型 `void setbuf(FILE *stream, char *buf);`

说明 设置 `stream` 的缓冲区为 `buf`（大小须为 `BUFSIZ`）；`buf` 为 NULL 时关闭缓冲。

返回值 无返回值。

附加说明 必须在打开文件后、进行任何 I/O 操作前调用；`buf` 的生命周期须持续到文件关闭。

相关函数 `setbuffer`、`setvbuf`、`setlinebuf`

```
#include <stdio.h>

int main(void)
{
    char buf[BUFSIZ];
    FILE *fp = fopen("/tmp/a.txt", "w");
    setbuf(fp, buf);        /* 使用自定缓冲区 */
    fputs("data", fp);
    fclose(fp);
    return 0;
}
```

setbuffer — 设置文件流的缓冲区

头文件 `#include <stdio.h>`

原型 `void setbuffer(FILE *stream, char *buf, size_t size);`

说明 与 `setbuf` 相同，但可自行指定缓冲区大小 `size`；`buf` 为 NULL 时关闭缓冲。

返回值 无返回值。

附加说明 为 BSD 兼容函数，建议改用标准的 `setvbuf`。

相关函数 `setbuf`、`setvbuf`

```
#include <stdio.h>

int main(void)
{
    char buf[4096];
    FILE *fp = fopen("/tmp/a.txt", "w");
    setbuffer(fp, buf, sizeof(buf));
    fputs("data", fp);
    fclose(fp);
    return 0;
}
```

setlinebuf — 设置文件流为线性缓冲区

头文件 `#include <stdio.h>`

原型 `void setlinebuf(FILE *stream);`

说明 将 `stream` 设置为行缓冲（line buffered）模式，遇到换行符即刷新缓冲区，常用于交互式输出。

返回值 无返回值。

附加说明 为 BSD 兼容函数，标准写法为 `setvbuf(stream, NULL, _IOLBF, 0)`。

相关函数 `setvbuf`、`setbuf`、`fflush`

```
#include <stdio.h>

int main(void)
{
    setlinebuf(stdout);    /* 立即显示每行输出 */
    printf("line buffered\n");
    return 0;
}
```

setvbuf — 设置文件流的缓冲区

头文件 `#include <stdio.h>`

原型 `int setvbuf(FILE *stream, char *buf, int mode, size_t size);`

说明 设置 `stream` 的缓冲方式：`mode` 可取 `_IOFBF`（全缓冲）、`_IOLBF`（行缓冲）、`_IONBF`（不缓冲）。`buf` 为 NULL 时由系统自动配置 `size` 字节的缓冲区。

返回值 成功返回 0，失败返回非 0。

附加说明 必须在打开文件后、进行任何 I/O 操作前调用；这是设置缓冲区的标准做法。

相关函数 `setbuf`、`setbuffer`、`fflush`

```
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("/tmp/a.txt", "w");
    setvbuf(fp, NULL, _IOLBF, 0);     /* 行缓冲 */
    fputs("hello\n", fp);
    fclose(fp);
    return 0;
}
```

`tmpfile` — 建立临时文件

头文件	`#include <stdio.h>`
原型	`FILE *tmpfile(void);`
说明	以读写方式（`w+b`）建立唯一的临时文件，该文件在关闭或程序结束时会自动删除。
返回值	成功返回文件指针，失败返回 NULL 并设置 `errno`。
附加说明	文件通常建立在 `/tmp` 下，无需自行处理文件名，比 `mktemp` 安全。
相关函数	`mkstemp`、`mktemp`、`fopen`

```
#include <stdio.h>

int main(void)
{
    FILE *fp = tmpfile();
    fputs("temporary data\n", fp);
    rewind(fp);
    char buf[64];
    fgets(buf, sizeof(buf), fp);
    printf("%s", buf);
    fclose(fp);          /* 文件自动删除 */
    return 0;
}
```

ungetc — 将一指定字符写回文件流中

头文件 `#include <stdio.h>`

原型 `int ungetc(int c, FILE *stream);`

说明 将字符 `c` 退回流 `stream` 的缓冲区，使其可被下一次读取再次取得，并把读写位置回退一个字节。

返回值 成功返回退回的字符 `c`，失败返回 `EOF`。

附加说明 通常只能退回一个字符；`EOF` 不能被退回；常用于词法分析时预读一个字符。

相关函数 `fgetc`、`getc`

```
#include <stdio.h>
#include <ctype.h>

int main(void)
{
    int c = getchar();
    if (isdigit(c)) {
        ungetc(c, stdin);        /* 退回以便重新读取 */
        printf("first char is a digit: %c\n", getchar());
    }
    return 0;
}
```

第13章 进程及流程控制

本章函数用于进程的建立、结束、替换、追踪、优先级控制与跳转。

abort — 以异常方式结束进程

头文件 `#include <stdlib.h>`

原型 `void abort(void);`

说明 以异常方式立即结束进程，会向进程发送 `SIGABRT` 信号；若信号处理函数未使进程结束，则产生 core 文件供调试。

返回值 无返回值（不会返回）。

附加说明 不会执行 `atexit` 注册的函数，也不刷新 stdio 缓冲区（依实现而定）。

相关函数 `exit`、`assert`、`raise`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    puts("before abort");
    abort();                /* 异常结束，可能产生 core */
    puts("never reached");
    return 0;
}
```

assert — 若测试的条件不成立则终止进程

头文件 `#include <assert.h>`

原型 `void assert(int expression);`

说明 测试 `expression` 是否成立；若不成立（为 0），则向 stderr 输出出错信息（含表达式、文件名与行号）并调用 `abort` 结束进程。

返回值 无返回值。

附加说明 定义宏 `NDEBUG` 后 `assert` 会被关闭，常用于发布版本；适合检查内部逻辑而非用户输入。

相关函数 `abort`、`exit`

```
#include <assert.h>
#include <stdio.h>

int main(void)
{
    int x = 5;
    assert(x > 0);       /* 成立，继续执行 */
    assert(x == 0);      /* 不成立，输出错误信息并终止 */
    return 0;
}
```

atexit — 设置程序正常结束前调用的函数

头文件 `#include <stdlib.h>`

原型 `int atexit(void (*function)(void));`

说明 注册 `function`，使其在程序正常结束（`exit` 或从 `main` 返回）前被调用。可注册多个函数，执行顺序与注册顺序相反。

返回值 成功返回 0，失败返回非 0。

附加说明 由 `abort` 或信号结束的程序不会执行这些函数。

相关函数 `exit`、`on_exit`

```
#include <stdlib.h>
#include <stdio.h>

void bye(void)
{
    printf("cleanup done\n");
}

int main(void)
{
    atexit(bye);
    puts("main ends");
    return 0;   /* 返回前自动调用 bye() */
}
```

execl — 执行文件

头文件 `#include <unistd.h>`

原型 `int execl(const char *path, const char *arg, ...);`

说明 以 `path` 指定的文件替换当前进程的映像，参数以可变参数形式给出，最后必须以 `(char *)NULL` 结尾。第一个参数惯例为程序名。

返回值 成功则不返回；失败返回 -1 并设置 `errno`（如 `ENOENT`、`EACCES`）。

附加说明 执行成功后原程序代码不再存在，因此常用 `fork` + `exec` 的组合。

相关函数 `execv`、`execlp`、`execve`、`fork`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    execl("/bin/ls", "ls", "-l", "/tmp", (char *)NULL);
    perror("execl");        /* 只有失败才会执行到这里 */
    return 1;
}
```

execle — 执行文件

头文件 `#include <unistd.h>`

原型 `int execle(const char *path, const char *arg, ..., char *const envp[]);`

说明 与 `execl` 相同，但可自行指定环境变量数组 `envp`，数组最后一项须为 NULL。

返回值 成功不返回，失败返回 -1 并设置 `errno`。

相关函数 `execl`、`execve`、`execvp`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    char *env[] = {"PATH=/bin:/usr/bin", "LANG=C", NULL};
    execle("/bin/ls", "ls", "-l", (char *)NULL, env);
    perror("execle");
    return 1;
}
```

execlp — 从 PATH 环境变量中查找文件并执行

头文件 `#include <unistd.h>`

原型 `int execlp(const char *file, const char *arg, ...);`

说明 与 `execl` 相同，但不必给出完整路径，会依 `PATH` 环境变量搜索可执行文件 `file`；若 `file` 含 `/` 则视为路径。

返回值 成功不返回，失败返回 -1 并设置 `errno`。

相关函数 `execl`、`execvp`、`system`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    execlp("ls", "ls", "-l", (char *)NULL);   /* 依 PATH 查找 ls */
    perror("execlp");
    return 1;
}
```

execv — 执行文件

头文件 `#include <unistd.h>`

原型 `int execv(const char *path, char *const argv[]);`

说明 与 `execl` 相同，但参数以字符串数组 `argv` 提供，数组最后一项须为 NULL。

返回值 成功不返回，失败返回 -1 并设置 `errno`。

相关函数 `execl`、`execvp`、`execve`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    char *argv[] = {"ls", "-l", "/tmp", NULL};
    execv("/bin/ls", argv);
    perror("execv");
    return 1;
}
```

execve — 执行文件

头文件 `#include <unistd.h>`

原型 `int execve(const char *filename, char *const argv[], char *const envp[]);`

说明 真正的系统调用：以 `filename` 指定的文件替换当前进程映像，参数与环境变量均以数组提供。其余 exec 函数最终都会调用它。

返回值 成功不返回，失败返回 -1 并设置 `errno`。

相关函数 `execv`、`execle`、`fork`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    char *argv[] = {"/bin/ls", "-l", NULL};
    char *envp[] = {"PATH=/bin", NULL};
    execve("/bin/ls", argv, envp);
    perror("execve");
    return 1;
}
```

execvp — 执行文件

头文件 `#include <unistd.h>`

原型 `int execvp(const char *file, char *const argv[]);`

说明 与 `execv` 相同，但会依 `PATH` 环境变量搜索可执行文件。

返回值 成功不返回，失败返回 -1 并设置 `errno`。

附加说明 常用于在子进程中执行用户命令。

相关函数 `execv`、`execlp`、`system`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    char *argv[] = {"ls", "-l", NULL};
    execvp("ls", argv);
    perror("execvp");
    return 1;
}
```

exit — 正常结束进程

头文件 `#include <stdlib.h>`

原型 `void exit(int status);`

说明 正常结束进程：先调用 `atexit` / `on_exit` 注册的函数，刷新并关闭所有 stdio 流，删除临时文件，最后把 `status & 0377` 传给父进程。通常用 `EXIT_SUCCESS`(0) 与 `EXIT_FAILURE`(1)。

返回值 无返回值（不会返回）。

相关函数 `_exit`、`atexit`、`wait`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("/tmp/a.txt", "w");
    fputs("bye", fp);
    exit(EXIT_SUCCESS);   /* 自动刷新并关闭 fp */
}
```

_exit — 结束进程执行

头文件 `#include <unistd.h>`

原型 `void _exit(int status);`

说明 立即结束进程，直接把 `status & 0377` 交给父进程，不刷新 stdio 缓冲区，也不执行 `atexit` 注册的函数。

返回值 无返回值（不会返回）。

附加说明 常用于 `fork` 后的子进程，避免重复刷新父进程的缓冲区。

相关函数 `exit`、`fork`、`wait`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    printf("no flush");   /* 不会被刷新 */
    _exit(0);
}
```

fork — 建立一个新的进程

头文件

原型 `pid_t fork(void);`

说明 复制当前进程建立子进程。子进程继承父进程的代码、数据与打开的文件描述词，但拥有独立的进程空间；两个进程都从 `fork` 返回处继续执行。

返回值 父进程返回子进程的 pid（大于 0），子进程返回 0，失败返回 -1 并设置 `errno`（如 `EAGAIN`）。

附加说明 子进程必须用 `wait` / `waitpid` 回收，否则会留下僵尸进程。

相关函数 `execve`、`wait`、`waitpid`、`_exit`

```
#include <sys/types.h>
#include <unistd.h>
```

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
    } else if (pid == 0) {
        printf("child: pid=%d ppid=%d\n", getpid(), getppid());
        _exit(0);
    } else {
        printf("parent: child=%d\n", pid);
        wait(NULL);
    }
    return 0;
}
```

getpgid — 取得进程组识别码

头文件 `#include <unistd.h>`

原型 `pid_t getpgid(pid_t pid);`

说明 取得 `pid` 指定进程的进程组识别码；`pid` 为 0 时表示调用进程本身。

返回值 成功返回进程组识别码，失败返回 -1 并设置 `errno`（`ESRCH`）。

相关函数 `setpgid`、`getpgrp`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    printf("pgid = %d\n", getpgid(0));
    return 0;
}
```

getpgrp — 取得进程组识别码

头文件 `#include <unistd.h>`

原型 `pid_t getpgrp(void);`

说明 取得调用进程的进程组识别码，等价于 `getpgid(0)`。

返回值 返回进程组识别码，不会失败。

相关函数 `getpgid`、`setpgrp`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    printf("pgrp = %d\n", getpgrp());
    return 0;
}
```

getpid — 取得进程识别码

头文件 `#include <unistd.h>`

原型 `pid_t getpid(void);`

说明 取得调用进程的进程识别码（pid）。

返回值 返回进程识别码，不会失败。

相关函数 `getppid`、`fork`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    printf("pid = %d\n", getpid());
    return 0;
}
```

getppid — 取得父进程的进程识别码

头文件 `#include <unistd.h>`

原型 `pid_t getppid(void);`

说明 取得调用进程的父进程识别码；父进程结束后会被 init（pid 1）收养。

返回值 返回父进程识别码，不会失败。

相关函数 `getpid`、`fork`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    printf("ppid = %d\n", getppid());
    return 0;
}
```

`getpriority` — 取得程序进程执行优先权

头文件

原型 `int getpriority(int which, int who);`

说明 取得进程、进程组或用户的执行优先权（nice 值）。`which` 可取 `PRIO_PROCESS`、`PRIO_PGRP`、`PRIO_USER`；`who` 为对应的 id，0 表示调用者自身。

返回值 返回 -20（最高优先）至 19（最低优先）之间的 nice 值；因 -1 也是合法值，判断错误需先清 `errno`。

相关函数 `setpriority`、`nice`

```
#include <sys/time.h>
#include <sys/resource.h>
```

```
#include <sys/time.h>
#include <sys/resource.h>
#include <errno.h>
#include <stdio.h>

int main(void)
{
    errno = 0;
    int prio = getpriority(PRIO_PROCESS, 0);
    printf("nice = %d\n", prio);
    return 0;
}
```

`longjmp` — 跳转到原先 setjmp 保存的堆栈环境

头文件 `#include <setjmp.h>`

原型 `void longjmp(jmp_buf env, int val);`

说明 跳回先前由 `setjmp` 保存的堆栈环境，使程序从该 `setjmp` 处继续执行，并将 `setjmp` 的返回值设为 `val`（`val` 为 0 时实际返回 1）。

返回值 无返回值（不会返回）。

附加说明 不可从已返回的函数中跳转；跳转后非 `volatile` 的局部变量值不确定。

相关函数 `setjmp`、`siglongjmp`

```
#include <setjmp.h>
#include <stdio.h>

static jmp_buf env;

void f(void)
{
    printf("in f(), jump back\n");
    longjmp(env, 2);
}

int main(void)
{
    int r = setjmp(env);
    if (r == 0) {
        printf("first time\n");
        f();
    } else {
        printf("back from longjmp, r=%d\n", r);
    }
    return 0;
}
```

nice — 改变进程优先顺序

头文件 `#include <unistd.h>`

原型 `int nice(int inc);`

说明 将调用进程的 nice 值增加 `inc`（正值降低优先级）。只有超级用户可使用负值来提高优先级。

返回值 成功返回新的 nice 值（注意可能为 -1，需配合 `errno` 判断），失败返回 -1。

相关函数 `getpriority`、`setpriority`

```
#include <unistd.h>
#include <errno.h>
#include <stdio.h>

int main(void)
{
    errno = 0;
    int n = nice(5);        /* 降低优先级 */
    printf("new nice = %d\n", n);
    return 0;
}
```

on_exit — 设置程序正常结束前调用的函数

头文件 `#include <stdlib.h>`

原型 `int on_exit(void (*function)(int, void *), void *arg);`

说明 注册程序正常结束前调用的函数，调用时会传入 `exit` 的状态值与注册时提供的 `arg` 指针。

返回值 成功返回 0，失败返回非 0。

附加说明 为 Linux/BSD 扩展函数，可移植性要求高时应使用 `atexit`。

相关函数 `atexit`、`exit`

```
#include <stdlib.h>
#include <stdio.h>

void bye(int status, void *arg)
{
    printf("status=%d arg=%s\n", status, (char *)arg);
}

int main(void)
{
    on_exit(bye, "done");
    return 0;
}
```

ptrace — 进程追踪

头文件 `#include <sys/ptrace.h>`

原型 `long ptrace(enum __ptrace_request request, pid_t pid, void *addr, void *data);`

说明 供父进程（或调试器）追踪与控制子进程的执行、读写其内存与寄存器。常用请求有 `PTRACE_TRACEME`（被追踪）、`PTRACE_ATTACH`、`PTRACE_PEEKDATA`、`PTRACE_POKEDATA`、`PTRACE_GETREGS`、`PTRACE_CONT`、`PTRACE_DETACH`、`PTRACE_SINGLESTEP`。

返回值 成功依请求返回不同值（`PTRACE_PEEK*` 返回读取的数据），失败返回 -1 并设置 `errno`（如 `ESRCH`、`EPERM`）。

附加说明 gdb、strace 等工具即基于此实现；受系统安全策略（如 `ptrace_scope`）限制。

相关函数 `waitpid`、`fork`

```
#include <sys/ptrace.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        ptrace(PTRACE_TRACEME, 0, NULL, NULL);
        execl("/bin/ls", "ls", NULL);
    } else {
        int status;
        waitpid(pid, &status, 0);
        printf("child stopped, status=%d\n", status);
        ptrace(PTRACE_CONT, pid, NULL, NULL);
        waitpid(pid, &status, 0);
    }
    return 0;
}
```

setjmp — 保存目前堆栈环境

头文件 `#include <setjmp.h>`

原型 `int setjmp(jmp_buf env);`

说明 保存当前的堆栈环境（程序计数器、栈指针、寄存器等）到 `env`，供 `longjmp` 跳回使用。

返回值 直接调用时返回 0；由 `longjmp` 跳回时返回 `longjmp` 给定的 `val`。

附加说明 通常用于深层嵌套调用中的错误处理（类似异常）。

相关函数 `longjmp`、`sigsetjmp`

```
#include <setjmp.h>
#include <stdio.h>

static jmp_buf env;

int main(void)
{
    if (setjmp(env) == 0) {
        printf("normal path\n");
        longjmp(env, 1);
    } else {
        printf("jumped back\n");
    }
    return 0;
}
```

setpgid — 设置进程组识别码

头文件 `#include <unistd.h>`

原型 `int setpgid(pid_t pid, pid_t pgid);`

说明 将 `pid` 指定进程的进程组识别码设置为 `pgid`；`pid` 为 0 表示调用进程，`pgid` 为 0 表示与 `pid` 相同（即成为进程组组长）。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（如 `EPERM`、`EACCES`）。

附加说明 常用于 shell 的作业控制；子进程在 `exec` 前调用较为安全。

相关函数 `getpgid`、`setpgrp`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    if (setpgid(0, 0) < 0)
        perror("setpgid");
    printf("pgid = %d\n", getpgid(0));
    return 0;
}
```

setpgrp — 设置进程组识别码

头文件 `#include <unistd.h>`

原型 `int setpgrp(void);`

说明 将调用进程的进程组识别码设置为其自身的 pid，等价于 `setpgid(0, 0)`。

返回值 成功返回 0，失败返回 -1 并设置 `errno`。

相关函数 `getpgrp`、`setpgid`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    setpgrp();
    printf("pgrp = %d pid = %d\n", getpgrp(), getpid());
    return 0;
}
```

`setpriority` — 设置程序进程执行优先权

头文件

原型 `int setpriority(int which, int who, int prio);`

说明 设置进程、进程组或用户的执行优先权（nice 值 `prio`，范围 -20 至 19）。降低优先级（正值）一般用户即可，提高优先级需 root 权限。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EPERM`、`ESRCH`）。

相关函数 `getpriority`、`nice`

```
#include <sys/time.h>
#include <sys/resource.h>
```

```
#include <sys/time.h>
#include <sys/resource.h>
#include <stdio.h>

int main(void)
{
    if (setpriority(PRIO_PROCESS, 0, 10) < 0)
        perror("setpriority");
    printf("nice = %d\n", getpriority(PRIO_PROCESS, 0));
    return 0;
}
```

`siglongjmp` — 跳转到原先 sigsetjmp 保存的堆栈环境

头文件 `#include <setjmp.h>`

原型 `void siglongjmp(sigjmp_buf env, int val);`

说明 与 `longjmp` 相同，但配合 `sigsetjmp` 使用，可一并恢复信号遮罩（signal mask），适用于信号处理函数中跳转。

返回值 无返回值（不会返回）。

相关函数 `sigsetjmp`、`longjmp`

```
#include <setjmp.h>
#include <signal.h>
#include <stdio.h>
#include <unistd.h>

static sigjmp_buf env;

void on_alarm(int sig)
{
    siglongjmp(env, 1);
}

int main(void)
{
    signal(SIGALRM, on_alarm);
    if (sigsetjmp(env, 1) == 0) {
        alarm(1);
        pause();
    }
    puts("back from signal handler");
    return 0;
}
```

sigsetjmp — 保存目前堆栈环境

头文件 `#include <setjmp.h>`

原型 `int sigsetjmp(sigjmp_buf env, int savesigs);`

说明 保存当前的堆栈环境；`savesigs` 非 0 时同时保存进程的信号遮罩，供 `siglongjmp` 恢复。

返回值 直接调用返回 0；由 `siglongjmp` 跳回时返回给定的 `val`。

相关函数 `siglongjmp`、`setjmp`

```
#include <setjmp.h>
#include <stdio.h>

static sigjmp_buf env;

int main(void)
{
    if (sigsetjmp(env, 1) == 0)
        siglongjmp(env, 3);
    else
        printf("returned via siglongjmp\n");
    return 0;
}
```

system — 执行 shell 命令

头文件 `#include <stdlib.h>`

原型 `int system(const char *command);`

说明 调用 `/bin/sh -c command` 执行指定的 shell 命令，执行期间会阻塞 `SIGINT`、`SIGQUIT` 并忽略 `SIGCHLD`。

返回值 `command` 为 NULL 时：shell 可用返回非 0，否则返回 0。执行成功返回命令的退出状态（需用 `WEXITSTATUS` 等宏解析）；无法启动 shell 返回 127；失败返回 -1。

附加说明 会创建子进程，效率低于 `fork` + `exec`；若 `command` 来自用户输入，存在命令注入风险。

相关函数 `execve`、`fork`、`popen`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    int ret = system("ls -l /tmp");
    printf("exit status = %d\n", WEXITSTATUS(ret));
    return 0;
}
```

`wait` — 等待子进程中断或结束

头文件

原型 `pid_t wait(int *status);`

说明 暂停调用进程，直到任一子进程结束或收到信号；`status` 不为 NULL 时存入子进程的结束状态（可用 `WIFEXITED`、`WEXITSTATUS`、`WIFSIGNALED`、`WTERMSIG` 等宏解析）。

返回值 成功返回结束子进程的 pid，失败返回 -1 并设置 `errno`（无子进程时为 `ECHILD`）。

附加说明 若不予回收，子进程会变成僵尸进程；若无子进程结束会一直阻塞。

相关函数 `waitpid`、`fork`、`exit`

```
#include <sys/types.h>
#include <sys/wait.h>
```

```
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        _exit(3);
    } else {
        int status;
        wait(&status);
        if (WIFEXITED(status))
            printf("child exited with %d\n", WEXITSTATUS(status));
    }
    return 0;
}
```

`waitpid` — 等待子进程中断或结束

头文件

原型 `pid_t waitpid(pid_t pid, int *status, int options);`

说明 等待指定的子进程。`pid` 大于 0 表示等待该 pid；等于 -1 表示任一子进程（同 `wait`）；等于 0 表示同进程组的子进程；小于 -1 表示指定进程组的子进程。`options` 可为 0 或 `WNOHANG`（无子进程结束则立即返回）、`WUNTRACED`（也返回已停止的子进程）、`WCONTINUED`。

返回值 成功返回状态改变的子进程 pid；使用 `WNOHANG` 且无子进程结束时返回 0；失败返回 -1。

相关函数 `wait`、`fork`

```
#include <sys/types.h>
#include <sys/wait.h>
```

```
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        sleep(2);
        _exit(0);
    }
    while (waitpid(pid, NULL, WNOHANG) == 0) {   /* 非阻塞轮询 */
        printf("child still running\n");
        sleep(1);
    }
    puts("child finished");
    return 0;
}
```

第14章 格式化输入输出函数

本章函数用于格式化输出与输入，声明于 `<stdio.h>`（`v` 系列需额外包含 `<stdarg.h>`）。

fprintf — 格式化输出数据至文件

头文件 `#include <stdio.h>`

原型 `int fprintf(FILE *stream, const char *format, ...);`

说明 按 `format` 格式化数据并输出到 `stream` 所指的文件流。

返回值 成功返回输出的字符数，失败返回负值。

相关函数 `printf`、`sprintf`、`vfprintf`

```
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("/tmp/a.txt", "w");
    fprintf(fp, "%s: %d\n", "count", 42);
    fclose(fp);
    return 0;
}
```

fscanf — 格式化字符串输入

头文件 `#include <stdio.h>`

原型 `int fscanf(FILE *stream, const char *format, ...);`

说明 从 `stream` 依 `format` 读取数据并存入后续指针参数所指的位置。

返回值 成功返回被赋值的参数个数，失败或读到文件尾返回 `EOF`。

附加说明 参数必须是指针；`%s` 存在溢出风险，应指定宽度如 `%31s`。

相关函数 `scanf`、`sscanf`、`vfscanf`

```
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("/tmp/a.txt", "r");
    char name[32];
    int n;
    if (fscanf(fp, "%31s %d", name, &n) == 2)
        printf("%s %d\n", name, n);
    fclose(fp);
    return 0;
}
```

printf — 格式化输出数据

头文件 `#include <stdio.h>`

原型 `int printf(const char *format, ...);`

说明 按 `format` 格式化数据并输出到标准输出。常用转换符：`%d`/`%i`（整数）、`%u`、`%x`、`%o`、`%ld`、`%f`、`%e`、`%g`、`%c`、`%s`、`%p`、`%%`；可用 `-`、`+`、`0`、宽度、`.精度`、`l`、`h` 等修饰。

返回值 成功返回输出的字符数，失败返回负值。

附加说明 格式字符串不可由用户输入控制（否则有格式化字符串漏洞）。

相关函数 `fprintf`、`sprintf`、`vprintf`

```
#include <stdio.h>

int main(void)
{
    printf("%-10s|%5.2f|%#x|\n", "pi", 3.14159, 255);
    return 0;
}
```

scanf — 格式化字符串输入

头文件 `#include <stdio.h>`

原型 `int scanf(const char *format, ...);`

说明 从标准输入依 `format` 读取数据并存入后续指针参数所指的位置。

返回值 成功返回成功赋值的数据个数，失败或遇到输入结束返回 `EOF`。

附加说明 输入格式不符时会把字符留在缓冲区，可能造成后续读取混乱；读取字符串务必指定宽度。

相关函数 `fscanf`、`sscanf`、`printf`

```
#include <stdio.h>

int main(void)
{
    int a, b;
    printf("input two integers: ");
    if (scanf("%d %d", &a, &b) == 2)
        printf("%d + %d = %d\n", a, b, a + b);
    return 0;
}
```

snprintf — 格式化字符串复制

头文件 `#include <stdio.h>`

原型 `int snprintf(char *str, size_t size, const char *format, ...);`

说明 按 `format` 格式化数据并写入 `str`，最多写入 `size` 字节（含结尾 `'\0'`），因此不会发生缓冲区溢出。

返回值 返回“若空间足够”本应写入的字符数（不含结尾 `'\0'`）；返回值 ≥ `size` 表示输出被截断。

附加说明 是 `sprintf` 的安全替代，建议优先使用。

相关函数 `sprintf`、`vsnprintf`

```
#include <stdio.h>

int main(void)
{
    char buf[16];
    int n = snprintf(buf, sizeof(buf), "%s-%d", "id", 12345);
    printf("buf=%s (need %d)\n", buf, n);
    return 0;
}
```

sprintf — 格式化字符串复制

头文件 `#include <stdio.h>`

原型 `int sprintf(char *str, const char *format, ...);`

说明 按 `format` 格式化数据并写入 `str`，结尾自动补上 `'\0'`。

返回值 成功返回写入的字符数（不含 `'\0'`），失败返回负值。

附加说明 不检查目标缓冲区大小，容易造成溢出，应使用 `snprintf`。

相关函数 `snprintf`、`vsprintf`

```
#include <stdio.h>

int main(void)
{
    char buf[64];
    sprintf(buf, "%d + %d = %d", 1, 2, 3);
    puts(buf);
    return 0;
}
```

sscanf — 格式化字符串输入

头文件 `#include <stdio.h>`

原型 `int sscanf(const char *str, const char *format, ...);`

说明 从字符串 `str` 依 `format` 读取数据并存入后续指针参数所指的位置，常用于解析已读入的文本。

返回值 成功返回被赋值的参数个数，失败返回 `EOF`。

相关函数 `scanf`、`fscanf`

```
#include <stdio.h>

int main(void)
{
    char *s = "2026-10-08";
    int y, m, d;
    if (sscanf(s, "%d-%d-%d", &y, &m, &d) == 3)
        printf("y=%d m=%d d=%d\n", y, m, d);
    return 0;
}
```

`vfprintf` — 格式化输出数据至文件

头文件

原型 `int vfprintf(FILE *stream, const char *format, va_list ap);`

说明 与 `fprintf` 相同，但参数以 `va_list` 提供，用于编写自定的可变参数输出函数。

返回值 成功返回输出的字符数，失败返回负值。

相关函数 `vprintf`、`vsnprintf`、`fprintf`

```
#include <stdio.h>
#include <stdarg.h>
```

```
#include <stdio.h>
#include <stdarg.h>

void log_out(FILE *fp, const char *fmt, ...)
{
    va_list ap;
    va_start(ap, fmt);
    vfprintf(fp, fmt, ap);
    va_end(ap);
}

int main(void)
{
    log_out(stderr, "error: %s (code %d)\n", "not found", 404);
    return 0;
}
```

vfscanf — 格式化字符串输入

头文件

原型 `int vfscanf(FILE *stream, const char *format, va_list ap);`

说明 与 `fscanf` 相同，但参数以 `va_list` 提供。

返回值 成功返回被赋值的参数个数，失败返回 `EOF`。

相关函数 `vscanf`、`vsscanf`、`fscanf`

```
#include <stdio.h>
#include <stdarg.h>
```

```
#include <stdio.h>
#include <stdarg.h>

int scan_file(FILE *fp, const char *fmt, ...)
{
    va_list ap;
    va_start(ap, fmt);
    int n = vfscanf(fp, fmt, ap);
    va_end(ap);
    return n;
}

int main(void)
{
    FILE *fp = fopen("/tmp/a.txt", "r");
    int v;
    scan_file(fp, "%d", &v);
    fclose(fp);
    return 0;
}
```

vprintf — 格式化输出数据

头文件

原型 `int vprintf(const char *format, va_list ap);`

说明 与 `printf` 相同，但参数以 `va_list` 提供。

返回值 成功返回输出的字符数，失败返回负值。

相关函数 `vfprintf`、`vsprintf`、`printf`

```
#include <stdio.h>
#include <stdarg.h>
```

```
#include <stdio.h>
#include <stdarg.h>

void my_printf(const char *fmt, ...)
{
    va_list ap;
    va_start(ap, fmt);
    vprintf(fmt, ap);
    va_end(ap);
}

int main(void)
{
    my_printf("%s %d\n", "answer", 42);
    return 0;
}
```

vscanf — 格式化字符串输入

头文件

原型 `int vscanf(const char *format, va_list ap);`

说明 与 `scanf` 相同，但参数以 `va_list` 提供。

返回值 成功返回被赋值的参数个数，失败返回 `EOF`。

相关函数 `vfscanf`、`vsscanf`、`scanf`

```
#include <stdio.h>
#include <stdarg.h>
```

```
#include <stdio.h>
#include <stdarg.h>

int my_scanf(const char *fmt, ...)
{
    va_list ap;
    va_start(ap, fmt);
    int n = vscanf(fmt, ap);
    va_end(ap);
    return n;
}

int main(void)
{
    int v;
    my_scanf("%d", &v);
    return 0;
}
```

`vsnprintf` — 格式化字符串复制

头文件

原型 `int vsnprintf(char *str, size_t size, const char *format, va_list ap);`

说明 与 `snprintf` 相同，但参数以 `va_list` 提供，且限制最多写入 `size` 字节。

返回值 返回若空间足够本应写入的字符数；≥ `size` 表示被截断。

附加说明 编写日志函数时最常用、最安全的组合。

相关函数 `snprintf`、`vsprintf`、`vfprintf`

```
#include <stdio.h>
#include <stdarg.h>
```

```
#include <stdio.h>
#include <stdarg.h>

void log_msg(const char *fmt, ...)
{
    char buf[256];
    va_list ap;
    va_start(ap, fmt);
    vsnprintf(buf, sizeof(buf), fmt, ap);
    va_end(ap);
    puts(buf);
}

int main(void)
{
    log_msg("user=%s id=%d", "root", 0);
    return 0;
}
```

`vsprintf` — 格式化字符串复制

头文件

原型 `int vsprintf(char *str, const char *format, va_list ap);`

说明 与 `sprintf` 相同，但参数以 `va_list` 提供。

返回值 成功返回写入的字符数，失败返回负值。

附加说明 不检查缓冲区大小，建议改用 `vsnprintf`。

相关函数 `vsnprintf`、`sprintf`

```
#include <stdio.h>
#include <stdarg.h>
```

```
#include <stdio.h>
#include <stdarg.h>

void build(char *buf, const char *fmt, ...)
{
    va_list ap;
    va_start(ap, fmt);
    vsprintf(buf, fmt, ap);
    va_end(ap);
}

int main(void)
{
    char buf[128];
    build(buf, "%s=%d", "port", 80);
    puts(buf);
    return 0;
}
```

vsscanf — 格式化字符串输入

头文件

原型 `int vsscanf(const char *str, const char *format, va_list ap);`

说明 与 `sscanf` 相同，但参数以 `va_list` 提供。

返回值 成功返回被赋值的参数个数，失败返回 `EOF`。

相关函数 `sscanf`、`vfscanf`、`vscanf`

```
#include <stdio.h>
#include <stdarg.h>
```

```
#include <stdio.h>
#include <stdarg.h>

int parse(const char *s, const char *fmt, ...)
{
    va_list ap;
    va_start(ap, fmt);
    int n = vsscanf(s, fmt, ap);
    va_end(ap);
    return n;
}

int main(void)
{
    int v;
    parse("123", "%d", &v);
    printf("%d\n", v);
    return 0;
}
```

第15章 文件及目录函数

本章函数用于文件的权限、属性、链接操作以及目录的读取与遍历。

access — 判断是否具有存取文件的权限

头文件 `#include <unistd.h>`

原型 `int access(const char *pathname, int mode);`

说明 检查调用进程对 `pathname` 的存取权限。`mode` 可取 `R_OK`（可读）、`W_OK`（可写）、`X_OK`（可执行）、`F_OK`（文件是否存在），可用按位或组合。

返回值 具备所有指定权限返回 0，否则返回 -1 并设置 `errno`（如 `EACCES`、`ENOENT`）。

附加说明 检查的是真实 uid/gid 的权限；检查结果与实际使用之间存在时间差，可能引发安全问题，应避免依赖它做权限判断。

相关函数 `stat`、`chmod`、`open`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    if (access("/etc/passwd", R_OK) == 0)
        puts("readable");
    else
        perror("access");
    return 0;
}
```

alphasort — 依字母顺序排序目录结构

头文件 `#include <dirent.h>`

原型 `int alphasort(const struct dirent **a, const struct dirent **b);`

说明 依字母顺序比较两个目录项的名称，供 `scandir` 作为排序函数使用。

返回值 `a` 小于、等于、大于 `b` 时分别返回负值、0、正值。

相关函数 `scandir`、`versionsort`

```
#include <dirent.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    struct dirent **list;
    int n = scandir("/tmp", &list, NULL, alphasort);
    for (int i = 0; i < n; i++) {
        printf("%s\n", list[i]->d_name);
        free(list[i]);
    }
    free(list);
    return 0;
}
```

`chdir` — 改变当前的工作目录

头文件 `#include <unistd.h>`

原型 `int chdir(const char *path);`

说明 将调用进程的当前工作目录改为 `path`。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`ENOENT`、`EACCES`）。

附加说明 只影响调用进程本身；为配合返回原目录，可先用 `open(".")` 保存，再用 `fchdir` 返回。

相关函数 `fchdir`、`getcwd`、`chroot`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    if (chdir("/tmp") < 0) {
        perror("chdir");
        return 1;
    }
    char cwd[256];
    printf("now in %s\n", getcwd(cwd, sizeof(cwd)));
    return 0;
}
```

chmod — 改变文件的权限

头文件

原型 `int chmod(const char *path, mode_t mode);`

说明 改变 `path` 指定文件的权限为 `mode`。权限可用八进制表示（如 `0644`），也可使用 `<sys/stat.h>` 中的 `S_IRUSR`、`S_IWUSR`、`S_IXUSR`、`S_IRWXG`、`S_ISUID`、`S_ISGID`、`S_ISVTX` 等常量组合。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EPERM`、`ENOENT`）。

附加说明 只有文件所有者或超级用户才能改变权限。

相关函数 `fchmod`、`chown`、`stat`

```
#include <sys/types.h>
#include <sys/stat.h>
```

```
#include <sys/types.h>
#include <sys/stat.h>
#include <stdio.h>

int main(void)
{
    if (chmod("/tmp/a.txt", S_IRUSR | S_IWUSR) < 0)
        perror("chmod");
    return 0;
}
```

chown — 改变文件的所有者

头文件

原型 `int chown(const char *path, uid_t owner, gid_t group);`

说明 改变 `path` 指定文件的所有者为 `owner`、所属组为 `group`；参数为 -1 表示对应项保持不变。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EPERM`）。

附加说明 只有超级用户可改变文件所有者；普通用户只能改变自己文件的组（且必须是其所属组）。对符号链接应使用 `lchown`。

相关函数 `fchown`、`lchown`、`chmod`

```
#include <sys/types.h>
#include <unistd.h>
```

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>

int main(void)   /* 需 root 权限 */
{
    if (chown("/tmp/a.txt", 0, 0) < 0)
        perror("chown");
    return 0;
}
```

chroot — 改变根目录

头文件 `#include <unistd.h>`

原型 `int chroot(const char *path);`

说明 将调用进程的根目录改为 `path`，此后该进程及其子进程无法访问该目录之外的文件（即 chroot jail）。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EPERM`）。

附加说明 需 root 权限；仅改变根目录并不等于完整的安全隔离，仍需注意权限与设备文件。

相关函数 `chdir`

```
#include <unistd.h>
#include <stdio.h>

int main(void)   /* 需 root 权限 */
{
    if (chroot("/tmp/jail") < 0) {
        perror("chroot");
        return 1;
    }
    chdir("/");
    return 0;
}
```

`closedir` — 关闭目录

头文件

原型 `int closedir(DIR *dirp);`

说明 关闭由 `opendir` 打开的目录流 `dirp`，并释放相关资源。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EBADF`）。

相关函数 `opendir`、`readdir`

```
#include <sys/types.h>
#include <dirent.h>
```

```
#include <sys/types.h>
#include <dirent.h>
#include <stdio.h>

int main(void)
{
    DIR *dp = opendir("/tmp");
    struct dirent *e;
    while ((e = readdir(dp)) != NULL)
        printf("%s\n", e->d_name);
    closedir(dp);
    return 0;
}
```

`fchdir` — 改变当前的工作目录

头文件	`#include <unistd.h>`
原型	`int fchdir(int fd);`
说明	将当前工作目录改为 `fd` 所指的目录（必须是已打开的目录描述词）。
返回值	成功返回 0，失败返回 -1 并设置 `errno`（`ENOTDIR`、`EBADF`）。
附加说明	常与 `open(".")` 配合，实现“进入目录后可安全返回”。
相关函数	`chdir`、`getcwd`、`open`

```
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>

int main(void)
{
    int fd = open(".", O_RDONLY);   /* 保存当前目录 */
    chdir("/tmp");
    printf("in /tmp: %s\n", getcwd(NULL, 0));
    fchdir(fd);                     /* 回到原目录 */
    close(fd);
    return 0;
}
```

`fchmod` — 改变文件的权限

头文件

原型 `int fchmod(int fd, mode_t mode);`

说明 与 `chmod` 相同，但操作对象为已打开的文件描述词 `fd`。

返回值 成功返回 0，失败返回 -1 并设置 `errno`。

附加说明 使用描述词可避免路径竞争（TOCTTOU）问题，较 `chmod` 安全。

相关函数 `chmod`、`fchown`

```
#include <sys/types.h>
#include <sys/stat.h>
```

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    int fd = open("/tmp/a.txt", O_WRONLY | O_CREAT, 0600);
    fchmod(fd, 0644);
    close(fd);
    return 0;
}
```

`fchown` — 改变文件的所有者

头文件

原型 `int fchown(int fd, uid_t owner, gid_t group);`

说明 与 `chown` 相同，但操作对象为已打开的文件描述词 `fd`。

返回值 成功返回 0，失败返回 -1 并设置 `errno`。

相关函数 `chown`、`lchown`、`fchmod`

```
#include <sys/types.h>
#include <unistd.h>
```

```
#include <sys/types.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>

int main(void)   /* 需 root 权限 */
{
    int fd = open("/tmp/a.txt", O_RDONLY);
    if (fchown(fd, 0, 0) < 0)
        perror("fchown");
    close(fd);
    return 0;
}
```

fstat — 由文件描述词取得文件状态

头文件

原型 `int fstat(int fd, struct stat *buf);`

说明 取得 `fd` 所指文件的状态，存入 `buf`。`struct stat` 常用成员：`st_mode`（类型与权限）、`st_ino`（inode）、`st_nlink`（链接数）、`st_uid`、`st_gid`、`st_size`（大小）、`st_atime` / `st_mtime` / `st_ctime`（访问/修改/状态变更时间）。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EBADF`）。

相关函数 `stat`、`lstat`

```
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
```

```
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>

int main(void)
{
    struct stat sb;
    int fd = open("/etc/hostname", O_RDONLY);
    fstat(fd, &sb);
    printf("size = %ld bytes\n", (long)sb.st_size);
    close(fd);
    return 0;
}
```

`ftruncate` — 改变文件大小

头文件

原型 `int ftruncate(int fd, off_t length);`

说明 将 `fd` 所指文件的长度改为 `length`；变长时以 0（文件空洞）填充，变短时超出部分被丢弃。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EINVAL`、`EBADF`）。

附加说明 文件必须以可写方式打开；以路径操作可使用 `truncate`。

相关函数 `truncate`、`lseek`

```
#include <unistd.h>
#include <sys/types.h>
```

```
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>

int main(void)
{
    int fd = open("/tmp/a.txt", O_RDWR | O_CREAT, 0644);
    ftruncate(fd, 1024);      /* 扩展为 1024 字节 */
    close(fd);
    return 0;
}
```

`ftw` — 遍历目录树

头文件 `#include <ftw.h>`

原型 `int ftw(const char *dir, int (*fn)(const char *file, const struct stat *sb, int flag), int depth);`

说明 从 `dir` 开始遍历目录树，为每个目录项调用一次 `fn`。`flag` 可为 `FTW_F`（文件）、`FTW_D`（目录）、`FTW_DNR`（不可读目录）、`FTW_NS`（stat 失败）、`FTW_SL`（符号链接）等；`depth` 为同时打开的最大目录数。

返回值 遍历完成且 `fn` 均返回 0 时返回 0；`fn` 返回非 0 时返回该值；失败返回 -1 并设置 `errno`。

附加说明 `fn` 返回 0 表示继续，返回非 0 表示中止；建议改用功能更完整的 `nftw`。

相关函数 `nftw`、`opendir`、`scandir`

```
#include <ftw.h>
#include <stdio.h>

int print_entry(const char *file, const struct stat *sb, int flag)
{
    if (flag == FTW_F)
        printf("%s (%ld bytes)\n", file, (long)sb->st_size);
    return 0;
}

int main(void)
{
    ftw("/tmp", print_entry, 16);
    return 0;
}
```

get_current_dir_name — 取得当前的工作目录

头文件 `#include <unistd.h>`

原型 `char *get_current_dir_name(void);`

说明 取得当前工作目录的绝对路径，空间由 `malloc` 配置。

返回值 成功返回路径字符串指针（需用 `free` 释放），失败返回 NULL 并设置 `errno`。

附加说明 为 GNU 扩展函数，需定义 `_GNU_SOURCE`；标准做法使用 `getcwd`。

相关函数 `getcwd`、`getwd`、`chdir`

```
#define _GNU_SOURCE
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    char *cwd = get_current_dir_name();
    printf("cwd = %s\n", cwd);
    free(cwd);
    return 0;
}
```

getcwd — 取得当前的工作目录

头文件 `#include <unistd.h>`

原型 `char *getcwd(char *buf, size_t size);`

说明 将当前工作目录的绝对路径写入 `buf`，`size` 为缓冲区大小；`buf` 为 NULL 时由函数用 `malloc` 配置（此时需自行 `free`）。

返回值 成功返回 `buf`，失败返回 NULL 并设置 `errno`（`ERANGE` 表示空间不足）。

相关函数 `chdir`、`getwd`、`get_current_dir_name`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    char cwd[512];
    if (getcwd(cwd, sizeof(cwd)))
        printf("cwd = %s\n", cwd);
    return 0;
}
```

getwd — 取得当前的工作目录

头文件 `#include <unistd.h>`

原型 `char *getwd(char *buf);`

说明 取得当前工作目录的绝对路径并存入 `buf`（需足够大，通常预留 `PATH_MAX` 字节）。

返回值 成功返回 `buf`，失败返回 NULL。

附加说明 已过时，建议使用 `getcwd` 以避免缓冲区溢出。

相关函数 `getcwd`、`get_current_dir_name`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    char buf[512];
    printf("cwd = %s\n", getwd(buf));
    return 0;
}
```

lchown — 改变文件的所有者

头文件

原型 `int lchown(const char *path, uid_t owner, gid_t group);`

说明 与 `chown` 相同，但若 `path` 为符号链接，则改变链接本身而非其指向的文件。

返回值 成功返回 0，失败返回 -1 并设置 `errno`。

相关函数 `chown`、`fchown`

```
#include <sys/types.h>
#include <unistd.h>
```

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>

int main(void)   /* 需 root 权限 */
{
    if (lchown("/tmp/link", 0, 0) < 0)   /* 只改链接本身 */
        perror("lchown");
    return 0;
}
```

link — 建立文件连接

头文件 `#include <unistd.h>`

原型 `int link(const char *oldpath, const char *newpath);`

说明 为已存在的 `oldpath` 建立一个新的硬链接 `newpath`，两者指向同一个 inode。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EEXIST`、`EXDEV`、`EMLINK`）。

附加说明 硬链接不能跨文件系统，也不能用于目录；建立符号链接应使用 `symlink`。

相关函数 `symlink`、`unlink`、`readlink`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    if (link("/tmp/a.txt", "/tmp/b.txt") < 0)
        perror("link");
    return 0;
}
```

lstat — 取得文件状态

头文件

原型 `int lstat(const char *path, struct stat *buf);`

说明 与 `stat` 相同，但 `path` 为符号链接时取得的是链接本身的状态，而非其指向文件的状态。

返回值 成功返回 0，失败返回 -1 并设置 `errno`。

相关函数 `stat`、`fstat`、`readlink`

```
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
```

```
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    struct stat sb;
    if (lstat("/tmp/link", &sb) == 0)
        printf("is link: %d\n", S_ISLNK(sb.st_mode));
    return 0;
}
```

nftw — 遍历目录树

头文件 `#include <ftw.h>`

原型 `int nftw(const char *dir, int (*fn)(const char *file, const struct stat *sb, int flag, struct FTW *s), int depth, int flags);`

说明 `ftw` 的增强版，回调多出 `struct FTW *s`（含 `base`、`level` 等层级信息），`flags` 可取 `FTW_CHDIR`（进入各目录）、`FTW_DEPTH`（深度优先，先处理目录内容）、`FTW_MOUNT`（不跨越挂载点）、`FTW_PHYS`（不跟随符号链接）。

返回值 遍历成功返回 0，`fn` 返回非 0 时返回该值，失败返回 -1。

相关函数 `ftw`、`opendir`、`stat`

```
#include <ftw.h>
#include <stdio.h>

int show(const char *file, const struct stat *sb, int flag, struct FTW *s)
{
    printf("level %d: %s\n", s->level, file);
    return 0;
}

int main(void)
{
    nftw("/tmp", show, 16, FTW_PHYS);
    return 0;
}
```

opendir — 打开目录

头文件

原型 `DIR *opendir(const char *name);`

说明 打开 `name` 指定的目录，返回目录流供 `readdir` 等函数读取。

返回值 成功返回目录流指针，失败返回 NULL 并设置 `errno`（`ENOENT`、`EACCES`、`ENOTDIR`）。

附加说明 使用完毕必须以 `closedir` 关闭。

相关函数 `readdir`、`closedir`、`scandir`

```
#include <sys/types.h>
#include <dirent.h>
```

```
#include <sys/types.h>
#include <dirent.h>
#include <stdio.h>

int main(void)
{
    DIR *dp = opendir("/etc");
    if (dp == NULL) {
        perror("opendir");
        return 1;
    }
    closedir(dp);
    return 0;
}
```

`readdir` — 读取目录

头文件

原型 `struct dirent *readdir(DIR *dirp);`

说明 从目录流 `dirp` 读取一个目录项并返回 `struct dirent`，其中 `d_ino` 为 inode 编号、`d_name` 为文件名；连续调用可读取所有项，读到目录尾返回 NULL。

返回值 成功返回 `struct dirent` 指针，读到目录尾返回 NULL（不设置 `errno`），出错返回 NULL 并设置 `errno`（`EBADF`）。

附加说明 返回的结构由静态内存提供，多次调用会被覆盖；目录项顺序与文件系统有关，不保证排序。

相关函数 `opendir`、`closedir`、`scandir`、`telldir`

```
#include <sys/types.h>
#include <dirent.h>
```

```
/* 列出目录内容，跳过 . 与 .. */
#include <sys/types.h>
#include <dirent.h>
#include <stdio.h>
#include <string.h>

int main(void)
{
    DIR *dp = opendir("/etc");
    struct dirent *e;
    while ((e = readdir(dp)) != NULL)
        if (strcmp(e->d_name, ".") && strcmp(e->d_name, ".."))
            printf("%s\n", e->d_name);
    closedir(dp);
    return 0;
}
```

readlink — 取得符号连接所指的文件

头文件 `#include <unistd.h>`

原型 `int readlink(const char *path, char *buf, size_t bufsiz);`

说明 读取符号链接 `path` 所指向的目标路径，存入 `buf`。

返回值 成功返回写入 `buf` 的字节数（不添加 `'\0'`），失败返回 -1 并设置 `errno`（`EINVAL` 表示不是符号链接）。

附加说明 `buf` 不会自动补 `'\0'`，需自行添加；取得完整绝对路径可用 `realpath`。

相关函数 `symlink`、`lstat`、`realpath`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    char buf[256];
    int n = readlink("/tmp/link", buf, sizeof(buf) - 1);
    if (n >= 0) {
        buf[n] = '\0';
        printf("points to %s\n", buf);
    }
    return 0;
}
```

realpath — 将相对目录路径转换成绝对路径

头文件 `#include <stdlib.h>`

原型 `char *realpath(const char *path, char *resolved_path);`

说明 将 `path` 中的相对路径、`.`、`..` 与所有符号链接展开，得到规范的绝对路径。

返回值 成功返回指向绝对路径的指针，失败返回 NULL 并设置 `errno`（`ENOENT`）。

附加说明 `resolved_path` 为 NULL 时由函数 `malloc` 配置（需 `free`），否则须至少有 `PATH_MAX` 字节。

相关函数 `readlink`、`getcwd`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    char *p = realpath("/tmp/link", NULL);
    printf("%s\n", p);
    free(p);
    return 0;
}
```

remove — 删除文件

头文件 `#include <stdio.h>`

原型 `int remove(const char *pathname);`

说明 删除 `pathname` 指定的文件；若为目录则相当于 `rmdir`，否则相当于 `unlink`。

返回值 成功返回 0，失败返回 -1 并设置 `errno`。

相关函数 `unlink`、`rmdir`、`rename`

```
#include <stdio.h>

int main(void)
{
    if (remove("/tmp/a.txt") < 0)
        perror("remove");
    return 0;
}
```

rename — 更改文件名称或位置

头文件 `#include <stdio.h>`

原型 `int rename(const char *oldpath, const char *newpath);`

说明 将文件（或目录）`oldpath` 改名为 `newpath`；若两者位于同一目录则为改名，否则为移动。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EXDEV`、`EISDIR`、`EEXIST`）。

附加说明 目标已存在时会被覆盖（同类型）；不同文件系统之间移动会失败，需自行拷贝。

相关函数 `link`、`unlink`、`remove`

```
#include <stdio.h>

int main(void)
{
    if (rename("/tmp/a.txt", "/tmp/c.txt") < 0)
        perror("rename");
    return 0;
}
```

`rewinddir` — 重设读取目录的位置为开头位置

头文件

原型 `void rewinddir(DIR *dirp);`

说明 将目录流 `dirp` 的读取位置重设到目录开头，但不关闭/重开目录，因此能看到新增的文件。

返回值 无返回值。

相关函数 `opendir`、`readdir`、`seekdir`、`telldir`

```
#include <sys/types.h>
#include <dirent.h>
```

```
#include <sys/types.h>
#include <dirent.h>
#include <stdio.h>

int main(void)
{
    DIR *dp = opendir("/tmp");
    readdir(dp);
    rewinddir(dp);           /* 回到开头重新读取 */
    struct dirent *e = readdir(dp);
    printf("first again: %s\n", e->d_name);
    closedir(dp);
    return 0;
}
```

scandir — 读取特定的目录数据

头文件 `#include <dirent.h>`

原型 `int scandir(const char *dir, struct dirent **namelist, int (*filter)(const struct dirent *), int (*compar)(const struct dirent , const struct dirent **));`

说明 读取 `dir` 目录中的项目，用 `filter` 筛选（NULL 表示全部），用 `compar` 排序（NULL 表示不排序，常用 `alphasort`），结果存入由函数配置的 `namelist` 数组。

返回值 成功返回项目个数，失败返回 -1 并设置 `errno`。

附加说明 `namelist` 及其每一项都由 `malloc` 配置，使用完毕须逐个 `free` 后再 `free(namelist)`。

相关函数 `alphasort`、`opendir`、`readdir`

```
/* 列出目录下所有以 ".log" 结尾的文件，并按字母排序 */
#include <dirent.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int only_log(const struct dirent *d)
{
    return strstr(d->d_name, ".log") != NULL;
}

int main(void)
{
    struct dirent **list;
    int n = scandir("/var/log", &list, only_log, alphasort);
    for (int i = 0; i < n; i++) {
        puts(list[i]->d_name);
        free(list[i]);
    }
    free(list);
    return 0;
}
```

seekdir — 设置下回读取目录的位置

头文件

原型 `void seekdir(DIR *dirp, long offset);`

说明 设置目录流 `dirp` 下一次 `readdir` 的位置，`offset` 由 `telldir` 取得。

返回值 无返回值。

相关函数 `telldir`、`rewinddir`、`readdir`

```
#include <sys/types.h>
#include <dirent.h>
```

```
#include <sys/types.h>
#include <dirent.h>
#include <stdio.h>

int main(void)
{
    DIR *dp = opendir("/etc");
    readdir(dp);
    long pos = telldir(dp);     /* 记住位置 */
    readdir(dp);
    seekdir(dp, pos);           /* 回到刚才的位置 */
    printf("%s\n", readdir(dp)->d_name);
    closedir(dp);
    return 0;
}
```

stat — 取得文件状态

头文件

原型 `int stat(const char *path, struct stat *buf);`

说明 取得 `path` 指定文件的状态并存入 `buf`；若 `path` 为符号链接，取得的是其指向文件的状态。可用 `S_ISREG`、`S_ISDIR`、`S_ISLNK`、`S_ISCHR`、`S_ISFIFO` 等宏判断类型，或用 `S_IRUSR` 等判断权限。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`ENOENT`、`EACCES`）。

相关函数 `fstat`、`lstat`、`access`

```
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
```

```
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
#include <stdio.h>
#include <time.h>

int main(void)
{
    struct stat sb;
    if (stat("/etc/hostname", &sb) < 0) {
        perror("stat");
        return 1;
    }
    printf("size=%ld mode=%o mtime=%s",
           (long)sb.st_size, sb.st_mode & 07777, ctime(&sb.st_mtime));
    return 0;
}
```

symlink — 建立文件符号连接

头文件 `#include <unistd.h>`

原型 `int symlink(const char *oldpath, const char *newpath);`

说明 建立一个指向 `oldpath` 的符号链接（软链接） `newpath`； `oldpath` 可以不存在，也可指向目录。

返回值 成功返回 0，失败返回 -1 并设置 `errno` （`EEXIST`、`EACCES`）。

附加说明 符号链接可跨文件系统，与硬链接不同。

相关函数 `link`、`readlink`、`unlink`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    if (symlink("/etc/hostname", "/tmp/hn") < 0)
        perror("symlink");
    return 0;
}
```

telldir — 取得目录流的读取位置

头文件

原型 `long telldir(DIR *dirp);`

说明 取得目录流 `dirp` 当前的读取位置，供 `seekdir` 恢复使用。

返回值 返回当前的读取位置，失败返回 -1 并设置 `errno`。

相关函数 `seekdir`、`rewinddir`、`readdir`

```
#include <sys/types.h>
#include <dirent.h>
```

```
#include <sys/types.h>
#include <dirent.h>
#include <stdio.h>

int main(void)
{
    DIR *dp = opendir("/etc");
    readdir(dp);
    printf("pos = %ld\n", telldir(dp));
    closedir(dp);
    return 0;
}
```

`truncate` — 改变文件大小

头文件

原型 `int truncate(const char *path, off_t length);`

说明 将 `path` 指定的文件长度改为 `length`；变长部分以 0 填充，变短则丢弃超出部分。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EINVAL`、`EACCES`）。

附加说明 必须对文件有写权限；已打开的文件可用 `ftruncate`。

相关函数 `ftruncate`、`lseek`

```
#include <unistd.h>
#include <sys/types.h>
```

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    if (truncate("/tmp/a.txt", 0) < 0)   /* 清空文件 */
        perror("truncate");
    return 0;
}
```

umask — 设置建立新文件时的权限遮罩

头文件

原型 `mode_t umask(mode_t mask);`

说明 设置文件权限遮罩，使之后建立文件的实际权限为 `mode & ~mask`，例如 `umask(022)` 时 `open(..., 0666)` 得到的权限是 0644。

返回值 返回设置前的遮罩值（此函数不会失败）。

附加说明 遮罩为该进程的属性，子进程会继承；常见默认值为 022。

相关函数 `open`、`creat`、`chmod`

```
#include <sys/types.h>
#include <sys/stat.h>
```

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>

int main(void)
{
    mode_t old = umask(0022);
    int fd = open("/tmp/a.txt", O_WRONLY | O_CREAT, 0666);  /* 实际 0644 */
    close(fd);
    umask(old);
    return 0;
}
```

`unlink` — 删除文件

头文件	`#include <unistd.h>`
原型	`int unlink(const char *pathname);`
说明	删除 `pathname` 指定的目录项；若该文件无其他硬链接且无进程打开，则释放其占用的空间。
返回值	成功返回 0，失败返回 -1 并设置 `errno`（`EISDIR`、`EACCES`）。
附加说明	删除目录应使用 `rmdir`；若文件正被进程打开，删除会延后到所有描述词关闭后生效，因此可用来建立“临时文件”。
相关函数	`link`、`remove`、`rmdir`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    if (unlink("/tmp/a.txt") < 0)
        perror("unlink");
    return 0;
}
```

`utime` — 修改文件的存取时间和更改时间

头文件

原型 `int utime(const char *filename, const struct utimbuf *times);`

说明 修改 `filename` 的存取时间（atime）与更改时间（mtime）。`struct utimbuf` 含 `actime` 与 `modtime`；`times` 为 NULL 时两者都设为当前时间。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EACCES`、`EPERM`）。

附加说明 只有文件所有者或超级用户可设置非当前时间；需要微秒精度可使用 `utimes`。

相关函数 `utimes`、`stat`

```
#include <sys/types.h>
#include <utime.h>
```

```
#include <sys/types.h>
#include <utime.h>
#include <time.h>
#include <stdio.h>

int main(void)
{
    struct utimbuf t;
    t.actime = t.modtime = time(NULL);   /* 设为当前时间 */
    if (utime("/tmp/a.txt", &t) < 0)
        perror("utime");
    return 0;
}
```

`utimes` — 修改文件的存取时间和更改时间

头文件

原型 `int utimes(const char *filename, const struct timeval tv[2]);`

说明 以微秒精度修改文件的存取时间与更改时间。`tv[0]` 为 atime，`tv[1]` 为 mtime；传 NULL 表示设为当前时间。

返回值 成功返回 0，失败返回 -1 并设置 `errno`。

相关函数 `utime`、`stat`

```
#include <sys/types.h>
#include <sys/time.h>
```

```
#include <sys/time.h>
#include <stdio.h>

int main(void)
{
    struct timeval tv[2];
    gettimeofday(&tv[0], NULL);
    tv[1] = tv[0];
    if (utimes("/tmp/a.txt", tv) < 0)
        perror("utimes");
    return 0;
}
```

第16章 信号函数

本章函数用于信号的发送、处理、阻塞与等待，主要声明于 `<signal.h>`。

alarm — 设置信号传送闹钟

头文件 `#include <unistd.h>`

原型 `unsigned int alarm(unsigned int seconds);`

说明 设置定时器，在 `seconds` 秒后向进程发送 `SIGALRM` 信号；`seconds` 为 0 表示取消已设置的闹钟。新的设置会取代旧的。

返回值 返回先前闹钟剩余的秒数；若先前未设置则返回 0。

附加说明 信号默认动作是终止进程，因此必须安装处理函数；精度以秒为单位，需要更高精度可用 `setitimer`。

相关函数 `signal`、`pause`、`sleep`

```
#include <unistd.h>
#include <signal.h>
#include <stdio.h>

void handler(int sig)
{
    printf("SIGALRM received (%d)\n", sig);
}

int main(void)
{
    signal(SIGALRM, handler);
    alarm(2);
    pause();                /* 等待信号 */
    return 0;
}
```

`kill` — 传送信号给指定的进程

头文件

原型 `int kill(pid_t pid, int sig);`

说明 向 `pid` 指定的进程（或进程组）发送 `sig` 信号。`pid` 大于 0 表示指定进程；等于 0 表示同组进程；等于 -1 表示所有有权限的进程；小于 -1 表示进程组 `-pid`。`sig` 为 0 表示仅检测进程是否存在。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`ESRCH`、`EPERM`）。

相关函数 `raise`、`signal`、`sigqueue`

```
#include <sys/types.h>
#include <signal.h>
```

```
#include <sys/types.h>
#include <signal.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        pause();              /* 等待被终止 */
        _exit(0);
    }
    sleep(1);
    kill(pid, SIGTERM);       /* 通知子进程结束 */
    printf("signal sent\n");
    return 0;
}
```

pause — 让进程暂停直到信号出现

头文件 `#include <unistd.h>`

原型 `int pause(void);`

说明 使调用进程睡眠（挂起），直到收到任何会被处理的信号为止。

返回值 只有在信号被捕捉且处理函数返回时才返回 -1，并设置 `errno` 为 `EINTR`。

附加说明 常用于与 `alarm` 配合实现定时等待。

相关函数 `alarm`、`sigsuspend`、`sigpause`

```
#include <unistd.h>
#include <signal.h>
#include <stdio.h>

void on_int(int sig) { printf("got signal %d\n", sig); }

int main(void)
{
    signal(SIGINT, on_int);
    printf("waiting for signal...\n");
    pause();
    return 0;
}
```

psignal — 列出信号描述和指定字符串

头文件 `#include <signal.h>`

原型 `void psignal(int sig, const char *s);`

说明 将字符串 `s`、冒号与 `sig` 对应的信号描述输出到标准错误（stderr），常用于错误信息提示。

返回值 无返回值。

附加说明 与 `perror` 类似，但针对信号编号；可用 `strsignal` 取得纯字符串。

相关函数 `strsignal`、`perror`

```
#include <signal.h>
#include <stdio.h>

int main(void)
{
    psignal(SIGSEGV, "child died");   /* child died: Segmentation fault */
    return 0;
}
```

raise — 传送信号给目前的进程

头文件 `#include <signal.h>`

原型 `int raise(int sig);`

说明 向调用进程自身发送 `sig` 信号，等价于 `kill(getpid(), sig)`（单线程程序中）。

返回值 成功返回 0，失败返回非 0。

相关函数 `kill`、`abort`、`signal`

```
#include <signal.h>
#include <stdio.h>

void handler(int sig) { printf("caught %d\n", sig); }

int main(void)
{
    signal(SIGUSR1, handler);
    raise(SIGUSR1);          /* 给自己发信号 */
    return 0;
}
```

sigaction — 查询或设置信号处理方式

头文件 `#include <signal.h>`

原型 `int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);`

说明 查询或设置 `signum` 信号的处理方式。`struct sigaction` 主要成员：`sa_handler`（处理函数，可为 `SIG_DFL`、`SIG_IGN` 或自定函数）、`sa_sigaction`（配合 `SA_SIGINFO` 使用，可取得更多信息）、`sa_mask`（处理期间额外阻塞的信号集）、`sa_flags`（如 `SA_RESTART`、`SA_SIGINFO`、`SA_NOCLDSTOP`）、`sa_restorer`（已废弃）。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EINVAL`）。

附加说明 比 `signal` 可移植、功能更完整，建议优先使用；`SIGKILL` 与 `SIGSTOP` 无法被捕捉或忽略。

相关函数 `signal`、`sigprocmask`、`sigemptyset`

```
#include <signal.h>
#include <stdio.h>
#include <string.h>

void handler(int sig)
{
    printf("got SIGINT (%d)\n", sig);
}

int main(void)
{
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;      /* 被中断的系统调用自动重启 */
    sigaction(SIGINT, &sa, NULL);
    pause();
    return 0;
}
```

sigaddset — 增加一个信号至信号集

头文件 `#include <signal.h>`

原型 `int sigaddset(sigset_t *set, int signum);`

说明 将信号 `signum` 加入信号集 `set`。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EINVAL` 表示信号编号无效）。

相关函数 `sigdelset`、`sigemptyset`、`sigfillset`

```
#include <signal.h>
#include <stdio.h>

int main(void)
{
    sigset_t set;
    sigemptyset(&set);
    sigaddset(&set, SIGINT);
    sigaddset(&set, SIGTERM);
    printf("SIGINT in set: %d\n", sigismember(&set, SIGINT));
    return 0;
}
```

sigdelset — 从信号集里删除一个信号

头文件 `#include <signal.h>`

原型 `int sigdelset(sigset_t *set, int signum);`

说明 从信号集 `set` 中删除信号 `signum`。

返回值 成功返回 0，失败返回 -1 并设置 `errno`。

相关函数 `sigaddset`、`sigemptyset`

```
#include <signal.h>
#include <stdio.h>

int main(void)
{
    sigset_t set;
    sigemptyset(&set);
    sigaddset(&set, SIGINT);
    sigdelset(&set, SIGINT);
    printf("in set: %d\n", sigismember(&set, SIGINT));   /* 0 */
    return 0;
}
```

sigemptyset — 初始化信号集

头文件 `#include <signal.h>`

原型 `int sigemptyset(sigset_t *set);`

说明 将信号集 `set` 初始化为空，即不包含任何信号。使用前必须先调用本函数或 `sigfillset`。

返回值 成功返回 0，失败返回 -1 并设置 `errno`。

相关函数 `sigfillset`、`sigaddset`、`sigdelset`

```
#include <signal.h>
#include <stdio.h>

int main(void)
{
    sigset_t set;
    sigemptyset(&set);
    printf("empty: %d\n", sigismember(&set, SIGINT));   /* 0 */
    return 0;
}
```

sigfillset — 将所有信号加入至信号集

头文件 `#include <signal.h>`

原型 `int sigfillset(sigset_t *set);`

说明 将所有信号加入信号集 `set`，即包含全部信号。

返回值 成功返回 0，失败返回 -1 并设置 `errno`。

附加说明 配合 `sigprocmask(SIG_BLOCK, ...)` 可暂时阻塞所有信号，用于保护临界区。

相关函数 `sigemptyset`、`sigprocmask`

```
#include <signal.h>
#include <stdio.h>

int main(void)
{
    sigset_t set;
    sigfillset(&set);
    printf("all: %d\n", sigismember(&set, SIGINT));   /* 1 */
    return 0;
}
```

`sigismember` — 测试某个信号是否已加入至信号集里

头文件 `#include <signal.h>`

原型 `int sigismember(const sigset_t *set, int signum);`

说明 测试信号 `signum` 是否已加入信号集 `set`。

返回值 已加入返回 1，未加入返回 0，失败返回 -1 并设置 `errno`。

相关函数 `sigaddset`、`sigdelset`

```
#include <signal.h>
#include <stdio.h>

int main(void)
{
    sigset_t set;
    sigemptyset(&set);
    sigaddset(&set, SIGQUIT);
    if (sigismember(&set, SIGQUIT))
        puts("SIGQUIT is blocked");
    return 0;
}
```

`signal` — 设置信号处理方式

头文件 `#include <signal.h>`

原型 `void (*signal(int signum, void (*handler)(int)))(int);`

说明 设置 `signum` 信号的处理函数为 `handler`，也可以是 `SIG_DFL`（默认动作）或 `SIG_IGN`（忽略）。

返回值 返回先前的信号处理函数指针，失败返回 `SIG_ERR`。

附加说明 行为在不同 UNIX 系统上略有差异（是否自动重启被中断的系统调用），建议改用 `sigaction`；`SIGKILL`、`SIGSTOP` 无法被捕捉。

相关函数 `sigaction`、`raise`、`kill`

```
#include <signal.h>
#include <unistd.h>
#include <stdio.h>

void handler(int sig)
{
    printf("signal %d\n", sig);
}

int main(void)
{
    signal(SIGINT, handler);      /* Ctrl-C */
    signal(SIGTERM, handler);
    while (1)
        pause();
    return 0;
}
```

`sigpause` — 暂停直到信号到来

头文件 `#include <signal.h>`

原型 `int sigpause(int sigmask);`

说明 将信号遮罩设为 `sigmask`，然后暂停等待信号到来，返回后恢复原来的遮罩（通常用于实现原子性的“解除阻塞并等待”）。

返回值 总是返回 -1 并设置 `errno` 为 `EINTR`。

附加说明 已过时，建议使用 `sigsuspend`。

相关函数 `sigsuspend`、`pause`、`sigprocmask`

```
#include <signal.h>
#include <stdio.h>

void handler(int sig) { }

int main(void)
{
    signal(SIGUSR1, handler);
    printf("waiting\n");
    sigpause(0);        /* 解除阻塞并等待 */
    printf("resumed\n");
    return 0;
}
```

`sigpending` — 查询被搁置的信号

头文件 `#include <signal.h>`

原型 `int sigpending(sigset_t *set);`

说明 取得当前被阻塞（搁置，pending）而尚未递送的信号集合，存入 `set`。

返回值 成功返回 0，失败返回 -1 并设置 `errno`。

相关函数 `sigprocmask`、`sigismember`

```
#include <signal.h>
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    sigset_t mask, pend;
    sigemptyset(&mask);
    sigaddset(&mask, SIGINT);
    sigprocmask(SIG_BLOCK, &mask, NULL);
    raise(SIGINT);                  /* 被阻塞，进入 pending */
    sigpending(&pend);
    printf("SIGINT pending: %d\n", sigismember(&pend, SIGINT));
    return 0;
}
```

sigprocmask — 查询或设置信号遮罩

头文件 `#include <signal.h>`

原型 `int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);`

说明 查询或改变进程的信号遮罩（阻塞的信号集合）。`how` 可为 `SIG_BLOCK`（加入 `set`）、`SIG_UNBLOCK`（移除 `set`）、`SIG_SETMASK`（直接设为 `set`）；`oldset` 不为 NULL 时保存原遮罩。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EINVAL`）。

附加说明 被阻塞的信号不会被递送，而是处于搁置状态，解除阻塞后立即递送；`SIGKILL` 与 `SIGSTOP` 无法被阻塞。

相关函数 `sigpending`、`sigsuspend`、`sigaction`

```
/* 保护临界区，暂时阻塞 SIGINT */
#include <signal.h>
#include <stdio.h>

int main(void)
{
    sigset_t new, old;
    sigemptyset(&new);
    sigaddset(&new, SIGINT);
    sigprocmask(SIG_BLOCK, &new, &old);
    puts("critical section");
    sigprocmask(SIG_SETMASK, &old, NULL);   /* 恢复 */
    return 0;
}
```

`sigsuspend` — 暂停直到信号到来

头文件 `#include <signal.h>`

原型 `int sigsuspend(const sigset_t *mask);`

说明 暂时把信号遮罩替换为 `mask` 并挂起进程，直到收到信号；返回前把遮罩恢复为原值。可原子地完成“解除阻塞 + 等待”，避免信号在两者之间丢失。

返回值 总是返回 -1 并设置 `errno` 为 `EINTR`。

相关函数 `sigprocmask`、`pause`、`sigpause`

```
#include <signal.h>
#include <stdio.h>
#include <unistd.h>

void handler(int sig) { }

int main(void)
{
    sigset_t mask, old;
    sigemptyset(&mask);
    sigaddset(&mask, SIGUSR1);
    sigprocmask(SIG_BLOCK, &mask, &old);   /* 先阻塞 */
    signal(SIGUSR1, handler);
    printf("waiting for SIGUSR1 (pid=%d)\n", getpid());
    sigsuspend(&old);                      /* 解除阻塞并等待 */
    printf("got it\n");
    return 0;
}
```

`sleep` — 让进程暂停执行一段时间

头文件	`#include <unistd.h>`
原型	`unsigned int sleep(unsigned int seconds);`
说明	使调用进程暂停执行 `seconds` 秒，或直到收到信号而被中断。
返回值	正常结束返回 0；被信号中断则返回尚未睡足的秒数。
附加说明	若在 `sleep` 期间安装 `SIGALRM` 处理函数可能影响 `alarm` 的行为；需要微秒级精度可用 `usleep` 或 `nanosleep`。
相关函数	`usleep`、`alarm`、`pause`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    printf("start\n");
    unsigned int left = sleep(3);
    printf("woke up, %u seconds left\n", left);
    return 0;
}
```

第17章 错误处理函数

ferror — 检查文件流是否有错误发生

头文件 `#include <stdio.h>`

原型 `int ferror(FILE *stream);`

说明 检查 `stream` 所指文件流的错误旗标是否被设置（即上一次 I/O 操作是否出错）。

返回值 有错误发生返回非 0，否则返回 0。

附加说明 错误旗标会一直保留，需用 `clearerr` 清除；通常与 `feof` 配合使用以判断读取失败的原因。

相关函数 `feof`、`clearerr`、`perror`

```
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("/root/secret.txt", "r");
    if (fp == NULL) {
        perror("fopen");
        return 1;
    }
    fgetc(fp);
    if (ferror(fp))
        printf("I/O error occurred\n");
    fclose(fp);
    return 0;
}
```

perror — 打印出错误原因信息字符串

头文件 `#include <stdio.h>`

原型 `void perror(const char *s);`

说明 将字符串 `s`、冒号与全局变量 `errno` 所对应的错误信息输出到标准错误（stderr），形式为 “s: 错误描述”。

返回值 无返回值。

附加说明 必须在出错后立刻调用，否则 `errno` 可能被后续函数覆盖；这是最常用的错误提示方式。

相关函数 `strerror`、`ferror`、`errno`

```
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("/nonexistent/file", "r");
    if (fp == NULL)
        perror("fopen");   /* fopen: No such file or directory */
    return 0;
}
```

strerror — 返回错误原因的描述字符串

头文件 `#include <string.h>`

原型 `char *strerror(int errnum);`

说明 依错误码 `errnum` 返回对应的错误描述字符串，内容由静态内存提供（多线程环境建议使用 `strerror_r`）。

返回值 返回错误描述字符串指针；错误码未知时返回类似 “Unknown error nnn” 的内容。

附加说明 适合自定错误信息格式或写入日志时使用。

相关函数 `perror`、`errno`、`strerror_r`

```
#include <string.h>
#include <errno.h>
#include <stdio.h>

int main(void)
{
    for (int e = 1; e <= 3; e++)
        printf("%d: %s\n", e, strerror(e));
    printf("ENOENT: %s\n", strerror(ENOENT));
    return 0;
}
```

第18章 管道相关函数

本章函数用于建立管道，实现进程间的单向数据流。

`mkfifo` — 建立具名管道

头文件

原型 `int mkfifo(const char *pathname, mode_t mode);`

说明 建立具名管道（FIFO）特殊文件 `pathname`，权限为 `mode`。具名管道存在于文件系统中，可被任意进程以路径打开，用于无亲缘关系的进程间通信。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EEXIST`、`EACCES`）。

附加说明 打开 FIFO 时若只以只读或只写方式打开，会阻塞直到另一端也被打开；可用 `O_NONBLOCK` 避免。数据不落盘，仅在内核缓冲中传递。

相关函数 `pipe`、`open`、`popen`

```
#include <sys/types.h>
#include <sys/stat.h>
```

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>

int main(void)
{
    mkfifo("/tmp/myfifo", 0666);
    int fd = open("/tmp/myfifo", O_WRONLY);
    write(fd, "hello fifo\n", 11);
    close(fd);
    return 0;
}
```

`pclose` — 关闭管道 I/O

头文件	`#include <stdio.h>`
原型	`int pclose(FILE *stream);`
说明	关闭由 `popen` 建立的管道，并等待对应命令结束。
返回值	返回命令的结束状态（可用 `WEXITSTATUS` 等宏解析）；无法取得状态则返回 -1 并设置 `errno`。
附加说明	不使用 `fclose` 是为了能取得子进程的退出状态。
相关函数	`popen`、`wait`、`system`

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    FILE *fp = popen("ls -l /tmp", "r");
    char buf[256];
    while (fgets(buf, sizeof(buf), fp))
        fputs(buf, stdout);
    int status = pclose(fp);
    printf("exit=%d\n", WEXITSTATUS(status));
    return 0;
}
```

pipe — 建立管道

头文件 `#include <unistd.h>`

原型 `int pipe(int pipefd[2]);`

说明 建立匿名管道，`pipefd[0]` 为读取端，`pipefd[1]` 为写入端。写入端的数据可从读取端按先进先出的顺序读出。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EMFILE`）。

附加说明 管道单向，无名称，通常配合 `fork` 用于有亲缘关系的进程间通信（父子进程需各自关闭不用的端口）；管道满时写入阻塞，空时读取阻塞。

相关函数 `mkfifo`、`fork`、`dup2`

```
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <sys/wait.h>

int main(void)
{
    int fd[2];
    char buf[128];
    pipe(fd);
    if (fork() == 0) {                  /* 子进程写入 */
        close(fd[0]);
        write(fd[1], "hello pipe", 10);
        close(fd[1]);
        _exit(0);
    }
    close(fd[1]);                       /* 父进程读取 */
    int n = read(fd[0], buf, sizeof(buf) - 1);
    buf[n] = '\0';
    printf("got: %s\n", buf);
    close(fd[0]);
    wait(NULL);
    return 0;
}
```

popen — 建立管道 I/O

头文件 `#include <stdio.h>`

原型 `FILE *popen(const char *command, const char *type);`

说明 建立管道并启动 shell 执行 `command`，返回文件指针。`type` 为 `"r"` 时可从该文件指针读取命令的输出；为 `"w"` 时可写入数据作为命令的输入。

返回值 成功返回文件指针，失败返回 NULL 并设置 `errno`。

附加说明 数据经由 shell 处理，注意命令注入风险；须以 `pclose` 关闭并回收子进程。

相关函数 `pclose`、`system`、`pipe`、`fork`

```
/* 读取命令输出 */
#include <stdio.h>

int main(void)
{
    FILE *fp = popen("date", "r");
    char buf[128];
    if (fgets(buf, sizeof(buf), fp))
        printf("now: %s", buf);
    pclose(fp);
    return 0;
}
```

第19章 Socket 相关函数

本章函数用于网络编程，主要声明于 `<sys/types.h>`、`<sys/socket.h>`、`<netinet/in.h>`、`<arpa/inet.h>`、`<netdb.h>`。编写网络程序时应使用 `htons`/`htonl` 等函数处理字节序，避免可移植性问题。

accept — 接受 socket 连线

头文件

原型 `int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);`

说明 从处于监听状态的 `sockfd` 接受一个客户端连接，并返回一个新的 socket 描述词专门用于与该客户端通信；`addr` 用于取得客户端地址，`addrlen` 为输入输出参数。若无连接请求，默认会阻塞（可设为非阻塞）。

返回值 成功返回新的 socket 描述词，失败返回 -1 并设置 `errno`（`EBADF`、`EINTR`）。

相关函数 `socket`、`bind`、`listen`、`connect`

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdio.h>

int main(void)
{
    int sfd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(8080);
    addr.sin_addr.s_addr = INADDR_ANY;
    bind(sfd, (struct sockaddr *)&addr, sizeof(addr));
    listen(sfd, 5);
    struct sockaddr_in cli;
    socklen_t len = sizeof(cli);
    int cfd = accept(sfd, (struct sockaddr *)&cli, &len);
    printf("client connected, fd=%d\n", cfd);
    close(cfd);
    close(sfd);
    return 0;
}
```

`bind` — 对 socket 定位

头文件

原型 `int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);`

说明 将地址（IP 与端口）绑定到 `sockfd` 上。服务器端通常必须调用，客户端一般由系统自动分配端口。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EADDRINUSE` 表示地址已被占用）。

附加说明 端口小于 1024 需 root 权限；出现 “Address already in use” 时可用 `setsockopt(SO_REUSEADDR)` 解决。

相关函数 `socket`、`listen`、`accept`

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <stdio.h>

int main(void)
{
    int sfd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(8888);
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    if (bind(sfd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("bind");
        return 1;
    }
    close(sfd);
    return 0;
}
```

`connect` — 建立 socket 连线

头文件

原型 `int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);`

说明 以 `sockfd` 向 `addr` 指定的服务器发起连接请求（TCP 为三次握手），成功后即可收发数据。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`ECONNREFUSED`、`ETIMEDOUT`、`EINPROGRESS` 表示非阻塞连接正在进行）。

相关函数 `socket`、`bind`、`accept`、`send`

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in svr = {0};
    svr.sin_family = AF_INET;
    svr.sin_port = htons(80);
    svr.sin_addr.s_addr = inet_addr("93.184.216.34");
    if (connect(fd, (struct sockaddr *)&svr, sizeof(svr)) < 0) {
        perror("connect");
        return 1;
    }
    write(fd, "GET / HTTP/1.0\r\n\r\n", 18);
    close(fd);
    return 0;
}
```

endprotoent — 结束网络协议数据的读取

头文件 `#include <netdb.h>`

原型 `void endprotoent(void);`

说明 关闭由 `getprotoent` 打开的协议数据文件（`/etc/protocols`）。

返回值 无返回值。

相关函数 `getprotoent`、`setprotoent`

```
#include <netdb.h>
#include <stdio.h>

int main(void)
{
    struct protoent *p;
    setprotoent(1);
    while ((p = getprotoent()) != NULL)
        printf("%s %d\n", p->p_name, p->p_proto);
    endprotoent();
    return 0;
}
```

endservent — 结束网络服务数据的读取

头文件 `#include <netdb.h>`

原型 `void endservent(void);`

说明 关闭由 `getservent` 打开的网络服务数据文件（`/etc/services`）。

返回值 无返回值。

相关函数 `getservent`、`setservent`

```
#include <netdb.h>
#include <stdio.h>

int main(void)
{
    struct servent *s;
    while ((s = getservent()) != NULL)
        printf("%s %d\n", s->s_name, ntohs(s->s_port));
    endservent();
    return 0;
}
```

gethostbyaddr — 由 IP 地址取得网络数据

头文件 `#include <netdb.h>`

原型 `struct hostent *gethostbyaddr(const void *addr, socklen_t len, int type);`

说明 依 IP（网络字节序的二进制地址）进行反向查询，取得主机名称等信息。`len` 为地址长度（IPv4 为 4），`type` 为地址族（如 `AF_INET`）。

返回值 成功返回 `struct hostent` 指针（含 `h_name`、`h_aliases`、`h_addr_list` 等），失败返回 NULL 并设置 `h_errno`。

附加说明 已过时，建议使用可重入的 `getnameinfo`。

相关函数 `gethostbyname`、`getnameinfo`、`herror`

```
#include <netdb.h>
#include <arpa/inet.h>
#include <stdio.h>

int main(void)
{
    struct in_addr ip;
    inet_aton("127.0.0.1", &ip);
    struct hostent *h = gethostbyaddr(&ip, sizeof(ip), AF_INET);
    if (h)
        printf("name = %s\n", h->h_name);
    return 0;
}
```

gethostbyname — 由主机名称取得网络数据

头文件 `#include <netdb.h>`

原型 `struct hostent *gethostbyname(const char *name);`

说明 依主机名称（如 `www.example.com`）查询其 IP 地址，结果存入 `struct hostent`，其中 `h_addr_list[0]` 为第一组网络字节序的 IP 地址。

返回值 成功返回 `struct hostent` 指针，失败返回 NULL 并设置 `h_errno`（如 `HOST_NOT_FOUND`）。

附加说明 返回静态内存，不可重入；新程序建议使用 `getaddrinfo`。

相关函数 `gethostbyaddr`、`getaddrinfo`、`inet_ntoa`

```
#include <netdb.h>
#include <arpa/inet.h>
#include <stdio.h>

int main(void)
{
    struct hostent *h = gethostbyname("localhost");
    if (h) {
        struct in_addr a;
        memcpy(&a, h->h_addr_list[0], sizeof(a));
        printf("%s -> %s\n", h->h_name, inet_ntoa(a));
    }
    return 0;
}
```

getprotobyname — 由网络协议名称取得协议数据

头文件 `#include <netdb.h>`

原型 `struct protoent *getprotobyname(const char *name);`

说明 依协议名称（如 `"tcp"`、`"udp"`、`"icmp"`）查询协议编号等数据。

返回值 成功返回 `struct protoent` 指针（含 `p_name`、`p_aliases`、`p_proto`），失败返回 NULL。

相关函数 `getprotobynumber`、`getprotoent`

```
#include <netdb.h>
#include <stdio.h>

int main(void)
{
    struct protoent *p = getprotobyname("tcp");
    if (p)
        printf("tcp = %d\n", p->p_proto);   /* 6 */
    return 0;
}
```

getprotobynumber — 由网络协议编号取得协议数据

头文件 `#include <netdb.h>`

原型 `struct protoent *getprotobynumber(int proto);`

说明 依协议编号（如 6、17）查询协议名称等数据。

返回值 成功返回 `struct protoent` 指针，失败返回 NULL。

相关函数 `getprotobyname`、`getprotoent`

```
#include <netdb.h>
#include <stdio.h>

int main(void)
{
    struct protoent *p = getprotobynumber(17);
    if (p)
        printf("17 = %s\n", p->p_name);   /* udp */
    return 0;
}
```

getprotoent — 取得网络协议数据

头文件 `#include <netdb.h>`

原型 `struct protoent *getprotoent(void);`

说明 从协议数据文件（`/etc/protocols`）读取一项协议数据，连续调用可逐项读取。

返回值 成功返回 `struct protoent` 指针，读到文件尾或失败返回 NULL。

附加说明 使用完毕应调用 `endprotoent`。

相关函数 `setprotoent`、`endprotoent`、`getprotobyname`

```
#include <netdb.h>
#include <stdio.h>

int main(void)
{
    struct protoent *p;
    setprotoent(0);
    while ((p = getprotoent()) != NULL)
        printf("%d %s\n", p->p_proto, p->p_name);
    endprotoent();
    return 0;
}
```

getservbyname — 依名称取得网络服务的数据

头文件 `#include <netdb.h>`

原型 `struct servent *getservbyname(const char *name, const char *proto);`

说明 依服务名称与协议（如 `"http"`、`"tcp"`）查询端口号等数据，端口号以网络字节序存放于 `s_port`，需用 `ntohs` 转换。

返回值 成功返回 `struct servent` 指针，失败返回 NULL。

相关函数 `getservbyport`、`getservent`

```
#include <netdb.h>
#include <arpa/inet.h>
#include <stdio.h>

int main(void)
{
    struct servent *s = getservbyname("http", "tcp");
    if (s)
        printf("http port = %d\n", ntohs(s->s_port));   /* 80 */
    return 0;
}
```

getservbyport — 依 port 号码取得网络服务的数据

头文件 `#include <netdb.h>`

原型 `struct servent *getservbyport(int port, const char *proto);`

说明 依端口号（须为网络字节序，可用 `htons` 转换）与协议查询服务名称等数据。

返回值 成功返回 `struct servent` 指针，失败返回 NULL。

相关函数 `getservbyname`、`getservent`

```
#include <netdb.h>
#include <arpa/inet.h>
#include <stdio.h>

int main(void)
{
    struct servent *s = getservbyport(htons(22), "tcp");
    if (s)
        printf("port 22 = %s\n", s->s_name);   /* ssh */
    return 0;
}
```

getservent — 取得主机网络服务的数据

头文件	`#include <netdb.h>`
原型	`struct servent *getservent(void);`
说明	从服务数据文件（`/etc/services`）读取一项服务数据，连续调用可逐项读取。
返回值	成功返回 `struct servent` 指针，读到文件尾或失败返回 NULL。
附加说明	使用完毕应调用 `endservent`。
相关函数	`setservent`、`endservent`、`getservbyname`

```
#include <netdb.h>
#include <arpa/inet.h>
#include <stdio.h>

int main(void)
{
    struct servent *s;
    while ((s = getservent()) != NULL)
        printf("%-12s %d/%s\n", s->s_name, ntohs(s->s_port), s->s_proto);
    endservent();
    return 0;
}
```

`getsockopt` — 取得 socket 状态

头文件

原型 `int getsockopt(int sockfd, int level, int optname, void *optval, socklen_t *optlen);`

说明 取得 `sockfd` 的选项值。`level` 通常为 `SOL_SOCKET`（通用选项）或协议号（如 `IPPROTO_TCP`），`optname` 如 `SO_TYPE`、`SO_ERROR`、`SO_RCVBUF`、`SO_SNDBUF`、`SO_KEEPALIVE`、`SO_REUSEADDR`、`TCP_NODELAY`。

返回值 成功返回 0，失败返回 -1 并设置 `errno`。

相关函数 `setsockopt`、`fcntl`

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
#include <sys/types.h>
#include <sys/socket.h>
#include <stdio.h>

int main(void)
{
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    int val;
    socklen_t len = sizeof(val);
    getsockopt(fd, SOL_SOCKET, SO_RCVBUF, &val, &len);
    printf("recv buffer = %d\n", val);
    close(fd);
    return 0;
}
```

herror — 打印出网络错误原因信息字符串

头文件 `#include <netdb.h>`

原型 `void herror(const char *s);`

说明 将字符串 `s`、冒号与 `h_errno` 所对应的网络错误信息输出到标准错误，用于 `gethostbyname` 等函数的错误提示。

返回值 无返回值。

附加说明 已过时，建议使用 `getaddrinfo` 配合 `gai_strerror`。

相关函数 `hstrerror`、`gethostbyname`、`perror`

```
#include <netdb.h>

int main(void)
{
    struct hostent *h = gethostbyname("no.such.host.example");
    if (h == NULL)
        herror("gethostbyname");
    return 0;
}
```

hstrerror — 返回网络错误原因的描述字符串

头文件 `#include <netdb.h>`

原型 `const char *hstrerror(int err);`

说明 依网络错误码 `err`（如 `HOST_NOT_FOUND`、`TRY_AGAIN`、`NO_RECOVERY`、`NO_ADDRESS`）返回对应的错误描述字符串。

返回值 返回错误描述字符串指针。

相关函数 `herror`、`gethostbyname`

```
#include <netdb.h>
#include <stdio.h>

int main(void)
{
    printf("%s\n", hstrerror(HOST_NOT_FOUND));
    return 0;
}
```

htonl — 将 32 位主机字符顺序转换成网络字符顺序

头文件 `#include <arpa/inet.h>`

原型 `uint32_t htonl(uint32_t hostlong);`

说明 将 32 位整数由主机字节序（可能是小端）转换为网络字节序（大端），用于 IP 地址等字段。

返回值 返回网络字节序的 32 位值。

相关函数 `htons`、`ntohl`、`ntohs`

```
#include <arpa/inet.h>
#include <stdio.h>

int main(void)
{
    uint32_t v = 0x01020304;
    printf("0x%08x -> 0x%08x\n", v, htonl(v));
    return 0;
}
```

htons — 将 16 位主机字符顺序转换成网络字符顺序

头文件 `#include <arpa/inet.h>`

原型 `uint16_t htons(uint16_t hostshort);`

说明 将 16 位整数由主机字节序转换为网络字节序，主要用于端口号。

返回值 返回网络字节序的 16 位值。

相关函数 `htonl`、`ntohs`、`bind`

```
#include <arpa/inet.h>
#include <stdio.h>

int main(void)
{
    printf("%d -> %d\n", 8080, ntohs(htons(8080)));
    return 0;
}
```

inet_addr — 将网络地址转成网络二进制的数字

头文件 `#include <arpa/inet.h>`

原型 `in_addr_t inet_addr(const char *cp);`

说明 将点分十进制字符串（如 `"192.168.1.1"`）转换为网络字节序的 32 位地址。

返回值 成功返回网络字节序地址（可直接赋给 `sin_addr.s_addr`），失败返回 `INADDR_NONE`（即 0xffffffff），因此无法表示 255.255.255.255。

附加说明 建议使用可明确返回成功与否的 `inet_aton` 或 `inet_pton`。

相关函数 `inet_aton`、`inet_ntoa`、`inet_pton`

```
#include <arpa/inet.h>
#include <stdio.h>

int main(void)
{
    in_addr_t a = inet_addr("192.168.1.1");
    printf("0x%08x\n", (unsigned)ntohl(a));
    return 0;
}
```

inet_aton — 将网络地址转成网络二进制的数字

头文件 `#include <arpa/inet.h>`

原型 `int inet_aton(const char *cp, struct in_addr *inp);`

说明 将点分十进制字符串转换为网络字节序地址并存入 `inp`，可处理 255.255.255.255 等特殊地址。

返回值 成功返回非 0，失败返回 0。

相关函数 `inet_addr`、`inet_ntoa`

```
#include <arpa/inet.h>
#include <stdio.h>

int main(void)
{
    struct in_addr a;
    if (inet_aton("127.0.0.1", &a))
        printf("0x%08x\n", (unsigned)ntohl(a.s_addr));
    return 0;
}
```

`inet_ntoa` — 将网络二进制的数字转换成网络地址

头文件	`#include <arpa/inet.h>`
原型	`char *inet_ntoa(struct in_addr in);`
说明	将网络字节序的二进制地址转换为点分十进制字符串。
返回值	返回字符串指针，内容为静态内存，下次调用会被覆盖。
附加说明	多线程环境建议使用 `inet_ntop`。
相关函数	`inet_aton`、`inet_addr`

```
#include <arpa/inet.h>
#include <stdio.h>

int main(void)
{
    struct in_addr a;
    inet_aton("10.0.0.1", &a);
    printf("%s\n", inet_ntoa(a));
    return 0;
}
```

`listen` — 等待连接

头文件

原型 `int listen(int sockfd, int backlog);`

说明 使 `sockfd` 进入监听状态，准备接受连接请求；`backlog` 为等待队列的最大长度（超出后新的连接可能被拒绝）。

返回值 成功返回 0，失败返回 -1 并设置 `errno`。

附加说明 仅用于面向连接的 socket（如 `SOCK_STREAM`）；调用顺序为 `socket` → `bind` → `listen` → `accept`。

相关函数 `socket`、`bind`、`accept`

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdio.h>

int main(void)
{
    int sfd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in a = {0};
    a.sin_family = AF_INET;
    a.sin_port = htons(9000);
    a.sin_addr.s_addr = INADDR_ANY;
    bind(sfd, (struct sockaddr *)&a, sizeof(a));
    listen(sfd, 10);
    printf("listening on 9000\n");
    close(sfd);
    return 0;
}
```

ntohl — 将 32 位网络字符顺序转换成主机字符顺序

头文件 `#include <arpa/inet.h>`

原型 `uint32_t ntohl(uint32_t netlong);`

说明 将 32 位整数由网络字节序转换为主机字节序。

返回值 返回主机字节序的 32 位值。

相关函数 `ntohs`、`htonl`、`htons`

```
#include <arpa/inet.h>
#include <stdio.h>

int main(void)
{
    struct in_addr a;
    inet_aton("1.2.3.4", &a);
    printf("%s\n", inet_ntoa(a));
    printf("host order = 0x%08x\n", (unsigned)ntohl(a.s_addr));
    return 0;
}
```

ntohs — 将 16 位网络字符顺序转换成主机字符顺序

头文件 `#include <arpa/inet.h>`

原型 `uint16_t ntohs(uint16_t netshort);`

说明 将 16 位整数由网络字节序转换为主机字节序，主要用于端口号。

返回值 返回主机字节序的 16 位值。

相关函数 `ntohl`、`htons`、`getservbyport`

```
#include <arpa/inet.h>
#include <stdio.h>

int main(void)
{
    printf("%d\n", ntohs(htons(80)));   /* 80 */
    return 0;
}
```

recv — 经 socket 接收数据

头文件

原型 `ssize_t recv(int sockfd, void *buf, size_t len, int flags);`

说明 从已连接的 socket 接收最多 `len` 字节数据到 `buf`。`flags` 可为 0 或 `MSG_PEEK`（窥视数据但不取走）、`MSG_OOB`（带外数据）、`MSG_WAITALL`（等满 `len` 才返回）、`MSG_DONTWAIT`（非阻塞）。

返回值 成功返回实际接收的字节数；对方已关闭连接返回 0；失败返回 -1 并设置 `errno`。

附加说明 TCP 是字节流，可能分多次到达，需循环接收直到收完预期长度。

相关函数 `recvfrom`、`send`、`read`

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
#include <sys/types.h>
#include <sys/socket.h>
#include <stdio.h>

void echo(int fd)
{
    char buf[1024];
    ssize_t n = recv(fd, buf, sizeof(buf), 0);
    if (n > 0)
        send(fd, buf, n, 0);
    else if (n == 0)
        printf("peer closed\n");
}
```

`recvfrom` — 经 socket 接收数据

头文件

原型 `ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags, struct sockaddr *src_addr, socklen_t *addrlen);`

说明 接收数据并取得发送方的地址，主要用于无连接的 UDP socket；`src_addr`、`addrlen` 可为 NULL。

返回值 成功返回接收的字节数，失败返回 -1 并设置 `errno`。

相关函数 `sendto`、`recv`、`recvmsg`

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <stdio.h>

int main(void)
{
    int fd = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in a = {0};
    a.sin_family = AF_INET;
    a.sin_port = htons(5000);
    a.sin_addr.s_addr = INADDR_ANY;
    bind(fd, (struct sockaddr *)&a, sizeof(a));
    char buf[128];
    struct sockaddr_in from;
    socklen_t len = sizeof(from);
    ssize_t n = recvfrom(fd, buf, sizeof(buf), 0,
                         (struct sockaddr *)&from, &len);
    printf("from %s: %.*s\n", inet_ntoa(from.sin_addr), (int)n, buf);
    close(fd);
    return 0;
}
```

`recvmsg` — 经 socket 接收数据

头文件

原型 `ssize_t recvmsg(int sockfd, struct msghdr *msg, int flags);`

说明 以 `struct msghdr` 结构接收数据，可一次接收多个缓冲区（散布读）、取得辅助数据（如 `SCM_RIGHTS` 传递文件描述词）与来源地址，功能比 `recvfrom` 更强。

返回值 成功返回接收的字节数，失败返回 -1 并设置 `errno`。

相关函数 `sendmsg`、`recvfrom`、`recv`

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
#include <sys/types.h>
#include <sys/socket.h>
#include <stdio.h>
#include <string.h>

int main(void)
{
    int fd = socket(AF_INET, SOCK_DGRAM, 0);
    char buf[128];
    struct sockaddr_in from;
    struct iovec iov = { buf, sizeof(buf) };
    struct msghdr msg;
    memset(&msg, 0, sizeof(msg));
    msg.msg_name = &from;
    msg.msg_namelen = sizeof(from);
    msg.msg_iov = &iov;
    msg.msg_iovlen = 1;
    ssize_t n = recvmsg(fd, &msg, 0);
    printf("recv %zd bytes\n", n);
    close(fd);
    return 0;
}
```

send — 经 socket 传送数据

头文件

原型 `ssize_t send(int sockfd, const void *buf, size_t len, int flags);`

说明 向已连接的 socket 发送 `len` 字节数据。`flags` 可为 0、`MSG_OOB`（带外数据）、`MSG_DONTWAIT`、`MSG_NOSIGNAL`（出错时不产生 `SIGPIPE`）。

返回值 成功返回实际发送的字节数（可能小于 `len`），失败返回 -1 并设置 `errno`。

附加说明 应循环发送直至全部写完；向已关闭的连接写入会产生 `SIGPIPE`。

相关函数 `sendto`、`recv`、`write`

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
#include <sys/types.h>
#include <sys/socket.h>
#include <string.h>
#include <stdio.h>

int main(void)
{
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    const char *msg = "hello server\n";
    ssize_t n = send(fd, msg, strlen(msg), MSG_NOSIGNAL);
    printf("sent %zd bytes\n", n);
    close(fd);
    return 0;
}
```

`sendmsg` — 经 socket 传送数据

头文件

原型 `ssize_t sendmsg(int sockfd, const struct msghdr *msg, int flags);`

说明 以 `struct msghdr` 结构发送数据，可一次发送多个缓冲区（聚集写）并附带辅助数据或目的地址。

返回值 成功返回发送的字节数，失败返回 -1 并设置 `errno`。

相关函数 `recvmsg`、`sendto`、`send`

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
#include <sys/types.h>
#include <sys/socket.h>
#include <string.h>
#include <stdio.h>

int main(void)
{
    int fd = socket(AF_INET, SOCK_DGRAM, 0);
    char *s = "hello";
    struct iovec iov = { s, strlen(s) };
    struct msghdr msg;
    memset(&msg, 0, sizeof(msg));
    msg.msg_iov = &iov;
    msg.msg_iovlen = 1;
    sendmsg(fd, &msg, 0);
    close(fd);
    return 0;
}
```

sendto — 经 socket 传送数据

头文件

原型 `ssize_t sendto(int sockfd, const void *buf, size_t len, int flags, const struct sockaddr *dest_addr, socklen_t addrlen);`

说明 向 `dest_addr` 指定的地址发送数据，主要用于无连接的 UDP socket；面向连接的 socket 中 `dest_addr` 可为 NULL。

返回值 成功返回发送的字节数，失败返回 -1 并设置 `errno`。

相关函数 `recvfrom`、`send`、`sendmsg`

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>
#include <stdio.h>

int main(void)
{
    int fd = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in to = {0};
    to.sin_family = AF_INET;
    to.sin_port = htons(5000);
    to.sin_addr.s_addr = inet_addr("127.0.0.1");
    const char *msg = "hello udp";
    sendto(fd, msg, strlen(msg), 0, (struct sockaddr *)&to, sizeof(to));
    close(fd);
    return 0;
}
```

setprotoent — 打开网络协议的数据文件

头文件 `#include <netdb.h>`

原型 `void setprotoent(int stayopen);`

说明 打开协议数据文件（`/etc/protocols`）并把读写位置设到开头；`stayopen` 非 0 表示读取期间保持文件打开。

返回值 无返回值。

相关函数 `getprotoent`、`endprotoent`

```
#include <netdb.h>
#include <stdio.h>

int main(void)
{
    setprotoent(1);
    struct protoent *p = getprotoent();
    printf("%s\n", p->p_name);
    endprotoent();
    return 0;
}
```

setservent — 打开主机网络服务的数据文件

头文件 `#include <netdb.h>`

原型 `void setservent(int stayopen);`

说明 打开服务数据文件（`/etc/services`）并把读写位置设到开头；`stayopen` 非 0 表示保持文件打开。

返回值 无返回值。

相关函数 `getservent`、`endservent`

```
#include <netdb.h>
#include <arpa/inet.h>
#include <stdio.h>

int main(void)
{
    setservent(0);
    struct servent *s = getservent();
    printf("%s %d\n", s->s_name, ntohs(s->s_port));
    endservent();
    return 0;
}
```

`setsockopt` — 设置 socket 状态

头文件

原型 `int setsockopt(int sockfd, int level, int optname, const void *optval, socklen_t optlen);`

说明 设置 `sockfd` 的选项。常用组合：`SOL_SOCKET` / `SO_REUSEADDR`（允许重用本地地址，服务器重启必备）、`SO_RCVBUF`、`SO_SNDBUF`、`SO_KEEPALIVE`、`SO_BROADCAST`、`SO_RCVTIMEO`（接收超时）、`IPPROTO_TCP` / `TCP_NODELAY`（禁用 Nagle 算法）、`IPPROTO_IP` / `IP_ADD_MEMBERSHIP`（加入组播）。

返回值 成功返回 0，失败返回 -1 并设置 `errno`。

相关函数 `getsockopt`、`bind`

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
/* 允许地址重用 + 设置接收超时 */
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdio.h>

int main(void)
{
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    int on = 1;
    setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &on, sizeof(on));
    struct timeval tv = {3, 0};   /* 3 秒 */
    setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));
    close(fd);
    return 0;
}
```

`shutdown` — 终止 socket 通信

头文件

原型 `int shutdown(int sockfd, int how);`

说明 关闭 socket 的部分或全部连接。`how` 可为 `SHUT_RD`（关闭读取）、`SHUT_WR`（关闭写入，会发送 FIN）、`SHUT_RDWR`（双向关闭）。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`ENOTCONN`）。

附加说明 与 `close` 不同：`shutdown` 作用于连接本身（可用于通知对端），`close` 只减少描述词引用计数，仍须在最后调用 `close` 释放资源。

相关函数 `close`、`socket`、`connect`

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
#include <sys/types.h>
#include <sys/socket.h>
#include <stdio.h>

int main(void)
{
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    shutdown(fd, SHUT_WR);   /* 通知对端不再发送数据 */
    close(fd);
    return 0;
}
```

socket — 建立一个 socket 通信

头文件

原型 `int socket(int domain, int type, int protocol);`

说明 建立 socket 并返回描述词。`domain` 为通信域，常用 `AF_INET`（IPv4）、`AF_INET6`（IPv6）、`AF_UNIX`（本机进程间）；`type` 为类型，常用 `SOCK_STREAM`（TCP，面向连接）、`SOCK_DGRAM`（UDP，无连接）、`SOCK_RAW`（原始套接字）；`protocol` 通常为 0（自动选择）。

返回值 成功返回 socket 描述词，失败返回 -1 并设置 `errno`（`EAFNOSUPPORT`、`EPROTONOSUPPORT`）。

相关函数 `bind`、`connect`、`accept`、`close`

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
#include <sys/types.h>
#include <sys/socket.h>
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) {
        perror("socket");
        return 1;
    }
    printf("socket fd = %d\n", fd);
    close(fd);
    return 0;
}
```

第20章 进程通信（IPC）函数

本章为 System V 进程间通信函数，包含消息队列、信号量与共享内存，需包含 `<sys/ipc.h>`、`<sys/msg.h>`、`<sys/sem.h>`、`<sys/shm.h>`。可用 `ipcs`、`ipcrm` 命令查看或删除 IPC 对象。

`ftok` — 将文件路径和计划代号转为 System V IPC key

头文件

原型 `key_t ftok(const char *pathname, int proj_id);`

说明 依已存在的文件 `pathname` 与其 inode 编号，加上低 8 位有效的 `proj_id`，产生一个供 `msgget`、`semget`、`shmget` 使用的 key 值。

返回值 成功返回 key 值，失败返回 -1 并设置 `errno`（`ENOENT`）。

附加说明 同一文件与同一 `proj_id` 会产生相同 key；也可直接使用 `IPC_PRIVATE` 建立仅供亲缘进程使用的对象。

相关函数 `msgget`、`semget`、`shmget`

```
#include <sys/types.h>
#include <sys/ipc.h>
```

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <stdio.h>

int main(void)
{
    key_t key = ftok("/tmp", 'a');
    printf("key = 0x%08x\n", (unsigned)key);
    return 0;
}
```

`msgctl` — 控制信息队列的运作

头文件

原型 `int msgctl(int msqid, int cmd, struct msqid_ds *buf);`

说明 对消息队列 `msqid` 执行控制操作。`cmd` 可为 `IPC_STAT`（取得队列状态存入 `buf`）、`IPC_SET`（设置 `msg_perm` 等属性）、`IPC_RMID`（删除队列，`buf` 可为 NULL）。

返回值 成功返回 0（`IPC_STAT` / `IPC_RMID` 也可能返回非负状态值，依实现而定），失败返回 -1 并设置 `errno`（`EACCES`、`EINVAL`）。

附加说明 消息队列不会随进程结束自动删除，使用完毕应显式调用 `msgctl(msqid, IPC_RMID, NULL)`。

相关函数 `msgget`、`msgsnd`、`msgrcv`

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
```

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <stdio.h>

int main(void)
{
    key_t key = ftok("/tmp", 'm');
    int msqid = msgget(key, 0666 | IPC_CREAT);
    struct msqid_ds ds;
    msgctl(msqid, IPC_STAT, &ds);
    printf("messages in queue: %lu\n", (unsigned long)ds.msg_qnum);
    msgctl(msqid, IPC_RMID, NULL);      /* 删除队列 */
    return 0;
}
```

`msgget` — 建立信息队列

头文件

原型 `int msgget(key_t key, int msgflg);`

说明 建立或取得消息队列。`msgflg` 为权限值（如 `0666`）与 `IPC_CREAT`（不存在则建立）、`IPC_EXCL`（与 `IPC_CREAT` 合用，已存在则失败）的组合；`key` 也可为 `IPC_PRIVATE`。

返回值 成功返回消息队列识别码，失败返回 -1 并设置 `errno`（`EEXIST`、`ENOENT`、`ENOSPC`）。

相关函数 `msgctl`、`msgsnd`、`msgrcv`

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
```

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <stdio.h>

int main(void)
{
    int msqid = msgget(ftok("/tmp", 'm'), IPC_CREAT | 0666);
    if (msqid < 0) {
        perror("msgget");
        return 1;
    }
    printf("msqid = %d\n", msqid);
    return 0;
}
```

`msgrcv` — 从信息队列读取信息

头文件

原型 `ssize_t msgrcv(int msqid, void *msgp, size_t msgsz, long msgtyp, int msgflg);`

说明 从队列 `msqid` 读取消息到 `msgp`（结构第一个成员须为 `long mtype`，其后为数据区）。`msgsz` 为数据区最大长度；`msgtyp` 为 0 表示取第一条，大于 0 表示取指定类型，小于 0 表示取类型小于等于 `|msgtyp|` 的最小类型消息；`msgflg` 可用 `IPC_NOWAIT`（无消息立即返回）、`MSG_NOERROR`（超长则截断）。

返回值 成功返回实际读到的字节数，失败返回 -1 并设置 `errno`（`ENOMSG`、`E2BIG`）。

相关函数 `msgsnd`、`msgget`、`msgctl`

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
```

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <stdio.h>
#include <string.h>

struct msgbuf {
    long mtype;
    char mtext[128];
};

int main(void)
{
    int msqid = msgget(ftok("/tmp", 'm'), 0666 | IPC_CREAT);
    struct msgbuf mb;
    ssize_t n = msgrcv(msqid, &mb, sizeof(mb.mtext), 1, 0);
    printf("type=%ld text=%.*s\n", mb.mtype, (int)n, mb.mtext);
    return 0;
}
```

`msgsnd` — 将信息送入信息队列

头文件

原型 `int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg);`

说明 将 `msgp` 所指的消息（第一个成员为 `long mtype`，必须大于 0）送入队列 `msqid`；`msgsz` 为数据区长度；`msgflg` 可为 0 或 `IPC_NOWAIT`（队列满时不等待而返回）。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EAGAIN`、`EACCES`、`EIDRM`）。

相关函数 `msgrcv`、`msgget`

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
```

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <stdio.h>
#include <string.h>

struct msgbuf {
    long mtype;
    char mtext[128];
};

int main(void)
{
    int msqid = msgget(ftok("/tmp", 'm'), 0666 | IPC_CREAT);
    struct msgbuf mb;
    mb.mtype = 1;
    strcpy(mb.mtext, "hello message queue");
    msgsnd(msqid, &mb, strlen(mb.mtext), 0);
    return 0;
}
```

`semctl` — 控制信号队列的操作

头文件

原型 `int semctl(int semid, int semnum, int cmd, ...);`

说明 对信号量集 `semid` 执行控制操作。`cmd` 常用：`IPC_STAT`、`IPC_SET`、`IPC_RMID`（删除信号量集）、`GETVAL`（取得单个信号量的值）、`SETVAL`（设置值，需提供 `union semun`）、`GETALL`、`SETALL`、`GETPID`、`GETNCNT`、`GETZCNT`。

返回值 依 `cmd` 而不同；`GETVAL` 返回信号量值，其他多数成功返回 0，失败返回 -1 并设置 `errno`。

附加说明 第四个参数类型为 `union semun`，需由程序自行定义：`union semun { int val; struct semid_ds *buf; unsigned short *array; };`

相关函数 `semget`、`semop`

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
```

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <stdio.h>

union semun {
    int val;
    struct semid_ds *buf;
    unsigned short *array;
};

int main(void)
{
    int semid = semget(ftok("/tmp", 's'), 1, IPC_CREAT | 0666);
    union semun arg;
    arg.val = 1;                        /* 初值为 1，相当于互斥锁 */
    semctl(semid, 0, SETVAL, arg);
    printf("value = %d\n", semctl(semid, 0, GETVAL));
    semctl(semid, 0, IPC_RMID);
    return 0;
}
```

`semget` — 配置信号队列

头文件

原型 `int semget(key_t key, int nsems, int semflg);`

说明 建立或取得包含 `nsems` 个信号量的信号量集；`semflg` 为权限值与 `IPC_CREAT`、`IPC_EXCL` 的组合。

返回值 成功返回信号量集识别码，失败返回 -1 并设置 `errno`（`EEXIST`、`ENOENT`）。

附加说明 新建时信号量值未初始化，必须用 `semctl(SETVAL)` 或 `SETALL` 设置初值。

相关函数 `semctl`、`semop`、`ftok`

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
```

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <stdio.h>

int main(void)
{
    int semid = semget(ftok("/tmp", 's'), 1, IPC_CREAT | IPC_EXCL | 0666);
    if (semid < 0) {
        perror("semget");
        return 1;
    }
    printf("semid = %d\n", semid);
    return 0;
}
```

semop — 信号处理

头文件

原型 `int semop(int semid, struct sembuf *sops, size_t nsops);`

说明 对信号量集执行一组原子操作。`struct sembuf` 含 `sem_num`（信号量编号）、`sem_op`（正值释放资源、负值申请资源、0 表示等待变为 0）、`sem_flg`（`IPC_NOWAIT`、`SEM_UNDO`，后者在进程结束时自动撤销操作，防止死锁）。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EAGAIN`、`EINTR`）。

附加说明 多个操作会原子地一并执行，可避免竞态；建议使用 `SEM_UNDO`。

相关函数 `semget`、`semctl`

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
```

```
/* P 操作（申请）与 V 操作（释放） */
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <stdio.h>

int main(void)
{
    int semid = semget(ftok("/tmp", 's'), 1, 0666);
    struct sembuf p = {0, -1, SEM_UNDO};   /* 申请 */
    struct sembuf v = {0,  1, SEM_UNDO};   /* 释放 */
    semop(semid, &p, 1);
    printf("in critical section\n");
    semop(semid, &v, 1);
    return 0;
}
```

shmat — attach 共享内存

头文件

原型 `void *shmat(int shmid, const void *shmaddr, int shmflg);`

说明 将 `shmid` 指定的共享内存映射到调用进程的地址空间；`shmaddr` 通常为 NULL（由系统选择地址），`shmflg` 可为 `SHM_RDONLY`（只读）或 0。

返回值 成功返回映射后的地址，失败返回 `(void *)-1` 并设置 `errno`（`EACCES`、`EINVAL`）。

附加说明 映射后即可像普通内存一样读写，是最快的 IPC 方式，但需自行配合同步机制（信号量）避免竞态。

相关函数 `shmdt`、`shmget`、`shmctl`

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
```

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#include <string.h>

int main(void)
{
    int shmid = shmget(ftok("/tmp", 'h'), 4096, IPC_CREAT | 0666);
    char *p = shmat(shmid, NULL, 0);
    strcpy(p, "hello shared memory");
    printf("%s\n", p);
    shmdt(p);
    return 0;
}
```

`shmctl` — 控制共享内存的操作

头文件

原型 `int shmctl(int shmid, int cmd, struct shmid_ds *buf);`

说明 对共享内存 `shmid` 执行控制操作。`cmd` 可为 `IPC_STAT`（取得状态）、`IPC_SET`（设置权限等）、`IPC_RMID`（标记删除，所有进程 detach 后真正释放）、`SHM_LOCK` / `SHM_UNLOCK`（锁定/解锁，禁止换出）。

返回值 成功返回 0（`IPC_STAT`、`IPC_SET` 依实现而定），失败返回 -1 并设置 `errno`（`EACCES`、`EINVAL`）。

附加说明 共享内存不会随进程结束自动删除，务必在不再使用时调用 `IPC_RMID`。

相关函数 `shmget`、`shmat`、`shmdt`

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
```

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>

int main(void)
{
    int shmid = shmget(ftok("/tmp", 'h'), 4096, IPC_CREAT | 0666);
    struct shmid_ds ds;
    shmctl(shmid, IPC_STAT, &ds);
    printf("size = %lu\n", (unsigned long)ds.shm_segsz);
    shmctl(shmid, IPC_RMID, NULL);      /* 标记删除 */
    return 0;
}
```

`shmdt` — detach 共享内存

头文件

原型 `int shmdt(const void *shmaddr);`

说明 将 `shmaddr`（`shmat` 的返回值）所指的共享内存从当前进程的地址空间脱离；并不删除共享内存对象本身。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EINVAL`）。

相关函数 `shmat`、`shmctl`

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
```

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>

int main(void)
{
    int shmid = shmget(ftok("/tmp", 'h'), 4096, 0666);
    char *p = shmat(shmid, NULL, 0);
    printf("%s\n", p);
    shmdt(p);          /* 脱离映射，共享内存本身仍存在 */
    return 0;
}
```

shmget — 配置共享内存

头文件

原型 `int shmget(key_t key, size_t size, int shmflg);`

说明 建立或取得大小为 `size` 字节的共享内存；`shmflg` 为权限值与 `IPC_CREAT`、`IPC_EXCL` 的组合。

返回值 成功返回共享内存识别码，失败返回 -1 并设置 `errno`（`EEXIST`、`EINVAL`、`ENOSPC`）。

附加说明 `size` 通常取页大小的整数倍；新建时内容会被清零。

相关函数 `shmat`、`shmctl`、`ftok`

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
```

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>

int main(void)
{
    int shmid = shmget(IPC_PRIVATE, 4096, IPC_CREAT | 0666);
    printf("shmid = %d\n", shmid);
    return 0;
}
```

第21章 记录函数

本章函数用于将信息写入系统日志（通常为 `/var/log/messages` 或 `/var/log/syslog`），需包含 `<syslog.h>`。

`closelog` — 关闭信息记录

头文件 `#include <syslog.h>`

原型 `void closelog(void);`

说明 关闭由 `openlog` 打开的与系统日志服务（syslogd）之间的连接，并释放相关资源。通常不必调用，程序结束会自动关闭。

返回值 无返回值。

相关函数 `openlog`、`syslog`

```
#include <syslog.h>

int main(void)
{
    openlog("myapp", LOG_PID, LOG_USER);
    syslog(LOG_INFO, "started");
    closelog();
    return 0;
}
```

openlog — 准备做信息记录

头文件 `#include <syslog.h>`

原型 `void openlog(const char *ident, int option, int facility);`

说明 打开与系统日志服务的连接并设置默认行为。`ident` 为每条消息前置的程序名；`option` 可为 `LOG_PID`（含进程号）、`LOG_CONS`（出错时输出到控制台）、`LOG_NDELAY`（立即打开连接）、`LOG_PERROR`（同时输出到 stderr）；`facility` 可为 `LOG_USER`、`LOG_DAEMON`、`LOG_LOCAL0`～`LOG_LOCAL7` 等。

返回值 无返回值。

相关函数 `syslog`、`closelog`

```
#include <syslog.h>

int main(void)
{
    openlog("mydaemon", LOG_PID | LOG_CONS, LOG_DAEMON);
    syslog(LOG_INFO, "daemon started");
    closelog();
    return 0;
}
```

syslog — 将信息记录至系统日志文件

头文件 `#include <syslog.h>`

原型 `void syslog(int priority, const char *format, ...);`

说明 将格式化后的信息写入系统日志。`priority` 由「设施」与「等级」组合而成，等级由高到低为 `LOG_EMERG`、`LOG_ALERT`、`LOG_CRIT`、`LOG_ERR`、`LOG_WARNING`、`LOG_NOTICE`、`LOG_INFO`、`LOG_DEBUG`（默认只记录到 `LOG_INFO` 以上）。

返回值 无返回值。

附加说明 适合守护进程记录运行状态；也可直接用 `vsyslog` 配合可变参数。

相关函数 `openlog`、`closelog`

```
#include <syslog.h>

int main(void)
{
    openlog("myapp", LOG_PID, LOG_USER);
    syslog(LOG_ERR, "open %s failed: %m", "/tmp/a.txt");  /* %m 表示 errno */
    syslog(LOG_INFO, "user %s logged in", "root");
    closelog();
    return 0;
}
```

第22章 环境变量函数

本章函数用于取得、设置与清除环境变量。

getenv — 取得环境变量内容

头文件 `#include <stdlib.h>`

原型 `char *getenv(const char *name);`

说明 依参数 `name` 在环境变量表中查找对应的内容。

返回值 成功返回指向该环境变量内容的字符串指针，未找到返回 NULL。

附加说明 返回的是 environ 中的指针，不应修改其内容；需要长期保存时应自行复制。

相关函数 `putenv`、`setenv`、`unsetenv`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    char *p = getenv("PATH");
    printf("PATH = %s\n", p ? p : "(not set)");
    return 0;
}
```

putenv — 改变或增加环境变量

头文件 `#include <stdlib.h>`

原型 `int putenv(char *string);`

说明 以 `string`（格式须为 `"name=value"`）改变或新增环境变量；若 `string` 不含 `=`，则删除该环境变量。

返回值 成功返回 0，失败返回非 0 并设置 `errno`（`ENOMEM`）。

附加说明 `string` 会成为环境的一部分，因此不能是局部（栈上）变量，也不应在之后修改；建议改用 `setenv`。

相关函数 `getenv`、`setenv`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    static char env[] = "MYVAR=hello";   /* 必须是长期有效的内存 */
    putenv(env);
    printf("MYVAR = %s\n", getenv("MYVAR"));
    return 0;
}
```

setenv — 改变或增加环境变量

头文件 `#include <stdlib.h>`

原型 `int setenv(const char *name, const char *value, int overwrite);`

说明 将环境变量 `name` 的值设为 `value`；`overwrite` 为 0 且该变量已存在时保持原值，非 0 则覆盖。

返回值 成功返回 0，失败返回 -1 并设置 `errno`（`EINVAL`、`ENOMEM`）。

附加说明 会自动复制字符串，比 `putenv` 安全，建议使用。

相关函数 `getenv`、`unsetenv`、`putenv`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    setenv("MYVAR", "hello", 1);
    printf("%s\n", getenv("MYVAR"));
    setenv("MYVAR", "world", 0);      /* 不覆盖，仍为 hello */
    printf("%s\n", getenv("MYVAR"));
    return 0;
}
```

unsetenv — 清除环境变量内容

头文件	`#include <stdlib.h>`
原型	`int unsetenv(const char *name);`
说明	删除名为 `name` 的环境变量；`name` 中不应含有 `=`。
返回值	成功返回 0，失败返回 -1 并设置 `errno`。
相关函数	`setenv`、`getenv`

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    setenv("MYVAR", "hello", 1);
    unsetenv("MYVAR");
    printf("MYVAR = %s\n", getenv("MYVAR") ? getenv("MYVAR") : "(removed)");
    return 0;
}
```

第23章 正则表达式

本章函数用于正则表达式的编译、匹配与错误处理，需包含 `<sys/types.h>` 与 `<regex.h>`。

regcomp — 编译正则表达式字符串

头文件

原型 `int regcomp(regex_t *preg, const char *pattern, int cflags);`

说明 将 `pattern` 编译为可供 `regexec` 使用的正则表达式，结果存入 `preg`。`cflags` 可为 0 或 `REG_EXTENDED`（使用扩展正则语法）、`REG_ICASE`（忽略大小写）、`REG_NOSUB`（不保存子表达式位置）、`REG_NEWLINE`（特殊处理换行符）。

返回值 成功返回 0，失败返回非 0 的错误码（可用 `regerror` 取得说明）。

附加说明 使用完毕必须用 `regfree` 释放。

相关函数 `regexec`、`regerror`、`regfree`

```
#include <sys/types.h>
#include <regex.h>
```

```
#include <sys/types.h>
#include <regex.h>
#include <stdio.h>

int main(void)
{
    regex_t re;
    int err = regcomp(&re, "^[a-z]+\\.[0-9]+$", REG_EXTENDED);
    if (err) {
        char buf[128];
        regerror(err, &re, buf, sizeof(buf));
        fprintf(stderr, "regcomp: %s\n", buf);
        return 1;
    }
    printf("matched: %d\n", regexec(&re, "abc.123", 0, NULL, 0) == 0);
    regfree(&re);
    return 0;
}
```

regerror — 取得正则搜索的错误原因

头文件

原型 `size_t regerror(int errcode, const regex_t *preg, char *errbuf, size_t errbuf_size);`

说明 将 `errcode`（由 `regcomp` 或 `regexec` 返回）转换为可读的错误描述字符串，并写入 `errbuf`。

返回值 返回存放完整错误描述所需的字节数；若大于 `errbuf_size`，则描述会被截断。

相关函数 `regcomp`、`regexec`

```
#include <sys/types.h>
#include <regex.h>
```

```
#include <sys/types.h>
#include <regex.h>
#include <stdio.h>

int main(void)
{
    regex_t re;
    int err = regcomp(&re, "a(", REG_EXTENDED);   /* 语法错误 */
    if (err) {
        char buf[128];
        regerror(err, &re, buf, sizeof(buf));
        printf("error: %s\n", buf);
    }
    return 0;
}
```

regexec — 进行正则表达式的搜索

头文件

原型 `int regexec(const regex_t *preg, const char *string, size_t nmatch, regmatch_t pmatch[], int eflags);`

说明 用编译好的正则 `preg` 匹配 `string`。`nmatch` 与 `pmatch` 用于取得整体及子表达式的匹配位置（`regmatch_t` 含 `rm_so` 起始、`rm_eo` 结束偏移）；`eflags` 可为 0、`REG_NOTBOL`、`REG_NOTEOL`。

返回值 匹配成功返回 0，失败返回 `REG_NOMATCH` 或其他错误码。

附加说明 `nmatch` 为 0 或 `REG_NOSUB` 时不保存匹配位置；可用 `pmatch[i].rm_so == -1` 判断某子表达式是否未参与匹配。

相关函数 `regcomp`、`regerror`、`regfree`

```
#include <sys/types.h>
#include <regex.h>
```

```
/* 取出匹配的子表达式 */
#include <sys/types.h>
#include <regex.h>
#include <stdio.h>

int main(void)
{
    regex_t re;
    regmatch_t m[3];
    regcomp(&re, "([0-9]+)-([0-9]+)", REG_EXTENDED);
    if (regexec(&re, "2026-10-08", 3, m, 0) == 0) {
        printf("full: %.*s\n", (int)(m[0].rm_eo - m[0].rm_so), "2026-10-08" + m[0].rm_so);
        printf("year: %.*s\n", (int)(m[1].rm_eo - m[1].rm_so), "2026-10-08" + m[1].rm_so);
        printf("month: %.*s\n", (int)(m[2].rm_eo - m[2].rm_so), "2026-10-08" +
m[2].rm_so);
    }
    regfree(&re);
    return 0;
}
```

`regfree` — 释放正则表达式使用的内存

头文件

原型 `void regfree(regex_t *preg);`

说明 释放由 `regcomp` 编译的正则表达式所占用的内存；释放后的 `preg` 不能再用于 `regexec`。

返回值 无返回值。

附加说明 每个 `regcomp` 都应对应一次 `regfree`，否则会造成内存泄漏。

相关函数 `regcomp`、`regexec`

```
#include <sys/types.h>
#include <regex.h>
```

```
#include <sys/types.h>
#include <regex.h>
#include <stdio.h>

int main(void)
{
    regex_t re;
    regcomp(&re, "^test", REG_EXTENDED);
    printf("%d\n", regexec(&re, "testing", 0, NULL, 0));
    regfree(&re);      /* 释放 */
    return 0;
}
```

第24章 动态函数

本章函数用于在运行时动态加载共享库（`.so`）并调用其中的函数，需包含 `<dlfcn.h>`，编译时加 `-ldl`。

dlclose — 关闭动态函数库文件

头文件 `#include <dlfcn.h>`

原型 `int dlclose(void *handle);`

说明 关闭由 `dlopen` 打开的共享库句柄 `handle`；系统会减少该库的引用计数，计数归零时真正卸载。

返回值 成功返回 0，失败返回非 0（可用 `dlerror` 取得原因）。

相关函数 `dlopen`、`dlsym`、`dlerror`

```
#include <dlfcn.h>
#include <stdio.h>

int main(void)
{
    void *h = dlopen("libm.so.6", RTLD_LAZY);
    dlclose(h);
    return 0;
}
```

dlerror — 动态函数错误处理

头文件 `#include <dlfcn.h>`

原型 `char *dlerror(void);`

说明 返回最近一次 `dlopen`、`dlsym`、`dlclose` 操作所产生的错误描述字符串。

返回值 有错误则返回描述字符串，自上次调用以来无错误则返回 NULL。

附加说明 每次调用后会清除错误状态，因此出错后应立即调用以取得信息。

相关函数 `dlopen`、`dlsym`、`dlclose`

```
#include <dlfcn.h>
#include <stdio.h>

int main(void)
{
    void *h = dlopen("/nonexistent/lib.so", RTLD_LAZY);
    if (h == NULL)
        printf("error: %s\n", dlerror());
    return 0;
}
```

`dlopen` — 打开动态函数库文件

头文件 `#include <dlfcn.h>`

原型 `void *dlopen(const char *filename, int flags);`

说明 加载 `filename` 指定的共享库（可为相对文件名，依 `LD_LIBRARY_PATH` 等规则搜索；为 NULL 时返回主程序的句柄），返回句柄。`flags` 可为 `RTLD_LAZY`（用到时才解析符号）、`RTLD_NOW`（立即解析所有符号）、`RTLD_GLOBAL`（符号可被后续库使用）、`RTLD_LOCAL`（默认，符号仅本库可见）。

返回值 成功返回句柄，失败返回 NULL（用 `dlerror` 取得原因）。

相关函数 `dlsym`、`dlclose`、`dlerror`

```
#include <dlfcn.h>
#include <stdio.h>

int main(void)
{
    void *h = dlopen("libm.so.6", RTLD_LAZY);
    if (h == NULL) {
        fprintf(stderr, "%s\n", dlerror());
        return 1;
    }
    printf("loaded\n");
    dlclose(h);
    return 0;
}
```

`dlsym` — 从共享对象中搜索动态函数

头文件 `#include <dlfcn.h>`

原型 `void *dlsym(void *handle, const char *symbol);`

说明 在 `handle` 所指的共享库中查找符号（函数或变量）`symbol` 并返回其地址；`handle` 可为 `RTLD_DEFAULT`（按默认顺序搜索）或 `RTLD_NEXT`。

返回值 成功返回符号地址，失败返回 NULL（用 `dlerror` 区分“符号值为 NULL”与“未找到”）。

附加说明 取得的函数指针需自行转换为正确的函数类型；C++ 符号存在名称修饰（name mangling），应使用 `extern "C"` 导出。

相关函数 `dlopen`、`dlclose`

```
/* 动态调用 libm 中的 sqrt */
#include <dlfcn.h>
#include <stdio.h>

int main(void)
{
    void *h = dlopen("libm.so.6", RTLD_LAZY);
    double (*my_sqrt)(double) = dlsym(h, "sqrt");
    printf("sqrt(16) = %f\n", my_sqrt(16.0));
    dlclose(h);
    return 0;
}
```

第25章 其他函数

getopt — 分析命令行参数

头文件 `#include <unistd.h>`

原型 `int getopt(int argc, char *const argv[], const char *optstring);`

说明 解析命令行选项。全局变量 `optarg` 指向选项参数，`optind` 为下一个待处理参数的下标，`optopt` 存放无法识别的选项，`opterr` 控制是否输出错误信息。`optstring` 中字符后带 `:` 表示该选项需要参数，带 `::` 表示参数可选；以 `+` 或 `-` 开头可改变扫描行为。

返回值 返回下一个选项字符；解析完毕返回 -1；遇到非法选项返回 `'?'`；缺少参数返回 `':'` 或 `'?'`。

附加说明 只处理短选项（单字符）；需要长选项（如 `--help`）可使用 `getopt_long`。

相关函数 `getopt_long`、`getenv`

```
/* 支持 -h 与 -o file */
#include <unistd.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    int opt;
    while ((opt = getopt(argc, argv, "ho:")) != -1) {
        switch (opt) {
        case 'h':
            printf("usage: %s [-h] [-o file]\n", argv[0]);
            break;
        case 'o':
            printf("output file: %s\n", optarg);
            break;
        default:
            return 1;
        }
    }
    printf("remaining args start at %d\n", optind);
    return 0;
}
```

isatty — 判断文件描述词是否是为终端机

头文件 `#include <unistd.h>`

原型 `int isatty(int fd);`

说明 判断 `fd` 所指的文件描述词是否连接到终端（tty）设备。

返回值 是终端返回 1，否则返回 0 并设置 `errno` 为 `ENOTTY`。

附加说明 常用于判断是否以交互方式运行，以便决定是否输出彩色提示或进度条。

相关函数 `ttyname`、`fileno`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    if (isatty(STDOUT_FILENO))
        printf("stdout is a terminal\n");
    else
        printf("stdout is redirected\n");
    return 0;
}
```

select — I/O 多工机制

头文件

原型 `int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout);`

说明 监视多个文件描述词，等待其中任何一个变为可读、可写或出现异常。`nfds` 为监视的最大描述词加 1；`timeout` 为 NULL 表示无限等待，为 0 表示立即返回（轮询）。需用 `FD_ZERO`、`FD_SET`、`FD_CLR`、`FD_ISSET` 宏操作描述词集合。

返回值 返回就绪的描述词总数；超时返回 0；失败返回 -1 并设置 `errno`（`EINTR` 表示被信号中断）。

附加说明 返回后集合会被修改为仅含就绪的描述词，因此每次调用前必须重新设置集合；`FD_SETSIZE`（通常 1024）限制了监视数量，高并发场景可用 `poll` 或 `epoll`。

相关函数 `poll`、`epoll_wait`、`fcntl`

```
#include <sys/select.h>
#include <sys/time.h>
```

```
/* 监视标准输入，最多等待 5 秒 */
#include <sys/select.h>
#include <sys/time.h>
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    fd_set rfds;
    struct timeval tv = {5, 0};
    FD_ZERO(&rfds);
    FD_SET(STDIN_FILENO, &rfds);
    int n = select(STDIN_FILENO + 1, &rfds, NULL, NULL, &tv);
    if (n == 0)
        printf("timeout, no input\n");
    else if (FD_ISSET(STDIN_FILENO, &rfds))
        printf("stdin is ready\n");
    return 0;
}
```

`ttyname` — 返回一终端机名称

头文件	`#include <unistd.h>`
原型	`char *ttyname(int fd);`
说明	返回 `fd` 所连接的终端设备名称（如 `/dev/pts/0`）。
返回值	成功返回终端名称字符串指针，若 `fd` 不是终端则返回 NULL 并设置 `errno` 为 `ENOTTY`。
附加说明	返回静态内存；多线程环境建议使用 `ttyname_r`。
相关函数	`isatty`、`ctermid`

```
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    char *name = ttyname(STDIN_FILENO);
    printf("terminal: %s\n", name ? name : "(not a tty)");
    return 0;
}
```

附录

附录A 编译程序 gcc

Linux 下 C 程序通常使用 gcc（GNU Compiler Collection）编译。常用格式：

```
gcc [选项] 源文件 [-o 可执行文件]
```

常用选项：

选项	说明
`-o file`	指定输出的可执行文件名，默认为 `a.out`
`-c`	只编译生成目标文件（`.o`），不链接
`-Wall`	打开常用的警告信息，建议始终开启
`-Wextra`	打开额外的警告信息
`-O0 / -O1 / -O2 / -O3`	指定优化等级，`-O2` 最常用
`-g`	生成调试信息，供 gdb 使用
`-I dir`	指定额外的头文件搜索目录
`-L dir`	指定额外的库文件搜索目录
`-l name`	链接名为 `libname.so` 或 `libname.a` 的库
`-static`	使用静态链接
`-E`	只进行预处理，输出预处理后的代码
`-S`	只编译到汇编，生成 `.s` 文件
`-std=c99 / -std=c11 / -std=gnu11`	指定语言标准
`-D macro`	在命令行定义宏，如 `-DNDEBUG`

链接库的常见情形：

```
gcc test.c -o test -lm          # 使用数学函数（sin、sqrt、pow 等）
gcc test.c -o test -lpthread    # 使用 POSIX 线程
gcc test.c -o test -ldl         # 使用动态加载函数（dlopen、dlsym 等）
gcc test.c -o test -lcrypt      # 使用 crypt
```

编译过程分为四个阶段：预处理（`-E`）、编译（`-S`）、汇编（`-c`）与链接。日常开发可一条命令完成：

```
gcc -Wall -O2 -o demo demo.c
./demo
```

附录B 宏与函数

本书许多“函数”实际以宏（macro）实现，例如 `<ctype.h>` 中的 `isalnum`、`isalpha`、`toupper`，以及 `getc`、`putc` 等。宏与真正的函数有以下差别：

1. **无类型检查**：宏在预处理阶段展开，参数类型不受检查，容易因书写错误导致隐蔽问题。

2. **副作用风险**：例如 `getc(fp++)` 这类写法可能使 `fp++` 被求值多次，行为未定义。

3. **不是地址**：宏没有函数地址，不能作为回调函数传给其他函数。

4. **多次求值**：形如 `#define MAX(a,b) ((a)>(b)?(a):(b))` 的宏会重复求值参数，`MAX(i++, j++)` 结果错误。

应对方式：

- 需要函数版本时，可在函数名外加括号，如 `(isalpha)(c)`，或在包含头文件前定义相应宏的禁用开关；
- 标准库给出的可替代函数往往更安全，例如用 `fgetc` 取代 `getc`，用 `snprintf` 取代 `sprintf`；
- 自定宏时应为每个参数与整个表达式加括号，并避免在参数中使用自增、自减运算。

常见的宏与函数对照：

宏（可能为宏实现）	建议使用的函数
`getc`、`putc`	`fgetc`、`fputc`
`isalpha`、`isdigit` 等	需要函数地址时使用 `(isalpha)`
`assert`	发布版本定义 `NDEBUG` 关闭
`va_arg` 系列	—（只能以宏实现）

附录C 不定参数

编写参数个数可变的函数需包含 `<stdarg.h>`，并配合 `va_list`、`va_start`、`va_arg`、`va_end` 使用。基本形式：

```
#include <stdarg.h>
#include <stdio.h>

double average(int count, ...)   /* 最后一个具名参数不可省略 */
{
    va_list ap;
    double sum = 0;
    va_start(ap, count);              /* 以最后一个具名参数初始化 */
    for (int i = 0; i < count; i++)
        sum += va_arg(ap, double);    /* 按类型依次取出 */
    va_end(ap);
    return sum / count;
}

int main(void)
{
    printf("%.2f\n", average(3, 1.0, 2.0, 3.0));   /* 2.00 */
    return 0;
}
```

使用要点：

1. 函数必须至少有一个具名参数，`va_start` 的第二个参数应是最后一个具名参数。

2. `va_arg` 必须按调用时实参的实际类型取出，`float` 会提升为 `double`，`char`、`short` 会提升为 `int`，类型不符会导致未定义行为。

3. 调用者与实现者必须通过某种方式约定参数个数或结束标记（如 `printf` 用格式串、exec 系列用 NULL 结尾）。

4. 每个 `va_start` 必须对应一个 `va_end`。

5. 需要把不定参数传给 `printf` 一类函数时，应使用 `vprintf`、`vfprintf`、`vsnprintf` 等以 `va_list` 为参数的函数。

此外，`__VA_ARGS__` 可用于定义接受不定参数的宏：

```
#define LOG(fmt, ...) fprintf(stderr, "[%s] " fmt "\n", __func__, __VA_ARGS__)
```

附录D Linux 常用头文件一览

头文件	主要内容
`<stdio.h>`	标准输入输出：`printf`、`fopen`、`fread` 等
`<stdlib.h>`	内存配置、转换、随机数、环境变量、进程控制
`<string.h>`	字符串与内存操作：`strcpy`、`memcpy` 等
`<ctype.h>`	字符测试与转换：`isalpha`、`tolower` 等
`<math.h>`	数学函数，编译时需加 `-lm`
`<time.h>`	时间与日期：`time`、`strftime` 等
`<unistd.h>`	系统调用：`fork`、`read`、`write`、`getpid` 等
`<fcntl.h>`	文件控制：`open`、`fcntl`、`flock`
`<sys/types.h>`	基本系统数据类型：`pid_t`、`size_t`、`mode_t` 等
`<sys/stat.h>`	文件状态：`stat`、`chmod`、`mkdir`、文件类型宏
`<sys/wait.h>`	进程等待：`wait`、`waitpid`
`<signal.h>`	信号处理：`signal`、`sigaction`、`kill`
`<setjmp.h>`	非局部跳转：`setjmp`、`longjmp`
`<dirent.h>`	目录操作：`opendir`、`readdir`、`scandir`
`<pwd.h>`、`<grp.h>`	用户与组信息：`getpwnam`、`getgrgid` 等
`<utmp.h>`	登录记录：`getutent`、`pututline`
`<sys/socket.h>`	socket 接口：`socket`、`bind`、`send`、`recv`
`<netinet/in.h>`	Internet 地址结构：`struct sockaddr_in`

头文件	主要内容
`<arpa/inet.h>`	地址转换：`inet_addr`、`inet_ntoa`、`htons`
`<netdb.h>`	名称解析：`gethostbyname`、`getservbyname`
`<sys/ipc.h>`、`<sys/msg.h>`、`<sys/sem.h>`、`<sys/shm.h>`	System V IPC
`<sys/select.h>`	I/O 多工：`select`
`<regex.h>`	正则表达式
`<dlfcn.h>`	动态加载共享库：`dlopen`、`dlsym`
`<errno.h>`	错误码定义：`errno`、`ENOENT` 等
`<syslog.h>`	系统日志：`openlog`、`syslog`

附录E 常见错误代码及原因

以下为 `<errno.h>` 中常见的错误代码及其含义，可用 `perror` 或 `strerror` 输出对应说明。

errno	值（Linux/x86）	含义
`EPERM`	1	操作不允许（权限不足）
`ENOENT`	2	文件或目录不存在
`ESRCH`	3	指定的进程不存在
`EINTR`	4	系统调用被信号中断
`EIO`	5	输入输出错误
`EBADF`	9	文件描述词无效
`ECHILD`	10	没有子进程可等待
`EAGAIN`	11	资源暂时不可用，稍后重试（非阻塞操作）
`ENOMEM`	12	内存不足
`EACCES`	13	权限被拒绝
`EFAULT`	14	地址无效（越界访问）
`EEXIST`	17	文件已存在
`ENODEV`	19	设备不存在
`ENOTDIR`	20	路径中某项不是目录
`EISDIR`	21	目标是一个目录
`EINVAL`	22	参数无效
`EMFILE`	24	打开的文件过多（超出进程限制）
`ENOSPC`	28	设备（磁盘）空间不足
`EPIPE`	32	管道已关闭，同时会产生 `SIGPIPE`
`EDEADLK`	35	可能发生死锁

errno	值（Linux/x86）	含义
`ENAMETOOLONG`	36	文件名过长
`ENOSYS`	38	功能未实现（系统调用不存在）
`ENOTEMPTY`	39	目录非空
`ELOOP`	40	符号链接层数过多
`ENOMSG`	42	没有所需类型的消息
`EIDRM`	43	标识符（IPC 对象）已被删除
`ENOTSUP`	95	操作不被支持
`ECONNREFUSED`	111	连接被拒绝（对端未监听）
`ECONNRESET`	104	连接被对端重置
`ETIMEDOUT`	110	连接超时
`EINPROGRESS`	115	操作正在进行（非阻塞连接）
`EALREADY`	114	连接已在进行中
`EADDRINUSE`	98	地址已被使用
`EADDRNOTAVAIL`	99	地址不可用
`ENOTCONN`	107	socket 未连接
`ERANGE`	34	结果超出可表示范围（如数值转换溢出）
`EDOM`	33	数学函数参数超出定义域
`EILSEQ`	84	非法的字节序列（字符编码错误）

错误处理的一般做法：

```
#include <errno.h>
#include <string.h>
#include <stdio.h>

int fd = open("nofile", O_RDONLY);
if (fd < 0) {
    fprintf(stderr, "open failed: %s (errno=%d)\n", strerror(errno), errno);
}
```

要点：

1. `errno` 只在函数返回错误时才有意义，成功时不会被清零，因此应先判断返回值。

2. 出错后应立即保存或输出 `errno`，避免被后续调用覆盖。

3. 多线程程序中 `errno` 是线程局部的，可安全使用。